

November 2024

TeLU Activation Function for Fast and Stable Deep Learning

Alfredo Fernandez
University of South Florida

Follow this and additional works at: <https://digitalcommons.usf.edu/etd>



Part of the [Artificial Intelligence and Robotics Commons](#), and the [Mathematics Commons](#)

Scholar Commons Citation

Fernandez, Alfredo, "TeLU Activation Function for Fast and Stable Deep Learning" (2024). *USF Tampa Graduate Theses and Dissertations*.
<https://digitalcommons.usf.edu/etd/10618>

This Thesis is brought to you for free and open access by the USF Graduate Theses and Dissertations at Digital Commons @ University of South Florida. It has been accepted for inclusion in USF Tampa Graduate Theses and Dissertations by an authorized administrator of Digital Commons @ University of South Florida.

TeLU Activation Function for Fast and Stable Deep Learning

by

Alfredo Fernandez

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science
Department of Computer Science and Engineering
College of Engineering
University of South Florida

Major Professor: Ankur Mali, Ph.D.
Lawrence Hall, Ph.D.
John Murray-Bruce, Ph.D.

Date of Approval:
November 7, 2024

Keywords: Neural Networks, Convergence Acceleration, Learning Stability, Computational Efficiency

Copyright © 2024, Alfredo Fernandez

Dedication

To my beloved wife, Mrs. Hannah Fernandez, whose support and encouragement made this work possible.

Acknowledgments

I would like to express my sincere gratitude to Dr. Ankur Mali, Dr. Lawrence Hall, and Dr. John Murray-Bruce for their invaluable contributions as members of my committee. I am thankful to my parents, Mr. Alfredo Fernandez and Mrs. Dalia Fernandez, for their unwavering support throughout my thesis journey, and to my sister, Ms. Virginia Fernandez, for her encouragement. Also, thank you to Dr. Reis for her expert insight and feedback in strengthening our computational efficiency theory and experimentation.

Table of Contents

List of Tables	iii
List of Figures	v
Abstract	vii
Chapter 1: Introduction	1
Chapter 2: Related Work	4
2.1 Early Activation Functions	4
2.2 Rectified Linear Units	5
2.3 ReLU Variants	5
2.4 Exponential Linear Units	6
2.5 Sigmoid Linear Units	7
2.6 Gaussian Error Linear Units	8
2.7 The Mish Activation Function	9
2.8 The Logish and Smish Activation Functions	9
Chapter 3: Motivation	12
Chapter 4: Formulation	14
4.1 Design Goals	14
4.2 Strategies	15
4.3 Design Requirements	16
4.4 Proposed Implementation	17
Chapter 5: Theoretical Framework	20
5.1 Persistent Gradients of the Saturation Region	20
5.2 Near-Linearity of Active Region	24
5.3 Runtime Efficiency	25
5.4 ReLU Compatability	27
5.5 Analytic Universal Approximation	35
5.5.1 Theoretical Benefits	41
5.5.2 Practical Benefits	42
5.6 Stability	44
Chapter 6: Experimental Validation	50
6.1 Persistent Gradients of the Saturation Region	50
6.1.1 FashionMNIST Dataset on MLP with Negative Biases	50
6.1.2 CIFAR-10 Dataset with DenseNet CNN	54
6.2 Near-Linearity of Active Region	57
6.2.1 ImageNet Dataset with ResNet18 Architecture	57
6.2.2 Text8 Dataset with Dynamic Pooling Transformer	60
6.3 Runtime Efficiency	62
6.4 ReLU Compatability	65
6.4.1 FashionMNIST Dataset with MLP with Varied Weight Decay	66

6.4.2 CIFAR-10 Dataset with SqueezeNext with Varied Optimizers	67
6.5 Analytic Universal Approximation	71
6.5.1 MNIST Dataset with Variational AutoEncoder	71
6.5.2 Penn TreeBank Dataset with RNN Architectures	72
6.5.3 FashionMNIST Dataset with MLP Robustness Experiment	73
6.6 Stability	76
6.6.1 FashionMNIST Dataset with MLP with Varied Network Depths	77
6.6.2 FashionMNIST Dataset with MLP with Varied Initialization Methods	79
6.6.3 CIFAR-100 Dataset with SqueezeNext with Varied Optimizers	79
6.6.4 Tiny ImageNet Dataset with ResNet34 with Varied Optimizers	82
6.6.5 Summary of Stability Experimental Validation	85
Chapter 7: Discussion	87
7.1 Persistent Gradients of Deactivation Region of TeLU	87
7.2 Near-Linearity of Active Region of TeLU	88
7.3 Computational Efficiency of TeLU	89
7.4 Configuration Compatability of ReLU and TeLU	89
7.5 Analytic Universal Approximation of TeLU	90
7.6 Learning and Numerical Stability of TeLU Networks	91
Chapter 8: Conclusion	92
Chapter 9: Future Work	93
References	95
Appendix A: Supplementary Tables	106
A.1 Computational Efficiency of Derivatives	106
A.2 Computation Efficiency of Derivatives	107
Appendix B: Supplementary Theorems	108
B.1 Close Approximation to ReLU Non-Linearity with TeLU	108
B.2 Zero-Centering of TeLU	110
B.3 Isolated Zero of TeLU	115
B.4 Bounded Saturation and Growth of TeLU	116
B.5 Robustness of TeLU	117
B.6 Lipschitz Continuity of TeLU	118
B.7 Smooth Loss Landscape of TeLU	119
B.8 Global Convergence of TeLU	120

List of Tables

Table 5.1.1:	Asymptotic Decay Qualifications.	23
Table 5.1.2:	Numerical Inactivity of Activation Functions.	23
Table 5.2.1:	Near-Linearity Distance Calculations.	25
Table 5.3.1:	Baseline Definitions of Activation Functions.	26
Table 5.3.2:	Computational Costs of Activation Functions.	27
Table 5.4.1:	Sub-Domain Distance of Activation Functions to ReLU.	28
Table 5.6.1:	Stability Heuristics of Activation Functions.	45
Table 6.1.1:	MLP FashionMNIST Negative-Bias Experiment Configuration.	51
Table 6.1.2:	MLP FashionMNIST Negative-Bias Test Accuracy Averaged over 10 Trials.	53
Table 6.1.3:	DenseNet CIFAR-10 Configuration.	55
Table 6.1.4:	DenseNet CIFAR-10 Accuracy Summary.	56
Table 6.2.1:	ResNet18 ImageNet Experiment Configuration.	58
Table 6.2.2:	ResNet18 ImageNet Test Accuracy Summary.	60
Table 6.2.3:	Dynamic-Pooling Transformer Text8 Experiment Configuration.	60
Table 6.2.4:	Dynamic-Pooling Transformer on Text8 Test Loss.	61
Table 6.3.1:	Timing of 10^6 Iterations of Single Neuron with Input Size of 10^6 on RTX2070.	63
Table 6.3.2:	Timing of 10^6 Iterations of Single Neuron with Input Size of 10^6 on Server GPUs.	63
Table 6.3.3:	Timing of 10^6 Iterations of Single Neuron with Input Size of 10^7	64
Table 6.3.4:	Timing of 10^7 Iterations of Single Neuron with Input Size of 10^6	64
Table 6.4.1:	MLP FashionMNIST Across Weight Decays Configuration.	66
Table 6.4.2:	SqueezeNext CIFAR-10 ReLU-Focused Configuration.	69
Table 6.4.3:	SqueezeNext CIFAR-10 Optimizer-Dependent Hyperparameters.	69
Table 6.4.4:	SqueezeNext CIFAR-10 Test Accuracy with Optimizers.	69

Table 6.5.1:	Variational AutoEncoder on MNIST Configuration.	72
Table 6.5.2:	VAE MNIST Reconstruction Error.	72
Table 6.5.3:	RNN Penn Treebank Test PPL Summary.	73
Table 6.5.4:	Elman RNN on Penn Treebank Experiment Configuration.	74
Table 6.5.5:	LSTM RNN on Penn Treebank Experiment Configuration.	74
Table 6.5.6:	MLP FashionMNIST Robustness Configuration.	75
Table 6.5.7:	MLP FashionMNIST Robustness Test Accuracy.	75
Table 6.6.1:	MLP FashionMNIST Cross-Depth Stability Experiment Configuration.	77
Table 6.6.2:	MLP FashionMNIST Cross-Initializer Stability Configuration.	79
Table 6.6.3:	SqueezeNext CIFAR-100 Optimizer-Independent Hyperparameters.	80
Table 6.6.4:	SqueezeNext CIFAR-100 Optimizer-Dependent Hyperparameters.	81
Table 6.6.5:	CIFAR-100 SqueezeNext Test Accuracy Summary.	81
Table 6.6.6:	ResNet34 TinyImageNet Optimizer-Independent Hyperparameters.	83
Table 6.6.7:	ResNet34 TinyImageNet Optimizer-Dependent Hyperparameters.	83
Table 6.6.8:	ResNet34 TinyImageNet SGD Test Accuracy.	83
Table 6.6.9:	ResNet34 TinyImageNet Momentum-Accelerated SGD Test Accuracy Summary. . . .	84
Table A.1.1:	Activation Function Derivative Computational Efficiency.	106
Table A.2.1:	Normalization Experimental Specifications.	107

List of Figures

Figure 2.2.1:	ReLU Activation Function and its First-Order Derivative.	6
Figure 2.4.1:	ELU Activation Function and its First-Order Derivative.	7
Figure 2.5.1:	SiLU Activation Function and its First-Order Derivative.	8
Figure 2.6.1:	GELU Activation Function and its First-Order Derivative.	9
Figure 2.7.1:	Mish Activation Function and its First-Order Derivative.	10
Figure 2.8.1:	Logish Activation Function and its First-Order Derivative.	11
Figure 2.8.2:	Smish Activation Function and its First-Order Derivative.	11
Figure 4.4.1:	TeLU Activation Function and its First-Order Derivative.	18
Figure 4.4.2:	Focused Set of Linear Unit Activation Functions.	19
Figure 4.4.3:	First Derivatives of Focused Set of Linear Unit Activation Functions.	19
Figure 5.1.1:	Saturation of Activation Function as Input Grows Negative.	21
Figure 5.1.2:	Quotient Comparisons of Vanishing Gradients as Input Grows Negative.	21
Figure 6.1.1:	MLP FashionMNIST Light-Bias 200-Epoch Validation Accuracy.	52
Figure 6.1.2:	MLP FashionMNIST Heavy-Bias 200-Epoch Validation Accuracy.	53
Figure 6.1.3:	MLP FashionMNIST Heavy-Bias 300-Epoch Validation Accuracy.	54
Figure 6.1.4:	DenseNet CIFAR-10 Validation Accuracy Progression.	56
Figure 6.2.1:	ResNet18 ImageNet Validation Accuracy over 90 Epochs.	58
Figure 6.2.2:	ResNet18 ImageNet Validation Accuracy over 50 Epochs.	59
Figure 6.2.3:	ResNet18 ImageNet Validation Accuracy over 20 Epochs.	59
Figure 6.2.4:	Dynamic-Pooling Transformer Text8 Validation Loss.	61
Figure 6.2.5:	Dynamic-Pooling Transformer on Text8 Validation Loss Short Experiment.	62
Figure 6.4.1:	MLP FashionMNIST Test Accuracy Across Wide Range of Weight Decays.	67
Figure 6.4.2:	MLP FashionMNIST Test Accuracy Across Focused Range of Weight Decays.	68

Figure 6.4.3:	SqueezeNext on CIFAR-10 with SGD Validation Accuracy.	70
Figure 6.4.4:	SqueezeNext on CIFAR-10 with Momentum-Accelerated SGD Validation Accuracy. .	71
Figure 6.5.1:	MLP FashionMNIST Robustness Test Accuracy.	76
Figure 6.6.1:	MLP FashionMNIST Test Accuracy Across Depths with Weight Decay 0.0005. . . .	78
Figure 6.6.2:	MLP FashionMNIST Test Accuracy Across Depths with Weight Decay of 0.001. . . .	78
Figure 6.6.3:	MLP FashionMNIST Test Accuracy Across Initializers.	80
Figure 6.6.4:	SqueezeNext on CIFAR-100 with SGD Validation Accuracy.	81
Figure 6.6.5:	SqueezeNext on CIFAR-100 with Momentum-Accelerated SGD Validation Accuracy. .	82
Figure 6.6.6:	ResNet34 TinyImageNet SGD Validation Accuracy.	84
Figure 6.6.7:	ResNet34 TinyImageNet Momentum-Accelerated SGD Validation Accuracy.	85

Abstract

We propose the Hyperbolic Tangent Exponential Linear Unit (TeLU), a neural network hidden activation function defined as $TeLU(x) = x \cdot \tanh(e^x)$. TeLU’s design is grounded in the core principles of key activation functions, achieving strong convergence by closely approximating the identity function in its active region while effectively mitigating the vanishing gradient problem in its saturating region. Its simple formulation enhances computational efficiency, leading to improvements in scalability and convergence speed. Unlike many modern activation functions, TeLU seamlessly combines the simplicity and effectiveness of ReLU with the smoothness and analytic properties essential for learning stability in deep neural networks. TeLU’s ability to mimic the behavior and optimal hyperparameter settings of ReLU, while introducing the benefits of smoothness and curvature, makes it an ideal drop-in replacement. Its analytic nature positions TeLU as a powerful universal approximator, enhancing both robustness and generalization across a multitude of experiments. We rigorously validate these claims through theoretical analysis and experimental validation, demonstrating TeLU’s performance across challenging benchmarks; including ResNet18 on ImageNet, Dynamic-Pooling Transformers on Text8, and Recurrent Neural Networks (RNNs) on the Penn TreeBank dataset. These results highlight TeLU’s potential to set a new standard in activation functions, driving more efficient and stable learning in deep neural networks, thereby accelerating scientific discoveries across various fields.

Chapter 1:

Introduction

The rapid advancements in deep learning have significantly expanded the capabilities of machine learning, enabling machines to perform tasks ranging from image recognition to natural language processing with remarkable accuracy [61, 65, 109]. Central to these advancements are neural networks, computational models inspired by the human brain, which consist of layers of interconnected neurons. Each neuron applies a mathematical operation to its inputs and passes the result to the next layer, enabling the network to learn and generalize from data [95]. As neural networks have grown in complexity and depth, their success has increasingly depended on the subtle choices made in their architecture, including the selection of activation functions [33].

Activation functions play a pivotal role in neural networks by introducing non-linearity into the model, allowing it to learn complex patterns and representations that would be impossible for a purely linear model [101]. Traditional activation functions such as the logistic sigmoid [95], hyperbolic tangent ($\tanh$), and rectified linear unit (ReLU) [29] have been foundational in the growth of deep learning. However, despite their widespread use, these functions come with significant limitations. Sigmoid and Tanh functions are prone to vanishing gradient problems, particularly in deep networks, which can stall the learning process [10, 48]. ReLU, while exhibiting improved convergence due to its linear active region, is prone to dead neurons — where units stop contributing to the learning process due to negative inputs being zeroed out [76, 115].

Modern activation functions, such as the Exponential Linear Unit (ELU) [16], mitigate the dying neuron issue and reduce the forward bias shift effects present in positive semi-definite non-linearities such as ReLU by approaching an odd symmetry near the origin. The utilization of negative activation pushes the expected activation closer to 0, resulting in effects that resemble batch normalization and thereby speeds up learning [1]. The Gaussian Error Linear Unit (GELU) [45] uses its non-monotonic growth to deliver a comparable symmetry near the origin with the added benefit of having a deactivation that saturates to zero. This gradual deactivation functions like a deterministic Gaussian form of standard neuron dropout [105], where neurons selectively participate based on the presence of features rather than their absence, enhancing the model’s robustness [114].

Despite these innovative developments, ReLU remains the most popular general-purpose nonlinearity, largely due to its simplicity and rapid convergence [34, 86]. We believe that ReLU’s widespread use in feed-forward neural networks has established it as the standard benchmark against which newer activation functions are compared. However, these newer functions often lack ReLU’s computational efficiency and effective gradient propagation [23]. We also observed that recently proposed activation functions typically do not perform optimally under the same hyperparameter settings as ReLU [111]. As a result, these modern nonlinearities can appear as inefficient alternatives that do not offer consistent improvements over ReLU.

Driven by this hypothesis, we set out to develop an activation function that not only preserves the enduring strengths of ReLU but also introduces innovations that exceed the benefits provided by modern activation functions. Recognizing the robustness and optimizer compatibility of smooth functions [79, 114], we focused on discovering an analytic non-linearity [14]. In this thesis, we present the Hyperbolic Tangent Exponential Linear Unit (TeLU), an activation function that retains ReLU’s rapid convergence while addressing key challenges such as the vanishing gradient problem and learning instability. TeLU also enhances robustness and provides greater stability during learning, making it a powerful and versatile tool for advancing deep neural networks.

We validate these claims through extensive empirical evaluations across various benchmark datasets and architectures. We demonstrate that TeLU outperforms traditional activation functions in terms of both training efficiency and final model accuracy. Moreover, our analysis reveals that this new activation function offers a more robust and stable training process, particularly in deep and complex networks. These findings suggest that TeLU could be a valuable addition to the toolkit of deep learning practitioners, paving the way for more powerful and effective neural network models.

The contributions of this thesis are as follows:

- *Proposal of the Hyperbolic Tangent Exponential Linear Unit (TeLU):* An artificial neuron activation function that integrates and optimizes the beneficial characteristics of previous nonlinearities.
- *Comprehensive Theoretical Analysis:* A detailed examination and comparison of popular activation functions’ near-linear behavior in the active region, persistent gradient in the saturation region, runtime efficiency, universal approximation, and stability properties that theoretically demonstrate TeLU’s advantages.
- *Extensive Experimental Evaluation:* A wide-ranging set of experimental validations of TeLU’s distinctive theoretical advantages conducted on Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), dynamic-pooling transformer, Recurrent Neural Network (RNN), and Variational AutoEncoder (VAE) architectures using competitive datasets such as ImageNet and Text8.

Chapter 2 explores related activation functions, detailing the sequence of innovations in activation functions. Chapter 3 discusses the limitations of existing activation functions, highlighting the need for an alternative solution. Chapter 4 outlines our design process and introduces the formulation of TeLU. Chapter 5 delves into the development of TeLU and provides the theoretical basis for its advantages. Section 5.1 examines the vanishing gradients of popular activation functions for increasingly negative inputs and describes how TeLU mitigates this issue. Section 5.2 explains how TeLU approximates the identity function for positive inputs, resulting in faster convergence compared to other activation functions. Section 5.3 demonstrates the computational efficiency of TeLU’s simple formulation. Section 5.4 discusses TeLU’s operational similarities to ReLU, enabling compatibility with existing ReLU projects. Section 5.5 proves that TeLU architectures are analytic universal approximators, outlining the corresponding stability and robustness benefits. Finally, Section 5.6 addresses learning stability, drawing on heuristics from literature to illustrate how TeLU enhances stability in deep neural network training.

Chapter 6 presents experimental validation of the theoretical results discussed in Chapter 5. Section 6.1 shows that TeLU’s mitigation of the vanishing gradient problem is crucial for recovering from strong vanishing gradient conditions as well as improving accuracy in convolutional neural networks. Section 6.2 provides extensive evidence of TeLU’s enhanced convergence properties using the ImageNet dataset with the ResNet34 architecture and the Text8 dataset with the Dynamic-Pooling Transformer architecture. Section 6.3 benchmarks the runtime of popular activation functions, illustrating TeLU’s computational efficiency across various systems. Section 6.4 highlights the tuning similarities between TeLU and ReLU, demonstrating that configurations optimized for ReLU yield superior accuracy when applied to TeLU. Section 6.5 showcases the benefits of TeLU’s analytic universal approximation in Variational AutoEncoders (VAEs), Recurrent Neural Networks (RNNs), and robustness benchmarks. Finally, Section 6.6 demonstrates TeLU’s unmatched learning stability across diverse architectural variations, initialization methods, and optimizers.

Chapter 6 provides a comprehensive summary of the value of the TeLU activation function, combining both theoretical insights and experimental findings to illustrate its advantages in neural network training. This chapter delves into how TeLU addresses challenges such as improved convergence, vanishing gradient mitigation, computational efficiency, and compatibility with existing training configurations, highlighting its practical implications across various architectures. Building on this, Chapter 7 concludes the research by revisiting the main objectives and contributions, emphasizing TeLU’s role as an effective drop-in replacement for ReLU. Finally, Chapter 8 looks forward to future research directions, proposing efforts to expand theoretical guarantees and conduct further experimental validations. It suggests exploring TeLU’s performance in more diverse architectures and refining its mathematical properties, setting the stage for ongoing advancements in the deep learning community.

Chapter 2:

Related Work

This chapter provides an abridged history of activation function development, starting with the early models like the step and sigmoid functions, which paved the way for more sophisticated approaches. It explores how innovations such as the Rectified Linear Unit (ReLU) overcame key limitations of previous models, improved training efficiency and convergence in deep networks. Subsequently, we will examine more recent advancements, such as the Exponential Linear Unit (ELU), Sigmoid Linear Unit (SiLU), and Gaussian Error Linear Unit (GELU), which further addressed issues like dead neurons and output bias with smooth nonlinearities. Each innovation reflects a progression in the understanding of how activation functions can enhance neural network learning dynamics. By evaluating these historical developments and their contributions, we gain valuable insight into the broader landscape of activation functions and their critical role in driving the ongoing advancements in deep learning.

2.1 Early Activation Functions

In neuroscience, it was once believed that biological neurons activated according to the Heaviside unit step function [81], which is either active with output 1 or inactive with output 0. As a differentiable approximation to the Heaviside function, the Logistic Sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ [95] became prevalent in early neural networks, offering a practical activation function capable of universal approximation [19, 30, 49]. However, the Logistic Sigmoid function suffers from the vanishing gradient problem as pre-activations approach $\pm\infty$ and activations saturate toward 0 or 1. The vanishing gradient problem can significantly hinder the training of deep architectures. During backpropagation, the small gradients from upstream neurons shrink the gradients of downstream activations due to the chain rule in calculus [96]. Consequently, parameter updates in the earlier layers become negligible when enough upstream neurons have gradients that approach zero.

The Hyperbolic Tangent ($\tanh$) activation function, defined as $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, helps mitigate the vanishing gradient problem by providing larger gradients than the Logistic function for pre-activation values around 0. This effect is nearly guaranteed in practice when appropriate L1 or L2 weight regularization is used. Additionally, the Hyperbolic Tangent is symmetric about the origin, resulting in an average activation

value closer to 0. This symmetry leads to improved convergence efficiency compared to positive functions like the Logistic Sigmoid [66, 67, 89, 97].

2.2 Rectified Linear Units

The Rectified Linear Unit (ReLU), defined as $ReLU(x) = \max(0, x)$ and shown in figure 2.2.1, was originally discovered by Fukushima [29]. It was later recognized for its benefits in accelerating convergence due to its computational efficiency, near-linearity, and reduced vanishing gradient problem [34]. ReLU’s simplicity in computation and differentiation leads to faster training times and improved scalability of neural architectures. Its linear behavior for positive inputs provides strong gradients and a faster convergence rate, though this comes with the trade-off of significant output bias [66].

In the negative domain, ReLU neurons are inactive, resulting in sparse activations that help reduce overfitting. This effectively acts as a form of built-in deterministic dropout regularization [105]. However, this inactivity can also lead to a permanent reduction in the network’s representational capacity, a phenomenon known as the “dying ReLU” problem. Research by Lu et al. [74] shows that the likelihood of a ReLU network experiencing neuron inactivity increases with network depth and decreases with network width. This makes deep ReLU architectures particularly challenging to train unless width is scaled accordingly, which in turn increases computational costs [100]. Additionally, the piecewise linear nature of ReLU introduces robustness challenges [36]: Input perturbations can deactivate essential neurons or activate previously inactive neurons that did not participate in training, potentially compromising the stability and performance of the model.

2.3 ReLU Variants

Despite its drawbacks, ReLU remains a popular default choice for hidden activation functions and has inspired numerous variants. The Leaky ReLU (LReLU) [76], defined as $LReLU(x) = \max(0.01x, x)$, addresses the issue of inactive neurons by assigning a small constant slope to negative inputs. Similarly, Parametric ReLU (PReLU) [42], $PReLU(x) = \max(\alpha x, x)$, treats the slope of negative inputs as learnable parameter α . While introducing additional learnable parameters in PReLU can increase the risk of overfitting, this can be managed with aggressive regularization and data augmentation techniques.

Random Leaky ReLU (RRReLU) [116], $RRReLU(x) = \max(\beta x, x)$, where $\beta \sim \mathcal{U}(l, u)$ and $l, u \in [0, 1)$ avoids the overhead of additional overfitting prevention by randomly setting the slope of the negative region using a uniform distribution over small positive values. This randomness introduces an implicit form of regularization, allowing RRReLU to achieve lower training accuracies than ReLU, LReLU, and PReLU. These variants trade ReLU’s deactivation region for unbounded behavior in an attempt to avoid the dying

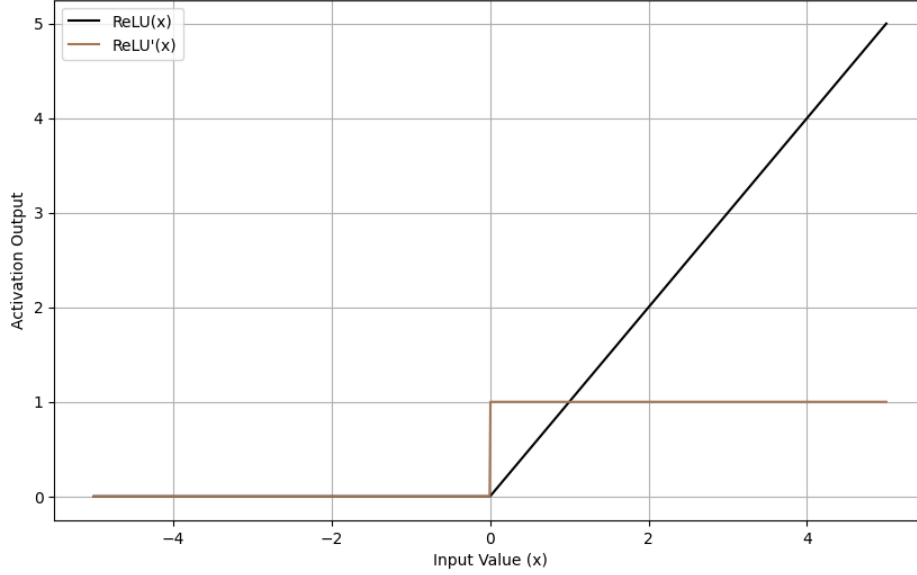


Figure 2.2.1: ReLU Activation Function and its First-Order Derivative. Plot of $\text{ReLU}(x) = \max(0, x) = \{x < 0 : 0, x\}$ and its first order derivative. For negative inputs, ReLU is 0. For positive inputs, ReLU is defined as the identity x .

neuron problem. However, piecewise linear functions like these have been found to exhibit poor robustness [114], making them susceptible to adversarial noise. As a result, smooth approximations of ReLU, such as $\text{Softplus}(x) = \ln(1 + e^x)$, may be more suitable for building secure applications.

2.4 Exponential Linear Units

Smooth nonlinearities also offer a significant advantage in their compatibility with second-order optimization algorithms like Natural Gradient Descent [70, 104]. Unlike first-order optimizers, which rely only on the gradient of the error function, second-order optimizers utilize the Hessian matrix, providing curvature information about the error surface. This additional information allows for more precise updates to model parameters, resulting in faster and more efficient convergence during training [96].

The Exponential Linear Unit (ELU) [16] attempts to approach the Natural Gradient [104] in learning without the use of second-order optimizers by reducing the output bias commonly found in popular activation functions. This reduction in output bias results in a Fisher information matrix [27] with small off-diagonal entries [89]. As a result, ELU helps bring neural networks closer to Fisher Optimal Learning [1], where learning is guided by the natural gradient for improved convergence [17, 67, 97]. ELU is defined as the identity function for positive inputs, preserving the rapid convergence of the Rectified Linear Unit (ReLU). For negative inputs, ELU is defined as $\text{ELU}(x) = e^x - 1$, producing a small negative value instead of zeroing

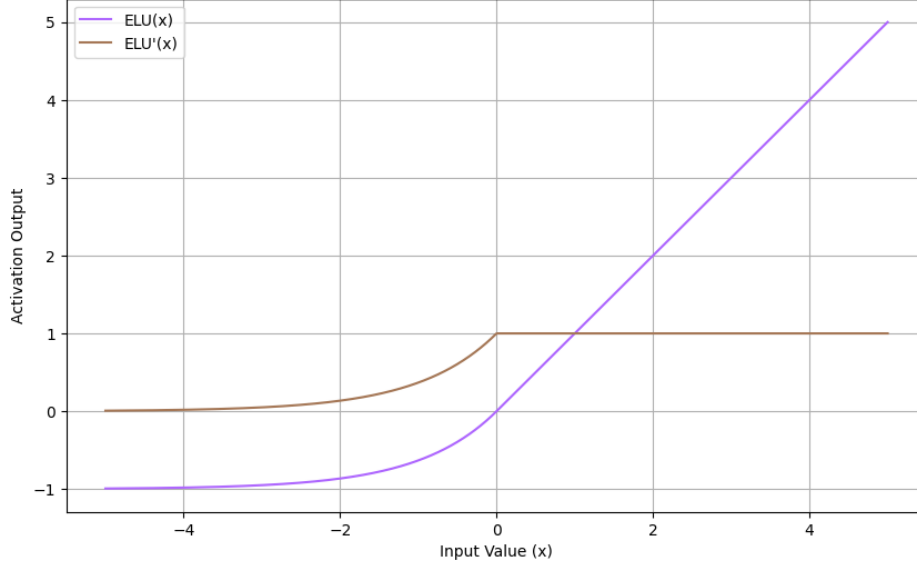


Figure 2.4.1: ELU Activation Function and its First-Order Derivative. Plot of $ELU(x) = x < 0 : e^x - 1, x$ and its first order derivative. For negative inputs, ELU is defined by $e^x - 1$ and thus saturates to an output of -1 as $x \rightarrow -\infty$. For positive inputs, ELU is defined as the identity x .

out the activation. Consequently, ELU saturates toward -1 rather than 0 as preactivation grows negative, meaning that ELU neurons are excluded from inference only when their preactivation is close to 0. This design sacrifices ReLU’s sparsity and deactivation in favor of capturing negative features. Although ELU has a continuous first derivative, its second derivative is not continuous, making it unsuitable for second-order optimization methods.

2.5 Sigmoid Linear Units

Unlike ELU, smooth non-monotonic ReLU approximations like SiLU retain a dense deactivation region while offering the advantage of continuous, non-zero differentiability. The Sigmoid Linear Unit (SiLU)[91], also referred to as the Swish activation function, was discovered through Neural Architecture Search (NAS) [13]. Mathematically, SiLU is defined as $SiLU(x) = x \cdot \sigma(x)$, where $\sigma(x) = \frac{1}{1+e^{-x}}$. Its discovery involved constructing trees from univariate and bivariate functions, with a focus on balancing computational efficiency and expressivity. During the search, it was observed that simpler functions which could be generalized as linearly-scaled cumulative distribution functions (LSCDFs), were often favored. SiLU’s smooth, non-monotonic nature results in smoother optimization landscapes, aiding in the effective training of deeper networks by avoiding issues like sharp transitions and local minima. SiLU’s seminal research [90] has shown that SiLU outperforms traditional activation functions, such as ReLU and Leaky ReLU, across various

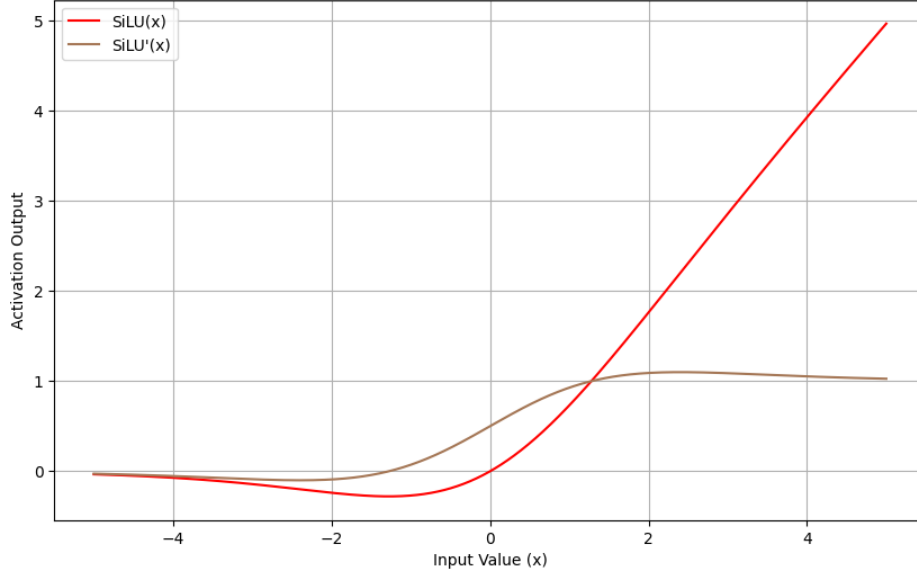


Figure 2.5.1: SiLU Activation Function and its First-Order Derivative. Plot of $\text{SiLU}(x) = x \cdot \frac{1}{1+e^{-x}}$ and its first order derivative. As input $x \rightarrow -\infty$, SiLU saturates to a deactivation output of 0. As input $x \rightarrow \infty$, SiLU’s active region approximates the identity x .

tasks. These findings sparked the growing recognition of smooth, non-monotonic activation functions in deep learning [45, 83].

2.6 Gaussian Error Linear Units

Hendrycks and Gimpel recognized the mathematical similarities between the inactive regions of LSCDFs, such as ReLU and SiLU, and the effects of dropout regularization [45]. The formulation of GELU was a reinterpretation of the SiLU activation function, where a scaled error function was employed in place of the logistic cumulative distribution function found within SiLU. The result, $\frac{x}{2} \cdot [1 + \text{erf}(x/\sqrt{2})]$ theoretically allows GELU to leverage soft pseudo-ensembles [6], where negative activations are neglected and positive activations are preserved. Pseudo-ensembles refer to subsets of a neural network that temporarily become inactive, enabling network subgraphs to learn to perform tasks independently like traditional ensemble methods [31]. This approach tends to enhance a model’s robustness by benefiting from the many-advisors principle, common in ensemble machine learning models. Supporting this strategy, the non-monotonic GELU activation function has been shown to outperform both ReLU and ELU in the MNIST [22], CIFAR-10, and CIFAR-100 [60] datasets.

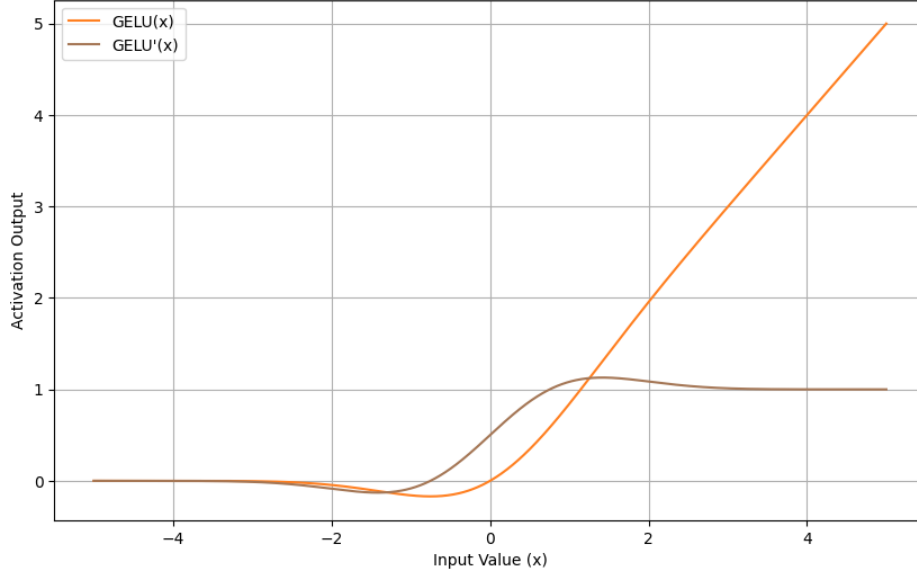


Figure 2.6.1: GELU Activation Function and its First-Order Derivative. Plot of $GELU(x) = \frac{x}{2} \cdot (1 + \operatorname{erf}(\frac{x}{\sqrt{2}}))$ and its first order derivative. As input $x \rightarrow -\infty$, GELU saturates to a deactivation output of 0. As input $x \rightarrow \infty$, GELU’s active region approximates the identity x .

2.7 The Mish Activation Function

The Mish activation function [83], defined as $Mish(x) = x \cdot \tanh(\ln(1 + e^x))$, introduces a new LSCDF that is similar to SiLU and GELU. Mish was found while observing the causes of the beneficial SiLU properties, and was validated against other competing formulations with preliminary experimentation. The cumulative distribution function used, $\tanh(\ln(1 + e^x))$, allows the Mish non-linearity to exhibit self-regularization and thus requires less external regularization by other means. This design choice allows Mish to demonstrate improvements over ReLU and SiLU, particularly in terms of stability when dealing with network depth and weight initialization strategies. The function also exhibits greater robustness to Gaussian noise than ReLU and SiLU in its introductory experiments [83].

2.8 The Logish and Smish Activation Functions

Similarly to Mish, the Logish function, $Logish(x) = x \cdot \ln(1 + \frac{1}{1+e^{-x}})$ [120], achieves non-monotonic non-linearity with a self-regularizing effect, uniquely approaching a slope of $\ln(2)$ as the input grows. The Smish function, $Smish(x) = x \cdot \tanh(\ln(1 + \frac{1}{1+e^{-x}}))$ [111], further evolves Logish by applying the hyperbolic tangent to its smooth binary function, creating a new non-linearity that tends towards a slope of 0.6 as the input increases, though at a higher computational cost. The impact of slope reductions in Logish and Smish is still an area of active investigation, but early experimental results from their respective foundational studies

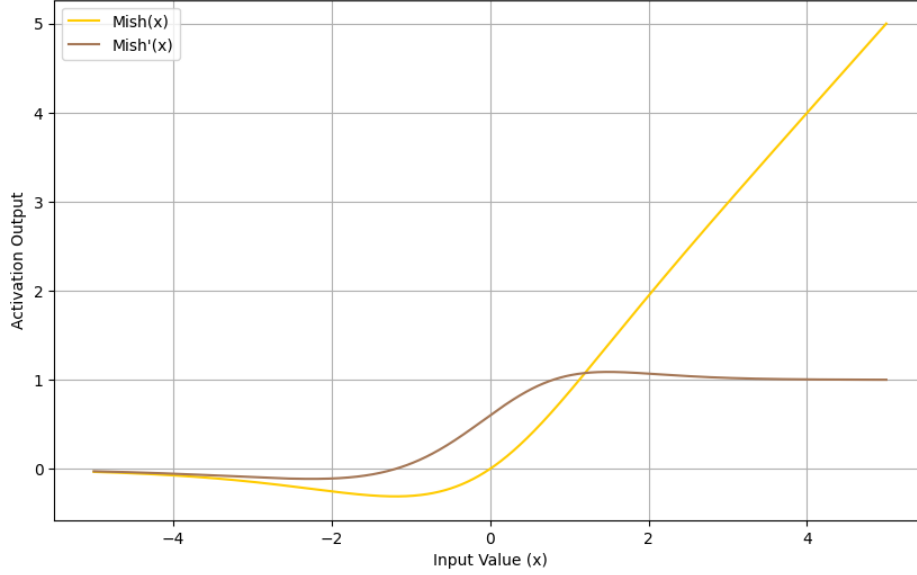


Figure 2.7.1: Mish Activation Function and its First-Order Derivative. Plot of $Mish(x) = x \cdot \tanh(\ln(1 + e^x))$ and its first order derivative. As input $x \rightarrow -\infty$, Mish saturates to a deactivation output of 0. As input $x \rightarrow \infty$, Mish’s active region approximates the identity x .

indicate that both functions exhibit greater stability compared to widely used activation functions like ReLU, SiLU, and Mish. This is particularly evident on the CIFAR10 dataset, even under conditions where minimal L2 regularization is applied.

Notably, the experiments demonstrating Smish’s enhanced stability and performance were conducted without utilizing weight decay regularization. This absence of external regularization methods suggests that Smish possesses inherent self-regularization properties, achieving high accuracy without reliance on explicit regularization techniques. Consequently, Smish could be particularly advantageous in scenarios where stability and self-regularization are critical, but computational efficiency is negligible.

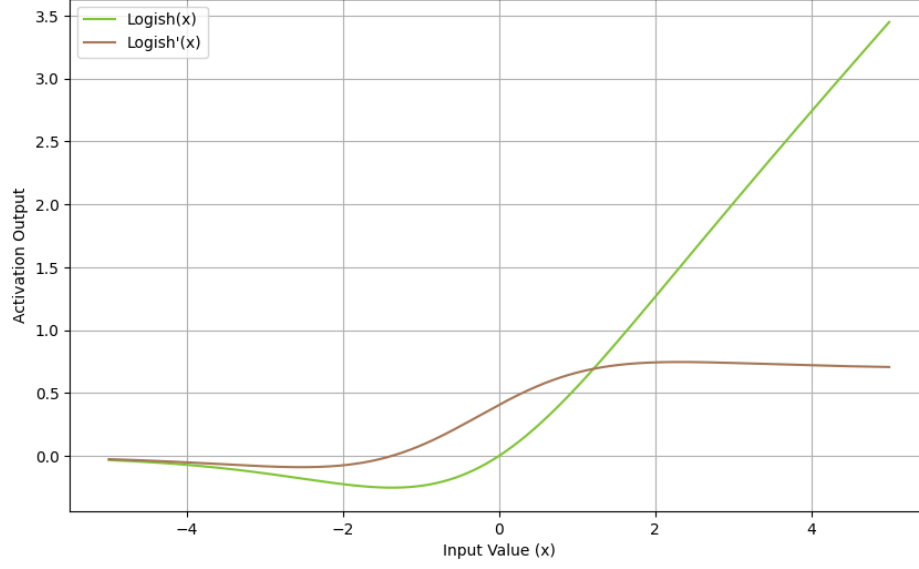


Figure 2.8.1: Logish Activation Function and its First-Order Derivative. Plot of $Logish(x) = x \cdot \ln(1 + \frac{1}{1+e^{-x}})$ and its first order derivative. As input $x \rightarrow -\infty$, Logish saturates to a deactivation output of 0. As input $x \rightarrow \infty$, Logish's active region approximates a linear growth with a slope of $\ln(2)$.

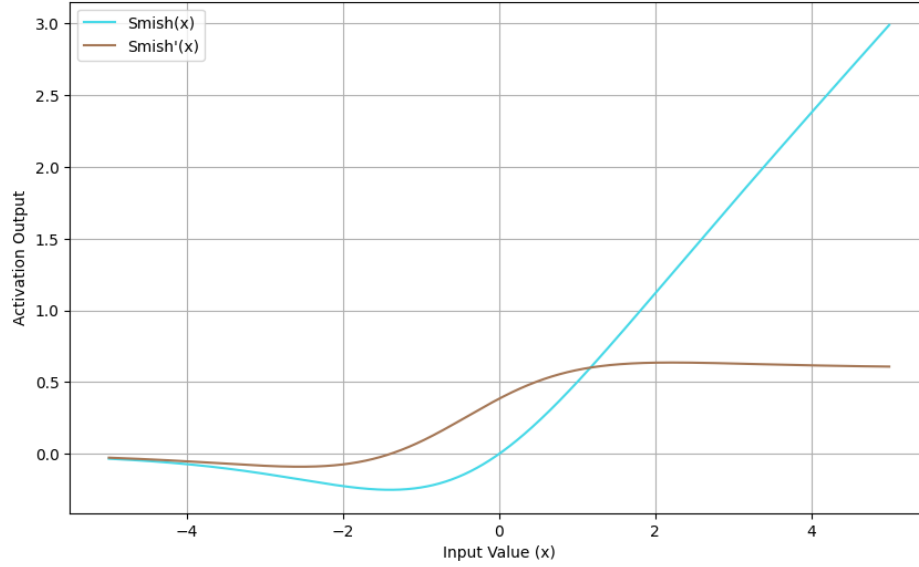


Figure 2.8.2: Smish Activation Function and its First-Order Derivative. Plot of $Smish(x) = x \cdot \tanh(\ln(1 + \frac{1}{1+e^{-x}}))$ and its first order derivative. As input $x \rightarrow -\infty$, Smish saturates to a deactivation output of 0. As input $x \rightarrow \infty$, Smish's active region approximates a linear growth with a slope of 0.6.

Chapter 3:

Motivation

Despite substantial progress in deep learning, the choice of activation functions has remained relatively unchanged, with ReLU continuing to dominate due to its computational efficiency and effective gradient propagation in the active region [11, 33, 34, 61]. While ReLU’s simple formulation leads to rapid convergence in shallow networks, its limitations become apparent in deeper architectures. The well-known “dying ReLU” problem can cause significant capacity loss, hindering optimization and reducing overall model expressivity [115]. This issue becomes more pronounced as networks scale in depth and complexity, where maintaining stable gradient flow is critical for effective training [74].

To address these issues, various alternatives such as ELU [16], SiLU [90], GELU [45], and Mish [83] have been proposed, each introducing modifications to smoothen activation transitions and reduce dead neurons. For example, ELU’s negative saturation encourages zero-centered activations, improving convergence stability. Similarly, non-monotonic functions like SiLU and GELU provide smoother gradients, preventing neuron saturation and improving gradient flow [93]. However, these designs often introduce increased computational complexity, and empirical evaluations have shown that their advantages are not universally consistent across diverse architectures and datasets [12, 23, 25, 33, 108].

Recent studies suggest that an ideal activation function should balance strong gradient propagation, zero-mean activations, smooth transitions, and computational efficiency [34, 50, 90]. By incorporating negative activations, non-monotonic functions typically result in a more symmetrical distribution of activations around zero, which helps to accelerate convergence [16, 23]. However, functions such as GELU and Mish, while possessing these desirable properties, are computationally demanding and produce weaker gradients, which slows down the learning process [33]. This disconnect between theoretical advantages and practical performance suggests that there is still no single activation function that can effectively unify the strengths of existing methods.

This fragmentation in the design space motivates the need for an activation function that combines the strengths of ReLU’s learning efficiency [34] with the stability and generalization capabilities of smooth functions [93]. Our proposed activation function addresses these challenges by integrating smooth transitions, near-zero-mean activations, and robust gradient dynamics, enabling it to achieve consistent performance

across a wide range of tasks. Supported by strong theoretical and empirical findings, our function is designed to overcome the limitations of current activations, leading to better convergence properties, enhanced stability, and improved generalization across both shallow and deep architectures.

By building on the strengths of its predecessors and addressing their individual weaknesses, our activation function establishes a new benchmark in activation design. It not only accelerates training and reduces computational overhead but also provides a more stable optimization landscape for large-scale models. This approach offers a unified solution that can set a new standard for activation functions, enabling more reliable and efficient learning in complex deep learning scenarios.

Chapter 4:

Formulation

4.1 Design Goals

Our primary design goal for a hidden activation function is to provide neural network architectures with enough expressivity to approximate any unknown target function defined by the task at hand. Assuming these target functions are continuous, we can theoretically reduce this goal to achieving a universal approximator [19]. Alongside expressivity, we seek an activation function that is computationally efficient for both forward and backward passes, reducing hardware demands and the duration of each training epoch. Additionally, we aim for an activation function that promotes fast convergence, lowering the number of epochs required for models to achieve success on hidden data. Combined, these requirements ensure training efficiency by minimizing both the time and number of epochs necessary to reduce loss on the training data.

In addition to training efficiency, maintaining training stability is crucial [40]. This means that models using our activation function should achieve consistently favorable evaluations on both training and unseen data, even with slight variations in model and training configurations. Model configuration includes architectural choices like depth, layer width, and the types of layers used, while training configuration covers optimization-related decisions such as the choice of algorithm, learning rate, and weight decay coefficient. A particularly harmful form of instability occurs when a model’s parameters lead to numerically unstable outputs [46]. When a neuron’s output overflows or underflows, it can cause the entire model to infer incorrectly, resulting in performance that barely exceeds random chance. Therefore, we aim to design an activation function that minimizes these instances of numerical instability, ensuring favorable and reliable training outcomes.

Our final set of goals focus on ensuring that the model’s success on training data translates effectively to unseen data. To achieve this, we need to design an activation function that promotes strong generalization, allowing a neural network to maintain its performance beyond the training set. Traditionally, overfitting is managed at the model and optimization level by reducing model capacity [35] or using regularization techniques like dropout [105], weight decay [95], and batch normalization [53]. It is well-known that combining various forms of regularization often leads to better generalization [37, 105, 117]. With this in mind, we aim to incorporate a modest yet meaningful level of self-regularization directly within our activation function,

operating under the hypothesis that it could reduce the reliance on traditional methods. However, we must be cautious; introducing too much self-regularization could inadvertently limit the model’s capacity. In addition, we seek to develop a robust activation function that enhances the model’s resilience, enabling it to perform reliably even when exposed to data with small perturbations.

4.2 Strategies

Leshno et al. [69] establish that a function can serve as a universal approximator if and only if it is continuous and non-polynomial. Thus, to satisfy our goal of creating a universal approximator, we must focus on developing a non-polynomial function. For computational efficiency, the activation function must also have a simple formulation with an easily differentiable first-order derivative. As demonstrated by *ReLU*, rapid convergence can be achieved through a straightforward function that maintains strong gradients in the active region, corresponding to positive outputs [34]. To fulfill this rapid convergence requirement, we can employ a linear unit that acts like the identity for positive inputs. However, we should avoid growth that exceeds the identity, as this could lead to exploding gradients and introduce numerical instability into the model. By maintaining this identity-like behavior for positive inputs, we reduce the risk of vanishing gradients in the active region. Additionally, it is crucial to sustain nonzero gradients in the saturation region of our linear unit, helping to prevent subgraphs in the neural network from stalling learning—a common issue in *ReLU* architectures [74].

To enhance learning stability in neural networks, we first analyze the strategy behind ELU’s formulation [16]. ELU uses approximate odd symmetry near the origin, producing negative outputs for all negative inputs. This design enables the saturation region to counterbalance the linear region, pulling the expected output closer to zero for mean-zero inputs [66]. ELU achieves this by avoiding deactivation for negative inputs, adopting a piecewise definition that saturates to -1 as inputs approach $-\infty$. We hypothesize that an activation function with a single definition and gradual deactivation could further improve stability. Non-monotonic activations often exhibit odd symmetry near zero, reducing output bias for input distributions with small standard deviations, as seen with standard normalization techniques [39, 63]. Additionally, minimizing nonlinear terms may reduce numerical instability by limiting the chances of underflow or overflow. Thus, we aim for a simple formulation that aligns with our computational efficiency goals.

Recognizing that generalization is enhanced by combining various regularization techniques [37, 105, 117], we aim to develop a self-regularizing activation function. Activation functions like GELU [45], Mish [83], Logish [120], and Smish [111] are notable for their inherent self-regularization properties. For example, Hendrycks and Gimpel [45] designed GELU as the expected value of a stochastic regularizer, while

the self-regularization in Mish, Logish, and Smish arises from additional terms in their formulations. This observation leads us to hypothesize that a new activation function with implicit regularization and enhanced generalization is likely to belong to the class of smooth, non-monotonic nonlinearities [83, 111, 120]. Furthermore, Rosca et al. [93] emphasize the critical role of smooth activation functions in improving both generalization and uncertainty estimates. By pursuing a smooth, non-monotonic activation function, we aim to harness the combined benefits of smoothness and implicit self-regularization.

Beyond traditional generalization, we aim for models using our activation function to also perform well on unseen data corrupted by small perturbations [107]. Xie et al. [114] have demonstrated that smooth activation functions like GELU and SiLU offer both accuracy and robustness advantages over piecewise linear activation functions. This inherent smoothness, characterized by differentiability at every point, is a defining feature of analytic functions [14]. An analytic function can be expressed as a convergent power series, which involves the representation of infinitely many derivatives. Analytic functions are also compatible with higher-order optimization techniques, such as Natural Gradient Descent [104], which utilize the curvature of the loss function rather than just its value, leading to more stable learning with fewer steps until convergence [9, 79, 80, 96]. However, as demonstrated by ELU, it is possible to approach the benefits of second-order optimization techniques without their direct use by incorporating a non-linearity that exhibits a degree of odd symmetry around the origin. This feature helps reduce forward output bias, thereby enhancing the learning efficiency by approach Fisher optimal learning [1, 17, 66, 67, 97].

4.3 Design Requirements

Based on our developed strategies, the search for an activation function will be directed by the following mathematical requirements:

- Should approximate linearity as inputs approach ∞
- Must be analytic, or infinitely differentiable at all points, without collapsing to zero
- Should exhibit near odd symmetry about $y = -x$ near the origin
- Must include a dense inactive region that saturates towards zero
- Should demonstrate minimal gradient decay as the output saturates towards zero
- Should have low computational complexity
- Must be non-polynomial to qualify as a universal approximator

In practice, we discovered that combining conflicting heuristics can lead to a more focused mathematical investigation. Thus, we present our goal summary in the form of the following dichotomies:

- The non-linearity must be nearly linear in the active region while remaining analytic. This ensures a convergence rate similar to ReLU and the identity, while being infinitely differentiable at all points to enhance stability and robustness.
- The nonlinearity should activate with both positive and negative values while having a dense inactive region. This allows the gradient of the unit to align with the natural gradient, while still retaining the ability to deactivate or drop out a neuron during training.
- The nonlinearity must asymptotically decrease towards 0 as inputs become negative, but should minimize the rate at which gradients approach 0 in the inactive region. This gives the optimizer the flexibility to deactivate a neuron without permanently losing the option to reactivate it later.
- The nonlinearity should be definable with minimal mathematical complexity without being polynomial or suffering from exploding gradients. This results in a computationally efficient universal approximator that can be trained with stability.

Additionally, the new non-linearity must perform competitively with ReLU when used in configurations optimized for ReLU-based architectures. This practical requirement ensures that the new activation function can serve as a viable alternative to ReLU and be seamlessly integrated into existing deep learning projects.

4.4 Proposed Implementation

After developing several unstable activation functions, we began closely analyzing the Mish activation function [83]. Mish, defined as $Mish(x) = x \cdot \tanh(\ln(1 + e^x))$, was noted in its foundational research for offering impressive stability and robustness compared to earlier functions. However, its drawbacks, namely computational inefficiency and damped gradients, prevent it from surpassing ReLU in convergence efficiency in practice [33, 34, 61]. Recognizing Mish’s potential, we identified that by removing its logarithmic term and the associated unit addition, we could significantly enhance its computational efficiency and gradient strength. This led to the creation of the Hyperbolic Tangent Exponential Linear Unit, $TeLU(x) = x \cdot \tanh(e^x)$, which demonstrated superior convergence efficiency [26]. According to the taxonomy proposed by Apicella et al. [3], TeLU is a classic, fixed-shape activation function.

We found that our mathematical and experimental requirements are best met by TeLU, shown in Figure 4.4.1. TeLU combines the linear properties of traditional activation functions with the nonlinear advantages

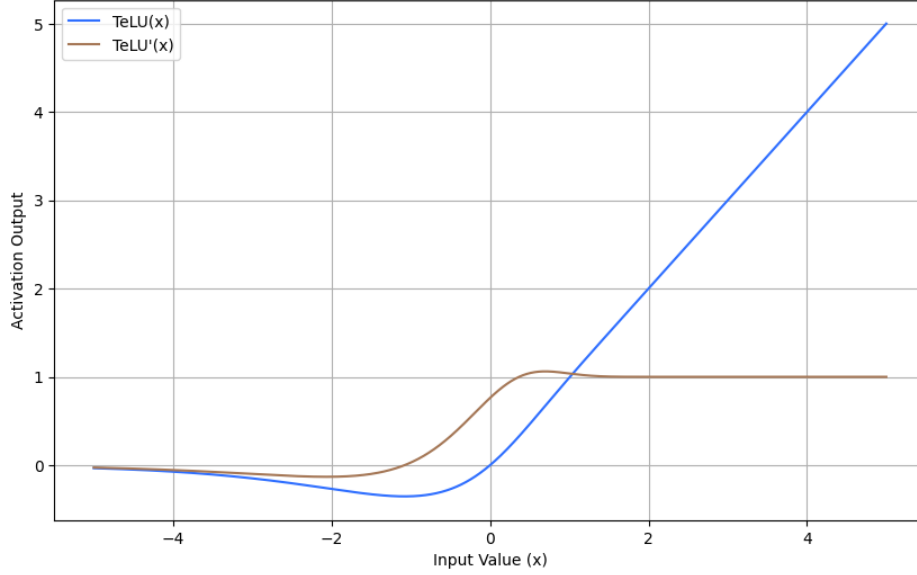


Figure 4.4.1: TeLU Activation Function and its First-Order Derivative. Plot of $TeLU(x) = x \cdot \tanh(e^x)$ and its first order derivative. As input $x \rightarrow -\infty$, TeLU slowly saturates to a deactivation output of 0. As input $x \rightarrow \infty$, TeLU’s active region quickly approximates the identity x .

of exponential and hyperbolic tangent functions. This blend strikes a balance between promoting efficient learning and avoiding exploding gradients. TeLU’s design, utilizing the hyperbolic tangent of the exponential function, naturally moderates the output, keeping it within a manageable range. Unlike earlier activation functions, TeLU provides a smooth transition across the origin, enhancing gradient flow and contributing to more stable and consistent learning dynamics [23, 93]. Figure 4.4.2 illustrates this behavior, highlighting TeLU’s continuity and slower saturation compared to other state-of-the-art functions.

Our formulation is in line with the successful heuristics identified in the SiLU Neural Architecture Search [90], which emphasizes the effectiveness of linearly-scaled cumulative distribution functions with minimal terms. This category includes activation functions such as ReLU, SiLU, GELU, and Mish, which are all known for their reliable performance as validated by independent research [23, 106]. The similarity of TeLU to both GELU and SiLU suggests that it may share their robustness properties, as indicated in studies on smooth activation functions [93, 114]. This consistency with existing literature is essential, as it strengthens the potential of TeLU to achieve comparable levels of success. The alignment with established results not only supports the validity of TeLU’s design but also increases the likelihood of its adoption in diverse deep learning applications. To illustrate, Figure 4.4.2 presents a comparison of the main activation functions discussed in this paper, while Figure 4.4.3 displays their corresponding first-order derivatives.

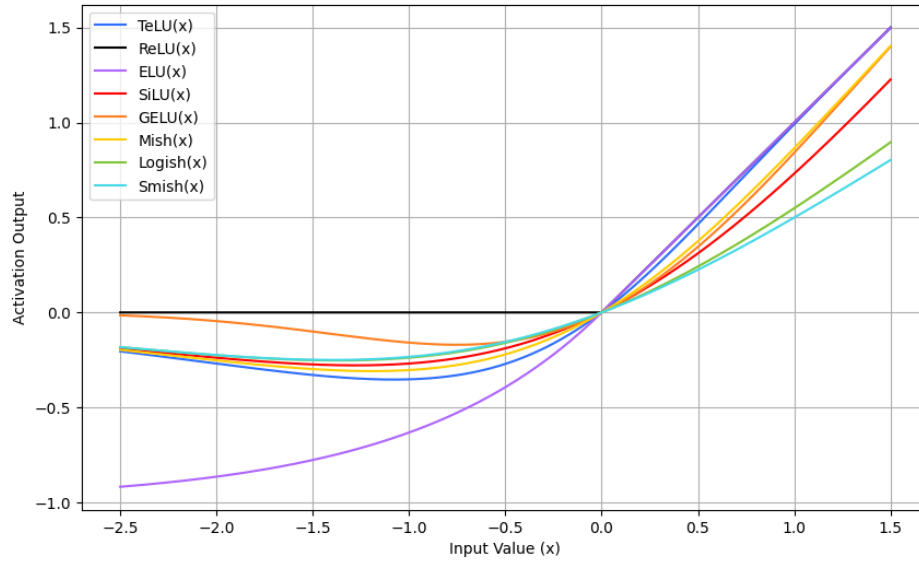


Figure 4.4.2: Focused Set of Linear Unit Activation Functions. Plot depicts our focused group of linear unit activation functions that will commonly be used throughout analysis and experiments.

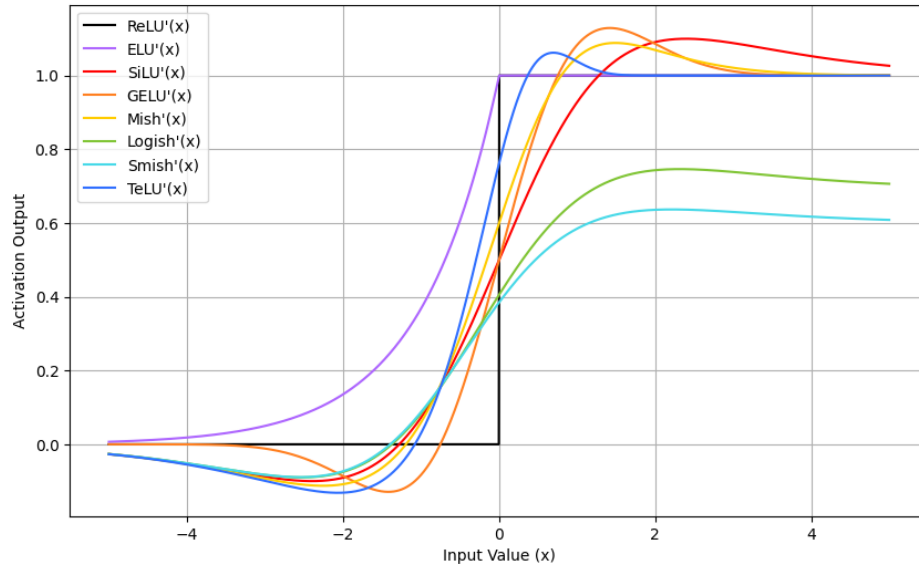


Figure 4.4.3: First Derivatives of Focused Set of Linear Unit Activation Functions. Plot depicts the first-order derivatives of our focused group of activation functions.

Chapter 5:

Theoretical Framework

This chapter provides an in-depth examination of the theoretical advantages of the TeLU activation function, expanding on key principles of activation function design and their impact on neural network performance. It explores essential properties like gradient behavior, convergence speed, computational efficiency, and overall stability, laying the theoretical groundwork for TeLU’s potential benefits. This analysis demonstrates how TeLU addresses common deep learning challenges, including inefficient learning, the vanishing gradient problem, and computational complexity.

5.1 Persistent Gradients of the Saturation Region

We have observed that lower-bounded activation functions are generally advantageous in deep learning. Apart from certain ReLU variants, all the activation functions we’ve discussed are lower-bounded. This aligns with the behavior of biological neurons, which either remain inactive or fire at a specific frequency. Interestingly, even without an initial bias toward lower-bounded functions, Nader and Azar’s evolutionary neural architecture search (NAS) demonstrated a preference for them across all tested datasets. From a many-advisors perspective, it seems that most tasks inherently favor lower-bounded functions. Moreover, we adhere to the heuristic that an activation function should saturate to an output of 0 when a neuron becomes inactive, making neuron inactivity analogous to dropout during inference. However, this presents the challenge of neurons being irreversibly dropped out during training if the gradient in the inactive region decays too quickly to zero.

To address this, our nonlinearity is designed to saturate to 0 when a neuron is inactive while ensuring that the gradient in the inactive region remains above zero for as long as possible. This approach maximizes the chances of the optimizer reactivating a neuron when needed. Ideally, just as we desire strong gradients in the active region, we also aim for relatively strong gradients in the saturating inactive region. To evaluate this aspect of each nonlinearity’s curvature, we first examine the graph of its first derivative. Figure 5.1.1 illustrates the absolute value of each activation function’s first derivative. The plot is shown over negative values near the origin to give an idea about the relative rate that each function’s derivative approaches 0. While some gradients clearly saturate faster than others, we sought a clearer comparison of their decay rates.

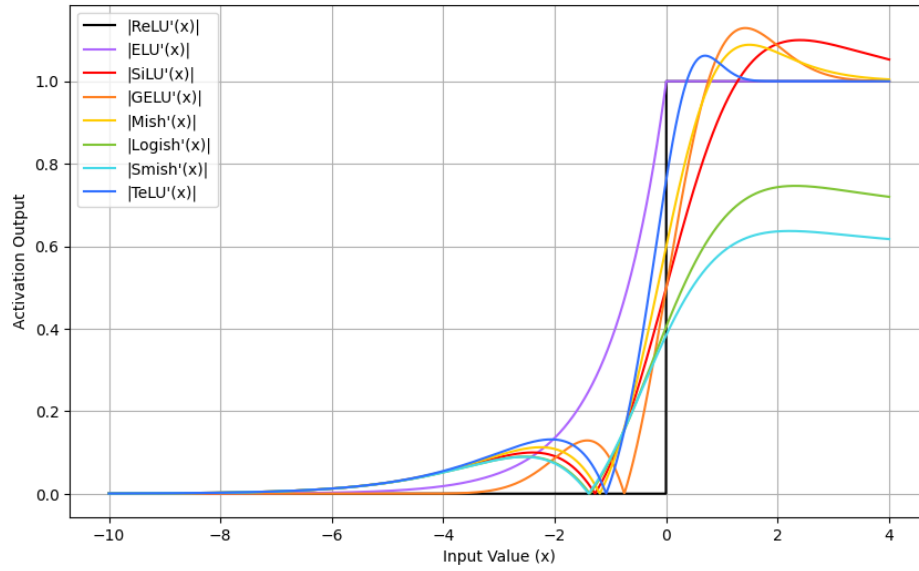


Figure 5.1.1: Saturation of Activation Function as Input Grows Negative. Shows the absolute value of the first derivatives of TeLU, ReLU, ELU, SiLU, GELU, Mish, Logish, and Smish. We notice how some derivatives approximate 0 faster than others, indicating that they decay towards 0 at varying rates.

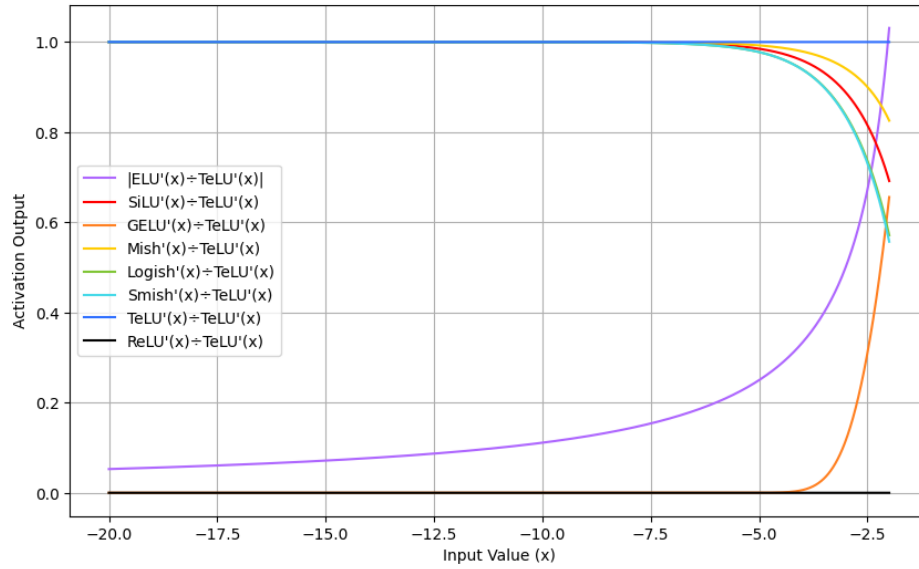


Figure 5.1.2: Quotient Comparisons of Vanishing Gradients as Input Grows Negative. Shows quotient comparison comprising first derivatives of each activation divided by TeLU's first derivative. Plotted values less than one show that TeLU's derivatives saturate slower towards 0 than others at small negative values. The dark blue line shows the derivative of TeLU divided by itself, which is equivalent to 1, as a means to easily compare to other, lesser, quotients in this range.

Figure 5.1.2 addresses this by showing the ratio of TeLU’s absolute derivative to each derivative. This comparison reveals that most non-monotonic nonlinearities maintain gradients that decay more slowly than their monotonic counterparts, albeit with the trade-off of a singular point at zero. This trade-off can be seen as a necessary compromise for gradient persistence. In practice, these solutions emerge primarily due to underflow in lower-precision floating-point variables, as they are likely irrational numbers. Additionally, Figure 5.1.2 indicates that TeLU exhibits stronger gradients at the onset of inactivity compared to other functions. To gain a clear understanding of the rates of gradient decay, we extend the concept of asymptotic growth classes to also encompass asymptotic decay. Therefore, we define the O , Ω , and Θ asymptotic classes as:

Definition 5.1.1. $f(n) \in O(g(n))$ if and only if there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$; where $n \in \mathbb{Z}$

Definition 5.1.2. $f(n) \in \Omega(g(n))$ if and only if there exists positive constants c and n_0 such that $c \cdot g(n) \leq f(n)$ for all $n \geq n_0$; where $n \in \mathbb{Z}$

Definition 5.1.3. $f(n) \in \Theta(g(n))$ if and only if there exists positive constants c_1 , c_2 , and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$; where $n \in \mathbb{Z}$

In traditional terms, a function $f(x)$ belongs to the class $O(g(x))$ if it is bounded above by a scaled version of $g(x)$ beyond a certain point. Similarly, $f(x)$ is in the class $\Omega(g(x))$ if $g(x)$ serves as a lower bound for $f(x)$ beyond a fixed input value. The function $f(x)$ is classified as $\Theta(g(x))$ if and only if it belongs to $O(g(x))$ and $\Omega(g(x))$. Common asymptotic growth classes include $O(1)$, $O(x)$, $O(x^c)$, $O(c^x)$, $O(x!)$, where, c is a positive constant. $a!$ expresses the factorial operation on variable a , which is $\prod_{i=1}^a i$ when $a \in \mathbb{Z}^+$. To extend the factorial operation to real numbers, we utilize the Gamma function $\Gamma(z) = \int_0^{\infty} t^{z-1} e^{-t} dt$.

To define asymptotic decay classes, we invert these growth classes, resulting in classifications like $\Theta(1)$, $\Theta(1/x)$, $\Theta(1/x^2)$, $\Theta(1/e^x)$, and $\Theta(1/x!)$. When we apply this framework to non-monotonic activation functions, we find that the limit of some functions divided by the corresponding $g(x)$ functions does not converge to a nonzero constant. Therefore, we extend the decay classes with $\Theta(x/e^x)$ and $\Theta(1/(x^2)!)$ to better describe the asymptotic decay of functions like TeLU, SiLU, Mish, Logish, and Smish. Interestingly, we discover that GELU does not belong $\Theta(1/x!)$ due to having a greater rate of decay, leading us to classify GELU as an element of $O(1/x!)$ and $\Omega(1/(x^2)!)$. We summarize these asymptotic decay classifications in 5.1.1, with each function mirrored about $x=0$ to more clearly illustrate the mathematical notation.

We further demonstrate the relevance of these asymptotic decay classifications by identifying the approximate regions where each nonlinearity’s gradient underflows when using Pytorch’s float32 precision. To

Table 5.1.1: Asymptotic Decay Qualifications. Shows the asymptotic decay classes that describes the rate at which each activation function’s derivative approaches 0. If the saturation of an activation function $f(x)$ can be described by a tight bound, $\Theta(g(x))$, then the limit of $\frac{-f(x)}{g(x)}$ as $x \rightarrow \infty$ evaluates to a non-zero integer. We could not determine this tight bound for GELU, but instead found upper O and lower Ω bounds. ReLU is simply defined as 0 for negative inputs, so it does not exhibit a particular decay.

Function	ReLU Limit	Asymptotic Class
ReLU	1.0	$\frac{d}{dx}ReLU(-x) \in \Theta(\frac{x}{e^x})$
ReLU	N/A	N/A
ELU	x	$\frac{d}{dx}ELU(-x) \in \Theta(\frac{1}{e^x})$
SiLU	1.0	$\frac{d}{dx}SiLU(-x) \in \Theta(\frac{x}{e^x})$
GELU	e^x	$\frac{d}{dx}GELU(-x) \in O(\frac{1}{x!}), \Omega(\frac{1}{(x^2)!})$
Mish	1.0	$\frac{d}{dx}Mish(-x) \in \Theta(\frac{x}{e^x})$
Logish	1.0	$\frac{d}{dx}Logish(-x) \in \Theta(\frac{x}{e^x})$
Smish	1.0	$\frac{d}{dx}Smish(-x) \in \Theta(\frac{x}{e^x})$

Table 5.1.2: Numerical Inactivity of Activation Functions. Table shows the domain at which each activation function evaluation results in numerical underflow when using a floating point precision of 32 bits. We also determine the evaluations of each activation function at $x = -10, -100$ to show the persistence of gradients both near and far from the origin.

$f(x)$	Null Domain	$f'(-10)$	$f'(-100)$
$ReLU(x)$	$(-\infty, -103.98]$	-4.2826e-06	-1.4013e-42
$ReLU(x)$	$(-\infty, 0.0]$	0.0	0.0
$ELU(x)$	$(-\infty, -103.98]$	7.5826e-10	3.7835e-44
$SiLU(x)$	$(-\infty, -103.98]$	-4.2826e-06	-1.4013e-42
$GELU(x)$	$(-\infty, -14.42]$	-9.2447e-18	0.0
$Mish(x)$	$(-\infty, -103.98]$	-4.2309e-06	-1.4153e-42
$Logish(x)$	$(-\infty, -103.98]$	-4.2309e-06	-1.4153e-42
$Smish(x)$	$(-\infty, -103.98]$	-4.2309e-06	-1.4153e-42

ensure a fair comparison, we defined each function using its typical subfunctions as listed in Table 4. For the first derivative of each nonlinearity, we examined the domain subset $[-200, 0]$, recording instances where the gradient reached zero with a step size of 0.0001. We record the results in 5.1.2 with a precision of two floating point decimal places.

Interestingly, the non-monotonic zero-gradient singularities were not detected, suggesting that they are unlikely to be common during training. Additionally, it is evident that ReLU, SiLU, Mish, Logish, and Smish delay their underflow regions due to their slower asymptotic decay. In contrast, GELU and ELU exhibit broader zero-gradient regions as a result of numerical underflow. ReLU, on the other hand, consistently exhibits zero gradients throughout its entire inactive region, which is characteristic of the "dying ReLU" problem. However, the shared underflow regions observed in ReLU, SiLU, Mish, Logish, and Smish arise from their exponential subfunctions experiencing numerical underflow, which causes them to default to 0. As

a result, the sigmoid component of these functions evaluates to 0, leading to a product of 0 when multiplied by the identity function. Similarly, the GELU activation function is driven to 0 due to the behavior of its error function implementation.

To better understand the comparative numerical decay of these nonlinearities, we examined the values of their derivatives at various points within their inactive regions. Although our full experiment considered 100 different points, for simplicity, we present the gradient values at inputs -100 and -10, which we believe sufficiently capture the overall behavior. These values confirm that TeLU and SiLU maintain stronger gradients at the onset of their post-singularity inactive regions. In contrast, Mish, Logish, and Smish initially exhibit weaker gradients but eventually lead to slightly stronger gradients. This relationship is similar to the comparison between the Hyperbolic Tangent and Sigmoid nonlinearities: while the Hyperbolic Tangent maintains stronger gradients earlier in its inactive region, the Logistic Sigmoid eventually shows slightly stronger gradients deeper into its inactive state.

5.2 Near-Linearity of Active Region

Activation functions that mimic the identity function in their unbounded regions benefit from strong gradients in their active areas, resulting in more efficient training [33, 61]. These strong unit gradients prevent learnable parameter updates from being scaled down by the upstream partial derivatives of saturating functions during backpropagation, allowing the model to reach local minima in the loss landscape more rapidly. At this stage, learning rate schedulers can then lower the effective learning rate to facilitate precise convergence. In contrast, activation functions with sub-identity growth in the active region often require specialized learning rate schedulers to counterbalance their weaker gradients. However, this added complexity increases inter-dependencies among training hyperparameters, leading to a less modular design.

To achieve similar benefits in convergence rate and modularity with a smooth activation function, we aimed to approximate linearity in the active region. In this context, $TeLU(x) = x \cdot \tanh(e^x)$ can be viewed as a variant of $Mish(x) = x \cdot \tanh(\ln(1 + e^x))$, but with the logarithmic term removed, allowing the hyperbolic tangent to saturate to 1 more rapidly. The identity function then multiplies with this saturated hyperbolic tangent, resulting in near-linearity in the active region. As shown in Table 5.2.1, TeLU’s activation for positive inputs aligns more closely with the identity than any other smooth function considered, as measured by both L1 and L2 metrics. The L1 distance metric from linearity is calculated as $\int_0^\infty |f(x) - mx| dx$ for each activation function $f(x)$, where m represents the slope that $f(x)$ approaches as input grows large. Similarly, the L2 distance metric is evaluated as $\int_0^\infty (f(x) - mx)^2 dx$.

Table 5.2.1: Near-Linearity Distance Calculations. Calculating the sums of L1 and L2 distances between each activation function’s active region and the linear function each non-linearity approximates throughout $0 \leq x < \infty$. Logish approximates a slope of $\ln(2)$ as $x \rightarrow \infty$, while Smish approximates a slope of 0.6. All other activation functions either approximate or match the identity function x in growth throughout their active regions. We use these respective slopes to calculate each distance.

Function	L1 Distance	L2 Distance	Max Slope
TeLU	0.0273	0.0008	1.0
SiLU	0.8225	0.1582	1.0
GELU	0.2500	0.0309	1.0
Mish	0.2407	0.0238	1.0
Logish	0.4289	0.0436	$\ln(2)$
Smish	0.2887	0.0201	0.6

Among the listed activation functions, TeLU stands out by exhibiting an active region that most efficiently and rapidly approximates the identity function. This is significant because, in theory, stronger gradients enhance gradient propagation during backpropagation, thereby minimizing the vanishing gradient problem. However, it is important to avoid gradients that persistently exceed 1, as they can lead to exploding gradients. Functions that grow linearly with a gradient of 1 effectively mitigate both vanishing and exploding gradient issues. Piecewise functions like ReLU and ELU are defined as the identity for positive inputs but lack persistent gradients in their negative regions. This deficiency can result in neurons becoming inactive or “dying,” a common issue in ReLU-based networks when the gradient becomes zero for negative inputs. In contrast, TeLU offers persistent, non-zero gradients across its entire domain, reducing the likelihood of neuron inactivation. As discussed in Section 5.1, the persistent gradients provided by TeLU facilitate more stable and consistent weight updates, leading to smoother training dynamics. The combination of persistent gradients in the deactivation region and strong unit gradients in the active region enables neural networks using TeLU to converge faster than those employing other activation functions. This makes TeLU a superior choice for achieving efficient and robust training outcomes.

5.3 Runtime Efficiency

The efficiency of a neuron during forward and backward passes can be significantly enhanced by a nonlinearity with a simple mathematical formulation. Many powerful optimization methods exist that allow major speedups in numerical calculations. Approaches such as dynamic programming [20], linear interpolation [88], low-level language implementations [28], and machine learning hardware accelerators [4] [55] [4] can each provide vast speedups to computations. Together, these optimization methods can provide indeterminately large performance speedups to activation function computation. These, however, come at engineering, hardware, or power costs. In this section, we abstract away such optimizations and focus on the baseline

Table 5.3.1: Baseline Definitions of Activation Functions. Base formulations of activation functions used to determine computational complexity in this section. Legal terms include only expression with universal recognition in mathematics, to provide a fair comparison.

Function	Algebraic Definition
TeLU	$x \cdot \tanh(e^x)$
ReLU	$\text{maximum}(0, x)$
ELU	if $x < 0$: $e^x - 1$; else: x
SiLU	$x \cdot \frac{1}{1+e^{-x}}$
GELU	$x \cdot \frac{1}{2}(1 + \text{erf}(\frac{x}{\sqrt{2}}))$
Mish	$x \cdot \tanh(\ln(1 + e^x))$
Logish	$x \cdot \ln(1 + \frac{1}{1+e^{-x}})$
Smish	$x \cdot \tanh(\ln(1 + \frac{1}{1+e^{-x}}))$

definitions of each non-linearity. We define these expressions to contain only common expressions, namely: x , e^x , $\ln(x)$, $\tanh(x)$, maximum, piece-wise, and the error function. We express each non-linearity in their common forms in 5.3.1, which reflects how each are defined in their initial works [16, 29, 45, 83, 90, 111, 120].

To assess the mathematical complexity of the activation functions in our study, we attempt to quantify their complexity. We hypothesize that a primary contributor to a function’s computational complexity is the number of piecewise operations it contains. We differentiate piecewise operations from typical nonlinear operations, as they are normally implemented by a conditional flow of control rather than a single calculation. These are followed by the non-piecewise non-linear operations each activation function contains. For this, we count the instances of logarithms, exponentials, trigonometric functions, and error functions present in each nonlinearity. To further assess computational complexity, we also tally the number of multiplicative (multiplication or division) and additive (addition or subtraction) operations required by each nonlinearity. that occurs outside of the nonlinear functions previously accounted for. These operations often involve long vectors within neural networks, making them computationally expensive. Finally, we consider the number of constant-valued vectors that need to be allocated for calculating the nonlinearity. Although these allocations can be optimized or amortized in various ways, they still present an existing memory cost. We summarize these quantifications in Table 5.3.2. As there are several plausible ways to express the first derivatives of these activation functions, we only perform such a heuristic assessment in Table A.1.1 in our Appendix.

Table 5.3.2: Computational Costs of Activation Functions. Summarizes the number of instances of nonlinearity computations, arithmetic operations, and constant terms for each activation function. These heuristics provide us with a theoretical evaluation of the computational complexity of each non-linearity.

Function	Piecewise	Nonlinearity	Multiplicative	Additive	Constants
TeLU	0	2	1	0	0
ReLU	1	0	0	0	1
ELU	1	1	0	1	1
SiLU	0	1	2	2	3
GELU	0	1	2	1	3
Mish	0	3	1	1	2
Logish	0	2	3	1	4
Smish	0	3	3	1	4

5.4 ReLU Compatability

The widespread success of the Rectified Linear Unit (ReLU) activation function in stateless architectures, such as Convolutional Neural Networks (CNNs), Transformers, Autoencoders, and Diffusion Models, has firmly established it as the default choice in deep learning research and development. Its simplicity, computational efficiency, and ability to mitigate the vanishing gradient problem have made it indispensable for training a variety of deep architectures. Consequently, an activation function that closely mimics ReLU’s behavior while offering potential improvements is more likely to gain traction within the community. Such a function should not only preserve ReLU’s core characteristics but also enhance network stability, gradient flow, and expressivity across diverse architectures.

To systematically identify an activation function that meets these criteria, we first define a rigorous evaluation metric that measures how closely a candidate function approximates ReLU’s behavior. Let $r(x) = \max(0, x)$ denote the ReLU function and $f(x)$ represent a candidate activation function. The total approximation error is defined as:

$$\mathcal{L}_{\text{total}} = \int_{-\infty}^{\infty} |r(x) - f(x)| dx.$$

This integral quantifies the overall deviation of $f(x)$ from ReLU across the entire input domain. However, because ReLU operates in two distinct regions—*inactive* ($x < 0$) and *active* ($x \geq 0$)—we further decompose the error into contributions from each region:

$$\mathcal{L}_{\text{inactive}} = \int_{-\infty}^0 |r(x) - f(x)| dx, \quad \mathcal{L}_{\text{active}} = \int_0^{\infty} |r(x) - f(x)| dx.$$

This decomposition allows us to analyze whether the approximation errors are concentrated in the suppression of negative values or in the scaling of positive values, which can indicate specific properties of $f(x)$

Table 5.4.1: Sub-Domain Distance of Activation Functions to ReLU. Calculating the sums of L1 distance between each activation function and ReLU along negative and positive subdomains

Function	$(-\infty, 0]$	$[0, \infty)$
TeLU	0.967	0.027
LReLU	∞	0.000
Softplus	0.822	∞
ELU	∞	0.000
SiLU	0.822	0.822
GELU	0.250	0.250
Mish	0.884	0.241
Logish	0.767	∞
Smish	0.760	∞

that may be beneficial or detrimental for particular model classes. We apply these metrics to a comprehensive set of commonly used activation functions, including TeLU, ELU, SiLU, GELU, Mish, Logish, Smish, Leaky ReLU ($LReLU(x) = \max(0.01x, x)$), and Softplus ($Softplus(x) = \ln(1 + e^x)$). Table 5.4.1 summarizes the computed proximity values, providing a robust comparison of how well each function replicates ReLU’s behavior in different regions. This systematic evaluation framework not only facilitates the selection of activation functions that can act as direct substitutes for ReLU but also guides the development of new functions that maintain ReLU’s favorable properties while potentially mitigating known issues such as the “dying ReLU” problem. Ultimately, our methodology supports the discovery of activation functions that can seamlessly integrate into diverse deep learning architectures, ranging from CNNs and Autoencoders to Transformers and emerging generative models like Diffusion Models, ensuring compatibility and enhancing overall model robustness.

To begin identifying an activation function that learns similarly to ReLU, we first assume that the evaluation of such an activation function closely approximates that of ReLU. We quantify the loss of this approximation by determining $\int_{-\infty}^{\infty} |r(x) - f(x)| dx$, where $r(x) = ReLU(x)$ and $f(x)$ represents each candidate function we are evaluating in terms of ReLU approximation. To gain further insight into this approximation, we also evaluate $\int_{-\infty}^0 |r(x) - f(x)| dx$ and $\int_0^{\infty} |r(x) - f(x)| dx$ to quantify the loss of approximation for both the inactive and active region of ReLU. We perform these calculations on TeLU, ELU, SiLU, GELU, Mish, Logish, Smish, $LReLU = \max(0.01x, x)$, and $Softplus(x) = \ln(1 + e^x)$ in Table 5.4.1.

In summary, our findings reveal that only non-monotonic activation functions within the candidate pool can approximate ReLU with a bounded L_1 -norm error. Specifically, these non-monotonic functions share critical mathematical properties that align closely with ReLU’s behavior:

- $f(0) = 0$
- $\lim_{x \rightarrow -\infty} f(x) = 0$

- $\lim_{x \rightarrow \infty} f(x) = \infty$
- $\lim_{x \rightarrow \infty} \frac{f(x)}{x} = 1$

These characteristics are not incidental but are fundamental requirements for any function intended to replicate ReLU’s unique role in deep learning architectures. First, having $f(0) = 0$ ensures that a neuron using such a function will remain inactive when the pre-activation input is zero, preserving the sparsity and inhibition properties essential for ReLU-based networks. Any deviation from this property would alter the activation pattern, impacting the network’s overall inference dynamics. Second, the asymptotic behavior as $x \rightarrow -\infty$ guarantees a dense deactivation region for large negative inputs, which helps induce sparsity by maintaining a large portion of neurons in an inactive state. Without this property, the activation function would produce spurious activations, undermining the benefits of sparse representations.

Moreover, the linear growth as $x \rightarrow \infty$ is a hallmark of ReLU’s effectiveness, enabling strong gradients for large positive inputs. If the growth rate is slower than linear, the function would suffer from vanishing gradients, leading to poor convergence during training. Conversely, if the function grows faster than x , it can exacerbate exploding gradients, resulting in numerical instability. Thus, a successful ReLU approximation must satisfy these growth conditions to ensure compatibility with ReLU-optimized architectures.

The observation that only non-monotonic functions meet these criteria suggests that non-monotonicity is not merely an artifact but a *necessary* feature for accurately emulating ReLU’s behavior. This becomes clearer when we examine the structural similarities across ReLU and other widely adopted activations such as TeLU, SiLU, GELU, and Mish. Each of these functions can be expressed as a scaled cumulative distribution function of the form $x \cdot \sigma(x)$, where $\sigma(x)$ is a smooth, monotonically increasing function with a range of $(0, 1)$. For ReLU, $\sigma(x)$ is the Heaviside step function:

$$\sigma(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}.$$

In TeLU, this distribution is $\sigma(x) = \tanh(e^x)$, while other activations have their own distinct, yet similar, cumulative distributions. From this perspective, we can interpret the non-monotonicity in these functions as a smooth approximation of ReLU’s abrupt transition at $x = 0$. The fact that ReLU can be viewed as a degenerate case of a generalized class of functions $x \cdot \sigma(x)$ suggests that any smooth approximator to ReLU must inherently introduce non-monotonic behavior to bridge the gap between active and inactive regions.

In fact, we can prove that *non-monotonicity* is the only class of single-definition functions capable of approximating ReLU while satisfying the essential properties listed in 5.4. To formalize this, we employ a proof by contradiction. Assume that a purely monotonic function satisfies all the conditions of 5.4. Then,

by construction, it must either fail to deactivate sufficiently in the inactive region ($x < 0$) or fail to grow linearly in the active region ($x > 0$). This contradiction implies that monotonicity cannot achieve both the sparse deactivation and linear activation required to replicate ReLU. Thus, non-monotonicity is not just a feature—it is a prerequisite for any smooth approximation of ReLU.

This insight shifts the focus from simply searching for ReLU substitutes to designing non-monotonic activation functions that retain ReLU’s fundamental properties while offering enhanced numerical stability and gradient behavior. By framing the problem in this way, we establish a theoretical foundation for developing next-generation activations that are both mathematically principled and empirically effective.

Theorem 5.4.1. *Let $r(x) = \text{ReLU}(x) = \max(0, x)$ be defined as:*

$$r(x) = \begin{cases} 0, & \text{if } x \leq 0, \\ x, & \text{if } x > 0. \end{cases}$$

This function has the following properties:

- $r(0) = 0$,
- $\lim_{x \rightarrow -\infty} r(x) = 0$,
- $\lim_{x \rightarrow \infty} r(x) = \infty$.

Let $f(x)$ be a single continuous and differentiable function defined on $\mathbb{R}$ that satisfies the same properties as $r(x)$ for all $x \in \mathbb{R}$. Then, any such $f(x)$ that approximates $r(x)$ must be non-monotonic. In other words, $f(x)$ must both increase and decrease throughout its domain.

Proof. Step 1: Properties of ReLU and Its Derivative

The ReLU function $r(x) = \max(0, x)$ has a piecewise linear form:

$$r(x) = \begin{cases} 0, & \text{if } x \leq 0, \\ x, & \text{if } x > 0. \end{cases}$$

Its derivative $r'(x)$ is given by:

$$r'(x) = \begin{cases} 0, & \text{if } x < 0, \\ 1, & \text{if } x > 0. \end{cases}$$

The derivative of ReLU changes abruptly at $x = 0$, making $r(x)$ non-differentiable at that point. This sharp transition is what any smooth approximating function $f(x)$ must capture to closely resemble $r(x)$.

Step 2: Conditions on $f(x)$ for Smooth Approximation Suppose $f(x)$ is a smooth, continuous, and differentiable function defined for all $x \in \mathbb{R}$. Furthermore, assume that $f(x)$ is a monotonic function, with an absence of either increasing or decreasing behavior along all $x \in \mathbb{R}$. To approximate $r(x)$, $f(x)$ must satisfy:

1. $f(0) = 0$, 2. $\lim_{x \rightarrow -\infty} f(x) = 0$, 3. $\lim_{x \rightarrow \infty} f(x) = \infty$.

Since $f(x)$ is continuous and differentiable at all points, there exist only three possible cases that describe the growth of $f(x)$ along all $x \in (-\infty, 0]$:

1. $f(x)$ is constant-valued across all $x \in (-\infty, 0]$, without increasing or decreasing.
2. $f(x)$ increases throughout $a < x < b$, where $a, b \in (-\infty, 0]$, by an amount $d \in \mathbb{R}^+$.
3. $f(x)$ decreases throughout $a < x < b$, where $a, b \in (-\infty, 0]$, by an amount $d \in \mathbb{R}^+$.

We proceed by following each case.

Case 1: Suppose $f(x)$ is constant for all $x \in (-\infty, 0]$. For $f(x)$ to remain constant over this interval, it must be defined as a constant value, say $f(x) = 0$, for all $x \in (-\infty, 0]$. Now, if $f(x)$ is to increase for $x > 0$, then at some point $c > 0$, $f(x)$ must exhibit growth, i.e., $f(x)$ must no longer remain constant. This implies that $f(x)$ must change its behavior at $x = 0$, transitioning from being constant for $x \in (-\infty, 0]$ to increasing for $x > 0$. For instance, we might redefine $f(x)$ as an increasing function, say $g(x)$, for $x > 0$. However, this construction results in a contradiction because the function $f(x)$ is no longer defined by a single expression throughout all $x \in \mathbb{R}$. Instead, it would require two distinct definitions: one for $x \in (-\infty, 0]$, where $f(x) = 0$, and another for $x > 0$, where $f(x)$ follows some increasing function. Such a piecewise definition contradicts the assumption that $f(x)$ is a single continuous function across $\mathbb{R}$.

In conclusion, for $f(x)$ to be both constant for $x \in (-\infty, 0]$ and increasing for $x > 0$, it would have to be defined by two distinct expressions, which contradicts the premise that $f(x)$ is a single function over all $x \in \mathbb{R}$. Therefore, $f(x)$ cannot satisfy both conditions without being discontinuous or piecewise defined.

Case 2: Let $f(x)$ be a continuous and monotonically increasing function on the interval (a, b) . By definition, this means that for any $x_1, x_2 \in (a, b)$ with $x_1 < x_2$, we have $f(x_1) \leq f(x_2)$. Therefore, $f(x)$ does not exhibit any decreasing behavior on (a, b) . Assume further that $\lim_{x \rightarrow -\infty} f(x) = 0$ and that f increases by an amount d over the interval (a, b) , meaning that $f(b) = f(a) + d$. Thus, the relationship between $f(a)$ and $f(b)$ is given by:

$$f(b) = f(a) + d,$$

where $d > 0$ due to the increasing nature of $f(x)$. By the Intermediate Value Theorem (IVT), since $f(x)$ is continuous on (a, b) , for any value $c \in (f(a), f(b))$, there exists some $x_0 \in (a, b)$ such that $f(x_0) = c$. In particular, for any small $\epsilon > 0$, we can choose $c = f(b) - \epsilon = d - \epsilon$. Thus, there exists some $x_0 \in (a, b)$ such that:

$$f(x_0) = d - \epsilon.$$

This does not imply that $f(x)$ decreases anywhere in (a, b) . Instead, because $f(x)$ is monotonically increasing, it passes through the value $d - \epsilon$ while continuing to increase from $f(a)$ to $f(b)$.

To summarize, $f(x)$ can take any value between $f(a)$ and $f(b)$ by the IVT, and since $f(x)$ is increasing, it achieves these values without any decrease. Therefore, the fact that $f(x)$ equals $d - \epsilon$ for some $x_0 \in (a, b)$ is entirely consistent with the premise that $f(x)$ is a monotonically increasing function. Hence, we conclude that the function $f(x)$ increases by d from $f(a)$ to $f(b)$, and the Intermediate Value Theorem guarantees that all values between $f(a)$ and $f(b)$ are attained in the interval (a, b) , without any need for $f(x)$ to decrease."

Case 3: Let $f(x)$ be a continuous and monotonically decreasing function on the interval (a, b) . By definition, for any $x_1, x_2 \in (a, b)$ with $x_1 < x_2$, we must have $f(x_1) \geq f(x_2)$. Therefore, $f(x)$ does not exhibit any increasing behavior on (a, b) .

Now, assume that $\lim_{x \rightarrow -\infty} f(x) = 0$ and that $f(x)$ decreases by an amount d throughout the interval $a < x < b$. This implies that:

$$f(b) = f(a) - d,$$

where $d > 0$ because $f(x)$ is decreasing. Therefore, $f(b)$ is less than $f(a)$. Additionally, assume that $f(0) = 0$ and $d < f(0) = 0$, meaning that $f(b) = d$ for some negative value of d , i.e., $f(b) = d < 0$. By the Intermediate Value Theorem (IVT), since $f(x)$ is continuous and decreasing, for any value $c \in (f(b), f(a))$, there must exist some $x_0 \in (a, b)$ such that $f(x_0) = c$. In particular, for any small $\epsilon > 0$, we can choose $c = d + \epsilon$, so there exists some $x_0 \in (a, b)$ such that:

$$f(x_0) = d + \epsilon,$$

where $d + \epsilon \in (d, 0)$.

However, this does not imply that $f(x)$ increases anywhere on (a, b) . Since $f(x)$ is monotonically decreasing, it achieves the value $d + \epsilon$ while continuing to decrease from $f(a)$ to $f(b)$. Thus, there is no need for the function to increase at any point, and no contradiction arises from the IVT. The assertion that $f(x)$

must increase at some point within $(-\infty, 0]$ is incorrect. A monotonic function can still take values like $d + \epsilon$ while remaining strictly decreasing. Therefore, the idea that $f(x)$ must both decrease and increase within $(-\infty, 0]$ is false and does not follow from the given assumptions.

In conclusion, the assumption that $f(x)$ is monotonic does not lead to any contradictions. As a result, the claim that $f(x)$ must be non-monotonic is unfounded. $f(x)$ can remain monotonically decreasing without violating the Intermediate Value Theorem or the given conditions.

□

Having shown that a function that closely approximates ReLU must be non-monotonic, we proceed to compare our non-monotonic candidates in terms of ReLU approximation. We recall that in Table 5.4.1, TeLU achieves the best approximation to ReLU along the active subdomain $x \in [0, \infty]$. On the other hand, GELU approximates ReLU best along the inactive subdomain $x \in (-\infty, 0]$. We argue that closely approximating ReLU while it exhibits strong gradients in the active region outweighs closely approximating it along the near-zero inactive region. The active region exhibits stronger gradients, and therefore has a larger influence on the way a neural network learns. For this reason, we hypothesize that ReLU should operate more similarly to TeLU than GELU. We detail our reasoning in the proceeding lemma:

Lemma 5.4.1. *Let $r(x) = \text{ReLU}(x) = \max(0, x)$ be defined as:*

$$r(x) = \begin{cases} 0, & \text{if } x \leq 0, \\ x, & \text{if } x > 0. \end{cases}$$

Define the active subdomain of $r(x)$ as $\mathcal{A} = [0, \infty)$ and the inactive subdomain as $\mathcal{I} = (-\infty, 0)$.

Let $t(x) = \text{TeLU}(x)$ and $g(x) = \text{GELU}(x)$ be two smooth, continuous, and differentiable activation functions such that:

- 1. $t(x) \approx r(x)$ for $x \in \mathcal{A}$, 2. $g(x) \approx r(x)$ for $x \in \mathcal{I}$.*

Furthermore, let $f(x)$ be an arbitrary continuous and differentiable function that approximates $r(x)$ over the entire domain $x \in \mathbb{R}$.

Define the approximation error for $f(x)$ over the active subdomain $\mathcal{A}$ and inactive subdomain $\mathcal{I}$ as:

$$E_{\mathcal{A}}(f) = \int_0^{\infty} |f(x) - r(x)|^2 dx,$$

$$E_{\mathcal{I}}(f) = \int_{-\infty}^0 |f(x) - r(x)|^2 dx.$$

If $E_{\mathcal{A}}(t) < E_{\mathcal{A}}(g)$ and $E_{\mathcal{I}}(g) < E_{\mathcal{I}}(t)$, then $t(x)$ provides a closer approximation to $r(x)$ in the active region, while $g(x)$ provides a closer approximation to $r(x)$ in the inactive region.

Moreover, the influence of $f(x)$ on neural network training is determined by the gradients in each subdomain:

$$I_{\mathcal{A}}(f) = \int_0^{\infty} |f'(x)|^2 dx,$$

$$I_{\mathcal{I}}(f) = \int_{-\infty}^0 |f'(x)|^2 dx.$$

If $I_{\mathcal{A}}(t) > I_{\mathcal{I}}(g)$, then $t(x)$ has a stronger impact on training in the active region than $g(x)$ in the inactive region.

If $t(x)$ and $g(x)$ are used to approximate $r(x)$, then $t(x)$ is a better surrogate for ReLU in neural network training due to the stronger gradients and smaller approximation error in the active subdomain $\mathcal{A}$. Thus, for tasks where the active region influences learning more heavily, $t(x)$ should behave more similarly to $r(x)$ than $g(x)$.

TeLU demonstrates gradient behavior that closely mirrors the strong identity growth seen in the ReLU activation function. Specifically, when the inputs are positive, both TeLU and ReLU produce active gradients that are approximately equal to one, leading to a convergence speed during training that is comparable for both functions.

This rapid convergence arises because strong gradients enable more significant parameter updates with each training iteration, helping the model learn faster. However, this property also implies that TeLU, like ReLU, does not impose much implicit regularization on the model. Implicit regularization refers to the tendency of some activation functions to naturally limit the magnitude of updates, thereby acting as a form of regulation on model complexity. In the case of ReLU and TeLU, the strong gradients reduce this implicit regularization effect, allowing the network to fit the data more efficiently.

In contrast, other activation functions, such as Logish or Smish, have weaker active gradients—especially as inputs move away from zero. This results in a "damping" effect where gradient updates are smaller, slowing down convergence. This behavior effectively regularizes the learning process by preventing overly large parameter updates, which can help in scenarios where controlling model complexity and preventing overfitting are desired. Thus, the distinct behavior of TeLU, with its strong gradients and rapid convergence, sets it apart from these other functions that inherently moderate the learning speed through gradient damping.

This characteristic leads to a more modular design of the neural network training procedure, as other components, such as the optimization algorithm and learning rate scheduler, can take on a more direct role in

managing learning steps. With the activation function playing a lesser role in regularization, the optimization algorithm can be fine-tuned to control learning aggressiveness through parameters like momentum and decay rates, while the learning rate scheduler dynamically adjusts based on training progress. This approach allows for more precise tailoring of the training dynamics using external techniques, resulting in more efficient and effective training. Consequently, network regularization and convergence behavior become more configurable, with the activation function primarily introducing non-linearity rather than acting as a built-in regularization tool.

In summary, non-monotonic functions are well-suited to smoothly approximate the $ReLU = \max(0, x)$ nonlinearity while mimicking ReLU’s saturating behavior as $x \rightarrow -\infty$, its deactivation at $x = 0$, and its unbounded linear growth as $x \rightarrow \infty$. The TeLU activation function, in particular, closely replicates the inference and learning dynamics of models using ReLU when substituted in its place. Both TeLU and ReLU exhibit strong gradients that help reduce the implicit regularization imposed by the activation function, enabling a more modular training design. These similarities make TeLU an excellent drop-in replacement for ReLU in deep neural networks.

5.5 Analytic Universal Approximation

Definition 5.5.1. A function $g : \mathbb{X} \rightarrow \mathbb{R}^n$ whose domain $\mathbb{X} \subset \mathbb{R}^n$ is a universal approximator over $\mathbb{X}$ if and only if there exist constant vectors α , w , and $b \in \mathbb{R}^n$ such that $|\sum_{j=1}^n \alpha_j \cdot g(w_j^\top x + b_j) - f(x)| < \epsilon$ for all $x \in \mathbb{X}$, where $f : \mathbb{X} \rightarrow \mathbb{R}^n$ is any arbitrary continuous function and ϵ is any arbitrarily small positive error.

In other words, a univariate function $g(x)$ is considered a universal approximator if it can be used in a finite linear combination of compositions, each with an affine transformation of the input, to approximate any multivariate continuous function over a bounded domain. In the context of deep learning, this concept guarantees that there exists a neural network with a single hidden layer and a universal approximation activation function that can effectively perform any task, provided the target function to be approximated is continuous and the input range is bounded. This principle underpins the capability of neural networks to model a vast range of complex functions with sufficient precision.

A pivotal breakthrough that underpins the use of sigmoidal activation functions in neural networks is the universal approximation theorem, first established independently by Cybenko (1989), Hornik et al. (1989), and Funahashi (1989). This foundational theorem asserts that a standard neural network with a single hidden layer of activation $\sigma : \mathbb{X} \rightarrow \mathbb{R}^n$ can uniformly approximate any continuous function over a bounded domain $\mathbb{X} \subset \mathbb{R}^n$ provided that σ is sigmoidal. A function σ is sigmoidal if and only if $\lim_{x \rightarrow -\infty} \sigma(x) = 0$ and $\lim_{x \rightarrow \infty} \sigma(x) = 1$. The compact domain $\mathbb{X} \subset \mathbb{R}^n$ constraint is a given within digital computer systems that

use a finite number of bits to represent numbers. In essence, this means that such networks, despite their shallow structure, possess the remarkable ability to approximate any continuous functions with arbitrary precision.

Building on this foundation, Hornik (1991) expanded the universal approximation theorem to include any continuous, bounded, and non-constant activation function, thereby broadening the potential of neural networks as universal approximators. Further refinement came from the work of Leshno et al. (1993), who demonstrated that for continuous activation functions, a standard neural network with one hidden layer can uniformly approximate any continuous function if the activation function employed by the hidden layer is non-polynomial. This highlights the importance of non-polynomial activation functions in ensuring the universal approximation capabilities of neural networks. Since $TeLU$ cannot be expressed as a polynomial, it is a universal approximator. However, we will demonstrate that $TeLU$ is a universal approximator ourselves with support from Cybenko's (1989) approach to gain a mathematical handle on the concept. This will allow us to be able to prove further properties regarding the quality of universal approximation of $TeLU$. Now, we will state and prove that $TeLU(x) = x \cdot \tanh(e^x)$ is a universal approximator:

Lemma 5.5.1. *Let $I_n = [a, b]^n$ be an n -dimensional hypercube, and let $C(I_n)$ denote the space of continuous functions on I_n , where $a, b \in \mathbb{R}$ and $a < b$. Suppose $f \in C(I_n)$ is a continuous function. Then, there exists a finite linear combination of the form*

$$g(x) = \sum_{i=1}^M c_i \cdot TeLU(w_i^T x + b_i)$$

where $TeLU(x) = x \cdot \tanh(e^x)$, such that for any $\epsilon > 0$,

$$|f(x) - g(x)| < \epsilon \quad \forall x \in I_n,$$

where $w_i \in \mathbb{R}^n$, and $c_i, b_i \in \mathbb{R}$.

Proof. We first analyze the asymptotic behavior of the function $TeLU(x) = x \cdot \tanh(e^x)$:

- As $x \rightarrow -\infty$, we have $e^x \rightarrow 0$ and therefore $\tanh(e^x) \rightarrow 0$, so

$$TeLU(x) = x \cdot \tanh(e^x) \rightarrow 0.$$

- As $x \rightarrow \infty$, $e^x \rightarrow \infty$ and $\tanh(e^x) \rightarrow 1$, thus

$$TeLU(x) = x \cdot \tanh(e^x) \rightarrow x.$$

Now define the function $\sigma(x) = TeLU(x) - TeLU(x - 1)$. We claim that $\sigma(x)$ behaves as a sigmoidal function:

- As $x \rightarrow -\infty$, both $TeLU(x) \rightarrow 0$ and $TeLU(x - 1) \rightarrow 0$, so

$$\sigma(x) \rightarrow 0.$$

- As $x \rightarrow \infty$, $TeLU(x) \rightarrow x$ and $TeLU(x - 1) \rightarrow x - 1$, thus

$$\sigma(x) = TeLU(x) - TeLU(x - 1) \rightarrow 1.$$

Thus, $\sigma(x)$ has the asymptotic properties of a sigmoidal function: $\sigma(x) \rightarrow 0$ as $x \rightarrow -\infty$ and $\sigma(x) \rightarrow 1$ as $x \rightarrow \infty$.

By Cybenko's universal approximation theorem, for any $f \in C(I_n)$ and any $\epsilon > 0$, there exist coefficients $\alpha_j \in \mathbb{R}$, vectors $y_j \in \mathbb{R}^n$, and biases $b_j \in \mathbb{R}$ such that

$$f(x) \approx \sum_{j=1}^N \alpha_j \sigma(y_j^T x + b_j),$$

where $\sigma(x)$ is a sigmoidal function. Substituting $\sigma(x) = TeLU(x) - TeLU(x - 1)$, this becomes:

$$f(x) \approx \sum_{j=1}^N \alpha_j [TeLU(y_j^T x + b_j) - TeLU(y_j^T x + b_j - 1)].$$

Now, we can rewrite the sum as a linear combination of $TeLU$ -based terms. Defining $M = 2N$ and $\oplus$ as a concatenating operator, we represent the following vector concatenations:

$$\hat{y} = y \oplus y, \quad \hat{\alpha} = \alpha \oplus -\alpha, \quad \hat{b} = b \oplus (b - 1),$$

which leads to the expression:

$$g(x) = \sum_{j=1}^M \hat{\alpha}_j TeLU(\hat{y}_j^T x + \hat{b}_j).$$

By Cybenko's theorem, there exists a sum $g(x)$ such that

$$|f(x) - g(x)| < \epsilon \quad \forall x \in I_n.$$

Thus, $TeLU(x) = x \cdot \tanh(e^x)$ is a universal approximator for continuous functions.

□

Having proven that *TeLU* is a universal approximator, we have shown that, in theory, a neural network with a single hidden layer that employs the *TeLU* activation function will approximate any unknown target function. An architecture that is capable of universal approximation enjoys the practical benefits of flexible modeling, being able to be trained to reach adequate performance in any task that shares its input and output dimensions. Additionally, with a universal approximator, deep learning engineers do not need to worry about if their chosen activation functions are capable of fitting to a task. Instead, they can focus their design efforts to scaling the model’s parameterization to better fit the desired task to prevent overfitting. Therefore, engineers that apply continuous non-polynomial functions to their fully connected layers may significantly reduce their search to tuning other hyperparameters such as network size, learning rate, and optimizer choice.

A universal approximator ensures that a network can approximate any unknown target function, but not that it will approximate it well enough without overfitting. Some universal approximators like *ReLU* are only capable of providing piecewise linear representations of tasks, and therefore lack representation power. The limited expressiveness of these *ReLU* approximations amounts to linear interpolation, which is prone to instability when attempting to model smooth relationships between input and output. Other non-linearities such as *ELU* offer a continuous first derivative, but a discontinuous second derivative, making it incompatible with second order optimization procedures that improve convergence speed and stability during training. Therefore, we require an additional property beyond standard universal approximator to concisely state this quality of universal approximation. We turn our attention towards the property of a function being analytic:

Definition 5.5.2. A function $f(x)$ is said to be analytic at a point x_0 if there exists a neighborhood around x_0 such that $f(x)$ can be represented as a convergent power series:

$$f(x) = \sum_{n=0}^{\infty} c_n (x - x_0)^n$$

for some constants c_n , where the series converges to $f(x)$ for all x in this neighborhood.

When a function is analytic, it is implied that it is indefinitely differentiable at all points. This property makes them highly compatible with second-order optimization strategies that utilize the Hessian matrix of the cost function. By incorporating the Hessian matrix, optimization algorithms can account for the curvature of the loss function, leading to more stable and efficient convergence. Moreover, analytic functions facilitate the smooth transfer of information through the network, resulting in more dispersed neuron activations, which contribute to more robust and generalized representations (CITE). Additionally, analytic functions

are compatible with a broader range of mathematical analysis techniques, such as spectral analysis and perturbation methods, which can further aid in understanding and improving the behavior of neural networks during training and inference. Since $TeLU$ is a smooth function composed of analytic functions, we claim and demonstrate that it is also an analytic function. In the following lemma, we demonstrate that $TeLU$ is indeed an analytic function:

Lemma 5.5.2. *$TeLU(x) = x \cdot \tanh(e^x)$ is an analytic function, where $x \in \mathbb{R}$.*

Proof. We begin by noting that $TeLU(x) = x \cdot \tanh(e^x)$ is the product of the identity function and the composition of the hyperbolic tangent and the exponential functions.

First, consider the exponential function e^x , which has the well-known Maclaurin series expansion:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

This series converges for all $x \in \mathbb{R}$, showing that e^x is analytic.

Next, we recall the Maclaurin series expansion for the hyperbolic tangent function $\tanh(x)$:

$$\tanh(x) = \sum_{n=1}^{\infty} \frac{2^{2n}(2^{2n}-1)B_{2n}}{(2n)!} x^{2n-1}$$

where B_{2n} are the Bernoulli numbers. This series also converges for all $|x| < \frac{\pi}{2}$, establishing that $\tanh(x)$ is analytic within this domain.

Since $\tanh(e^x)$ is the composition of the analytic functions $\tanh(x)$ and e^x , $\tanh(e^x)$ is analytic for all $x \in \mathbb{R}$.

Finally, $TeLU(x) = x \cdot \tanh(e^x)$ is the product of the analytic function $\tanh(e^x)$ and the identity function x , which is itself analytic. By the closure properties of analytic functions, the product of two analytic functions is also analytic.

Thus, $TeLU(x) = x \cdot \tanh(e^x)$ is an analytic function, as it can be expressed as a power series:

$$TeLU(x) = \sum_{n=0}^{\infty} b_n x^{n+1}$$

where $b_n \in \mathbb{R}$ are constants derived from the power series of $\tanh(e^x)$. This completes the proof. $\square$

Having established that $TeLU$ is analytic, it is important to revisit the practical benefits this property brings to neural network architectures. The smoothness and infinite differentiability of $TeLU$ enhance its compatibility with second order optimization methods such as Natural Gradient Descent (NGD) [85]. This leads to more stable convergence and improved training efficiency, especially in complex models. Additionally,

the smooth transfer of information throughout the network helps promote balanced neuron activations, allowing for robust feature representations and better generalization to unseen data. The analytic nature of TeLU also opens up opportunities for utilizing sophisticated mathematical tools, offering deeper insights and potential for further optimization during training and inference. We now show that $TeLU$, being a universal approximator that is also analytic, leads to analytic universal approximations:

Theorem 5.5.1. *The function $TeLU(x) = x \cdot \tanh(e^x)$ is an analytic universal approximator. In other words, any continuous function $f(x)$ can be approximated by a linear combination of $TeLU$ nonlinearities, and the resulting approximation will be analytic.*

Proof. We have already established that $TeLU(x) = x \cdot \tanh(e^x)$ is analytic. Let $f(x) \in C(\mathbb{R}^n)$ be an arbitrary continuous function, and let $g(x)$ be a linear combination of $TeLU$ nonlinearities that approximates $f(x)$. Thus, for any $\epsilon > 0$, there exists a function $g(x)$ of the form

$$g(x) = \sum_{j=1}^M \alpha_j \cdot TeLU(w_j^\top x + b_j)$$

such that the approximation error satisfies

$$|g(x) - f(x)| < \epsilon \quad \forall x \in \mathbb{R}^n.$$

Since $TeLU(x)$ is analytic, we can express each $TeLU(w_j^\top x + b_j)$ as a convergent power series. Therefore, the function $g(x)$ becomes a summation of power series expansions for each term:

$$g(x) = \sum_{j=1}^M \alpha_j \cdot \sum_{n=0}^{\infty} c_n (w_j^\top x + b_j)^n$$

where c_n are the coefficients of the power series expansion. Rewriting the expression, we have:

$$g(x) = \sum_{j=1}^M \sum_{n=0}^{\infty} \alpha_j c_n (w_j^\top x + b_j)^n = \sum_{j=1}^M h_j(x),$$

where $h_j(x)$ represents the power series expansion of each individual term $TeLU(w_j^\top x + b_j)$. Since the sum of analytic functions is also analytic, the approximation $g(x)$ is a sum of M analytic functions $h_j(x)$, and thus $g(x)$ is analytic.

Consequently, the linear combination of $TeLU(x)$ nonlinearities that approximates $f(x)$ is itself an analytic function. Therefore, we conclude that $TeLU(x) = x \cdot \tanh(e^x)$ is an analytic universal approximator. $\square$

With the proof that an architecture employing TeLU as its hidden activation function is capable of analytic universal approximations, the model achieves several theoretical and practical advantages. Theoretically, this capability ensures that the architecture can approximate a wide range of continuous and differentiable functions with arbitrary precision, which is essential in modeling complex, nonlinear systems. Analytic universal approximation guarantees that not only can the model represent any function, but it can do so with a smooth, well-behaved structure, preserving differentiability across all layers. This property is crucial in tasks requiring optimization through gradient-based methods, as it mitigates issues related to non-smoothness and sharp transitions in the loss landscape. The analytic nature also allows the model to generalize better by capturing underlying relationships in data, rather than fitting to noise or outliers.

Practically, having an architecture with analytic universal approximation capabilities means that the model can be applied to a broader set of problems across domains such as physics, engineering, and finance, where smooth and continuous approximations are needed for high-fidelity simulations and predictions. For instance, in control systems or physical simulations, TeLU-enabled architectures can model complex dynamics with high precision, leading to better predictive power and stability. Additionally, this property often results in more efficient training, as the smoothness in activation and loss functions can lead to faster convergence and more reliable gradient flow, addressing problems like vanishing gradients.

Having established that $TeLU(x) = x \cdot \tanh(e^x)$ is an analytic universal approximator, we now explore the theoretical and practical benefits of architectures capable of representing problems with analytic universal approximations.

5.5.1 Theoretical Benefits

Property 5.5.1. *Guaranteed Approximability of Continuous Functions:* An architecture using analytic universal approximators can approximate any continuous function $f(x) \in C(\mathbb{R}^n)$ to an arbitrary degree of accuracy. This is ensured by the universal approximation theorem.

Since $TeLU(x)$ is analytic, it guarantees that smooth, continuous functions can be approximated with a high degree of precision. The smoothness of analytic functions, which are infinitely differentiable within their domain, enables the architecture to handle complex functional relationships.

Property 5.5.2. *Convergence Guarantees:* Due to the well-behaved nature of analytic functions, architectures using $TeLU(x)$ exhibit stable and efficient convergence properties when optimized using gradient-based methods.

Since analytic functions have continuous derivatives, the resulting loss surface is smooth, allowing gradient-based optimization methods to converge faster and more reliably to optimal solutions without getting stuck in local minima.

Property 5.5.3. *Higher-Order Information:* Analytic functions are infinitely differentiable, meaning that architectures with analytic approximators can capture not only the function’s value but also higher-order derivatives, enabling more precise modeling of the underlying process.

This property is crucial in tasks that require higher-order information, such as physics simulations, where capturing the curvature of the function is essential. It also allows architectures to model complex dynamical systems that require higher-level functional representations.

Property 5.5.4. *Avoidance of Pathological Behavior:* Non-analytic functions may exhibit discontinuities or oscillations, which are not desirable in many practical applications. Analytic approximators avoid these issues by ensuring smooth and well-behaved representations.

Non-analytic functions can introduce undesirable behaviors, such as discontinuities or oscillations, which may complicate the learning process and reduce the model’s effectiveness in practical applications. By contrast, architectures using analytic approximators, like TeLU, ensure that the approximated functions are smooth and continuous. This smoothness prevents pathological behaviors, enabling the model to produce well-behaved representations that are more reliable and predictable, particularly in real-world scenarios where stability and smooth transitions are critical.

5.5.2 Practical Benefits

Property 5.5.5. *Improved Generalization:* An analytic universal approximator tends to generalize better to unseen data, as it avoids overfitting to noise or specific artifacts in the training data by leveraging smooth approximations.

In practice, smooth approximations help models capture the underlying patterns of the data without overfitting, making them more robust to noisy or incomplete datasets. This leads to improved performance on real-world tasks, particularly in domains with complex, non-linear relationships.

Property 5.5.6. *Efficiency in Optimization:* Architectures using analytic functions provide smooth loss landscapes, allowing for more efficient optimization. This reduces the likelihood of getting trapped in local minima and speeds up convergence during training.

Since the gradients of analytic functions are stable and predictable, optimization techniques such as gradient descent perform more effectively. In practical terms, this means that training models with analytic

approximators can be faster and require fewer iterations, making them suitable for large-scale machine learning tasks.

Property 5.5.7. *Better Interpretability and Differentiability:* Analytic universal approximators offer a smooth, interpretable functional form, which makes them easier to understand and analyze. They are also well-suited for tasks requiring sensitivity analysis or gradient-based algorithms.

Because analytic functions are smooth and continuous, they are more interpretable. Additionally, their differentiability allows for the use of gradient-based optimization methods in tasks such as reinforcement learning or control systems, where differentiability is critical.

Property 5.5.8. *Robustness to Numerical Precision:* Analytic functions, due to their smoothness, are less prone to numerical instability, making them more reliable when implemented on hardware with limited floating-point precision.

This is particularly important for edge computing or embedded systems, where precision is limited. Analytic universal approximators like $TeLU(x)$ provide stable computations, reducing the risk of errors caused by floating-point approximations.

Property 5.5.9. *Applicability in Physics-Informed Models:* Many real-world problems, such as those in physics or engineering, are governed by smooth, differentiable equations. Analytic approximators are well-suited for these domains because they can represent smooth transitions and capture physical laws more accurately.

In practical applications like fluid dynamics, electromagnetism, or mechanical systems, analytic functions provide a natural fit due to their inherent smoothness and ability to model physical systems with high precision.

Property 5.5.10. *Smoothness in Adversarial Settings:* Analytic functions are naturally resistant to adversarial perturbations because small changes in input lead to small, predictable changes in output, making it harder for adversarial examples to exploit vulnerabilities in the model.

In adversarial machine learning, smoothness helps protect the model from being tricked by small, carefully crafted perturbations. This makes architectures based on analytic approximators more robust against adversarial attacks, enhancing security and reliability in sensitive applications.

Thus the architectures that utilize analytic universal approximators, such as $TeLU(x) = x \cdot \tanh(e^x)$, offer significant theoretical and practical advantages. They guarantee smooth, continuous, and well-behaved approximations, provide better generalization, and are computationally efficient. These properties make

analytic universal approximators highly valuable for a wide range of applications, from scientific computing and optimization to adversarial robustness and interpret-ability.

5.6 Stability

In deep neural networks, stability refers to the network’s ability to produce consistent and reliable outputs when exposed to small perturbations or changes in input data, weights, or training conditions. Theoretical stability is crucial for ensuring that the network generalizes well to unseen data and avoids issues like exploding or vanishing gradients, which can lead to erratic behavior during training. Without proper attention to stability, networks may suffer from poor convergence, unstable training dynamics, or a lack of reliability in deployment.

Activation functions play a crucial role in influencing the stability of deep neural networks by determining how neurons respond to input signals and propagate gradients during training. Functions like ReLU [29] mitigate the vanishing gradient problem, enabling better gradient flow in deep networks, but can lead to dead neurons, causing instability. Conversely, sigmoid and tanh activations are bounded functions that may limit instability, but suffer from vanishing gradients which leads to slow convergence and unstable learning. More recent activation functions such as Leaky ReLU [76] and ELU [16], are designed to balance gradient flow and prevent instability by reducing output bias and thereby approach Fisher ideal learning. Smooth activation functions have also been observed to promote the learning stability of a mode, as demonstrated by Zheng et al [118] and Ramachandran et al [90].

The vanishing gradient and exploding gradient problems directly impact the stability of deep neural networks by making weight updates during training inefficient or unstable. In the vanishing gradient problem, small gradients in deep layers prevent effective weight updates, causing the network to learn slowly or stop learning altogether. On the other hand, exploding gradients cause large weight updates, leading to oscillations and divergent training. By addressing these issues with activation functions that mitigate both underflow and overflow, deep networks maintain stable learning, ensuring consistent weight updates and convergence.

Activation functions that exhibit near-zero expected output contribute to neural network stability by reducing output bias, ensuring that neurons do not consistently produce large positive or negative values. This mitigates the risk of saturating activation functions like sigmoid or tanh, where extreme outputs lead to vanishing gradients. By centering activations around zero, these functions allow for more balanced weight updates during backpropagation. This behavior aligns with Fisher’s Ideal Learning, which suggests that stable learning arises when gradients provide unbiased information about the parameter space, facilitating efficient and consistent training.

Table 5.6.1: Stability Heuristics of Activation Functions. Summary of heuristics on which properties of an activation function most impact the learning stability of a model. 'Nesting' quantifies the depth of nested non-linearities within each activation function which may be adverse to numerical stability. 'Vanishing' informs if a function saturates towards deactivation at an exponential rate or faster, resulting in more common vanishing gradients. 'Smooth' summarizes whether a function is differentiable at all points. 'Output Bias' measures the expected output of the activation function, given input taken from a standard Gaussian distribution.

Function	Nesting	Vanishing	Smooth	Output Bias
TeLU	2	No	Yes	0.2621
ReLU	1	Yes	No	0.3989
ELU	2	Yes	No	0.1605
SiLU	2	No	Yes	0.2066
GELU	2	Yes	Yes	0.2821
Mish	3	No	Yes	0.2404
Logish	2	No	Yes	0.1398
Smish	3	No	Yes	0.1201

Activation functions that are smooth and analytic contribute to learning stability in neural networks by ensuring that gradients change gradually and predictably during training. The smoothness of these functions avoids sudden jumps in gradient values, which can destabilize learning by causing erratic weight updates. Their analytic properties enable continuous differentiability, which is compatible with second order optimization techniques that utilize the curvature of the loss landscape to promote stability of learning. These characteristics allow for more consistent convergence, especially in deep networks, by preserving valuable gradient information throughout the layers through dispersity of activation.

Activation functions that are simple to calculate, such as ReLU and Leaky ReLU, contribute to learning stability by reducing the computational complexity of training deep neural networks. Simpler calculations minimize the risk of numerical errors like underflow or overflow, which can occur when nonlinear operations are repeatedly applied for the activation of a single neuron. Once underflow or overflow emerges within a term of a complex function, outer functions are likely to be driven towards additional underflow or overflow. Subsequent layers may also propagate the numerical instability to all the outputs, directly impacting inference. By avoiding these issues, simple activation functions help maintain numerical precision and prevent cascading errors in gradient computations, ensuring more stable learning, especially in deep architectures.

We present a detailed summary of the heuristics that we believe most significantly influence the learning stability of a model, focusing on specific properties of activation functions. First, we consider the 'Nesting' column, which reflects the complexity introduced by the number of nested non-linearities within an activation function. Higher levels of nested non-linearities can adversely affect the numerical stability of a model, particularly as the network depth increases. Next, the 'Vanishing' column assesses the tendency of an activation function to approach saturation and deactivation. If a function's derivative exhibits exponential

decay as inputs grow towards $-\infty$, it can result in a greater occurrence of vanishing gradients, hindering effective backpropagation [96] and slowing down learning in deep networks. In ReLU, this effect is exaggerated with derivatives of 0 for any negative input in what is known as the dying ReLU problem. Lu et al [74] discuss the strong effects of the dying ReLU problem in deep and narrow architectures, which are often necessary for efficient learning of complex behavior. The asymptotic decay of each activation function as inputs grow negative can be viewed in Table 5.1.1 within section 5.1.

Our 'Smooth' column refers to the non-zero differentiability of the activation function across its entire input domain. A function that is continuously differentiable at all points promotes smoother gradient flow and reduces potential disruptions during the training process. ReLU and ELU are not smooth functions due to their linear definition at positive inputs. Meanwhile, the non-monotonic functions are all smooth, as they can be indefinitely differentiated at all points without degenerating to zero. Finally, the 'Output Bias' column evaluates the expected output of the activation function when the input follows a standard Gaussian distribution. This is evaluated as $\int_{-\infty}^{\infty} p(x) \cdot f(x) dx$, where $p(x)$ is the unit Gaussian distribution $\mathcal{N}(0, 1)$, or $\frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{x^2}{2}}$. $f(x)$ is the mathematical definition of each activation function. This measure helps us understand whether the function introduces an inherent bias in its outputs, which can affect weight initialization and, ultimately, learning dynamics. The closer a function's output bias is to zero, the more it enhances the model's convergence efficiency [66].

We do not explicitly list exploding gradients in the Table, as their behavior requires a more nuanced, detailed analysis. While the activation functions in our Table are primarily linear units, behaving linearly as inputs grow large, the real advantage of our smooth non-monotonic functions—TeLU, SiLU, GELU, Mish, Logish, and Smish—emerges at small positive inputs. These functions exhibit sub-linear growth, meaning their outputs increase more slowly than their inputs under these conditions. This sub-linearity is critical in controlling gradient behavior, as it naturally prevents the derivatives from becoming excessively large during backpropagation. By reducing the likelihood of gradients exploding, non-monotonic activation functions contribute significantly to greater learning stability. This stability is essential for deep networks, particularly when training with complex data or architectures that are prone to issues with gradient propagation. As exploding gradients can severely hinder convergence and result in unstable weight updates, choosing activation functions that naturally limit gradient magnitudes offers a practical solution to this widespread problem.

Mitigating exploding gradients is not only about achieving smoother training but also about improving the network's overall learning efficiency. When gradients are kept within a controlled range, the model can train without sudden fluctuations in the loss function, resulting in more stable and predictable weight updates. This stability helps the optimization process stay on course, leading to faster convergence and better

generalization to new, unseen data. Activation functions like TeLU and its non-monotonic counterparts play a crucial role in promoting this stability by naturally preventing the gradients from becoming excessively large. By employing smooth, analytic functions like TeLU, these architectures inherently avoid issues such as abrupt changes or sharp boundaries in the model's output. This makes them particularly well-suited for tasks where continuity and smoothness are critical, such as in control systems, robotics, or financial modeling, where precise, stable responses are essential for success.

This can be theoretically shown as follows:

Theorem 5.6.1. *Let $f(x) \in C(\mathbb{R}^n)$ be a continuous function. Suppose we approximate $f(x)$ using two architectures: one based on an analytic universal approximator $TeLU(x) = x \cdot \tanh(e^x)$, and the other based on the non-analytic activation function $ReLU(x) = \max(0, x)$. The analytic universal approximator provides the following mathematically provable advantages:*

1. *Smoothness and Continuity of Derivatives: The analytic universal approximator is infinitely differentiable, whereas ReLU is piecewise linear and not differentiable at $x = 0$.*
2. *Gradient Propagation Stability: In deep networks, the gradient propagation using analytic functions remains bounded and stable, whereas ReLU introduces points where gradients vanish or explode, leading to poor training dynamics.*
3. *Error Propagation in Deep Networks: Analytic functions provide smoother error propagation, leading to better control of gradient norms, while ReLU propagates discontinuous gradients, leading to unstable optimization.*

Proof. We will prove each point in turn.

1. *Smoothness and Continuity of Derivatives:*

As demonstrated in 5.5 $TeLU(x) = x \cdot \tanh(e^x)$, TeLU is analytic. In being analytic, TeLU may be defined as a convergence power series and is therefore infinitely differentiable at all points.

2. *Gradient Propagation Stability:*

Consider a deep network with L layers. Let $g^{(l)}(x)$ represent the gradient of the output with respect to the input at layer l , defined as:

$$g^{(l)}(x) = \frac{d}{dx} \left(W^{(l)} \cdot a^{(l-1)}(x) \right),$$

where $W^{(l)}$ is the weight matrix and $a^{(l-1)}(x)$ is the activation of layer $l - 1$.

Using an analytic activation function like $TeLU(x)$, the gradient propagation remains continuous and smooth:

$$g^{(l)}(x) = W^{(l)} \cdot \frac{d}{dx} TeLU(x) = W^{(l)} \cdot \left(\tanh(e^x) + x \cdot e^x \cdot \text{sech}^2(e^x) \right),$$

which is bounded and continuous for all $x \in \mathbb{R}$. The gradient never vanishes nor explodes, as the derivative of $TeLU(x)$ is smooth for all x .

In contrast, for $ReLU(x)$, the gradient at layer l is:

$$g^{(l)}(x) = W^{(l)} \cdot \frac{d}{dx} ReLU(x) = W^{(l)} \cdot 1(x > 0).$$

Where $1(x > 0)$ evaluates to 1 if $x > 0$ and 0 if $x \leq 0$. At $x = 0$, the gradient can be discontinuous or zero, leading to two major issues: 1. *Vanishing gradients* when a large number of neurons have negative input, causing the gradient to become zero and halting learning. 2. *Exploding gradients* in deep layers, especially when weights $W^{(l)}$ are large, causing sharp changes in the gradient due to the piecewise linear nature of ReLU.

Thus, analytic universal approximators offer more stable gradient propagation across deep networks.

3. Error Propagation in Deep Networks:

Let $\mathcal{L}(x)$ represent the loss function of the network, and let the gradient of the loss with respect to the input at the final layer be $\frac{\partial \mathcal{L}}{\partial x}$.

For the analytic activation $TeLU(x)$, the gradient propagation follows from the smooth nature of $TeLU$:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot \frac{d}{dx} TeLU(x) = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot (\tanh(e^x) + x \cdot e^x \cdot \text{sech}^2(e^x)).$$

The gradient is well-controlled due to the smooth behavior of $TeLU(x)$, ensuring that error propagation through layers remains stable and predictable.

For $ReLU(x)$, the error propagation becomes problematic:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot \frac{d}{dx} ReLU(x) = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot \mathbb{I}(x > 0).$$

This creates sharp, discontinuous changes in the error gradient when x crosses zero, leading to unpredictable error propagation. Furthermore, the gradient can become zero when large numbers of neurons are inactive (i.e., in the dead zone), effectively halting the learning process for those weights.

Thus, analytic activations offer more stable and predictable error propagation in deep networks.

□

Smoothness and continuity of derivatives, gradient propagation stability, and error propagation stability are crucial for ensuring the effective training of deep neural networks. Smooth activation functions with continuous derivatives allow for stable and predictable gradient updates, preventing erratic changes during

backpropagation. This, in turn, promotes gradient propagation stability, which is essential for maintaining useful gradient values across many layers, avoiding the problems of vanishing or exploding gradients. Stable error propagation further ensures that the network can consistently learn from its mistakes, leading to more reliable convergence and better generalization to unseen data. Together, these properties form the foundation of stable and efficient learning in deep networks.

Chapter 6:

Experimental Validation

This chapter presents a detailed experimental validation of the TeLU activation function, building on the theoretical foundation established in the previous chapter. Here, we provide empirical results that support and confirm TeLU’s proposed benefits, such as enhanced gradient behavior, improved convergence speed, computational efficiency, and stability. By testing TeLU across various neural network architectures and datasets, we aim to demonstrate its effectiveness in addressing challenges like inefficient learning, the vanishing gradient problem, and computational complexity. These experiments serve to bridge the gap between theory and practice, offering concrete evidence of TeLU’s potential as a superior alternative to traditional activation functions.

6.1 Persistent Gradients of the Saturation Region

In section 5.1, we examined the rate at which the derivatives of various activation functions vanish as inputs approach $x \rightarrow -\infty$. By categorizing each function’s vanishing gradient behavior using asymptotic decay classes, we identified TeLU as belonging to the class with the most persistent gradients, $\Theta(\frac{x}{e^x})$. Additionally, we noted that TeLU maintains stronger derivatives compared to other nonlinearities, particularly at the onset of its deactivation region.

6.1.1 FashionMNIST Dataset on MLP with Negative Biases

We demonstrate the benefit of these characteristics experimentally by attempting to train Multi-Layer Perceptrons (MLPs) where neurons have been initialized with a bias of -10. The large negative bias dictates the activation of the neuron towards the saturating region of lower-bounded nonlinearities. By introducing this large negative bias, we evaluate the ability of the architecture to recover from a point where backpropagations yields vanishing gradients. This extreme case lets us focus on the direct effects of each activation function’s asymptotic saturation rate. We utilize two hidden layers and train on the FashionMNIST dataset [22] for 200 epochs. We optimize according to the SGD optimizer with a momentum of 0.9 and a weight decay of 0.0005. This reduced hidden layer count effectively reduces the number of epochs that we would

Table 6.1.1: MLP FashionMNIST Negative-Bias Experiment Configuration. Configuration of experiments ran on an MLP model on the FashionMNIST dataset with biases initialized to -5 to test architecture ability to recover from vanishing gradients.

Hyperparameter	Value
Train/Val/Test Split	50,000 / 10,000 / 10,000
Hidden Layer Count	2
Hidden Layer Width	128
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD
Momentum	0.9
Batch Size	128
Learning Rate	0.005
Weight Decay	0.0005
Number of Epochs	200
Number of Trials	10

need to run this experiment to witness activation function recovery. We outline our configuration in table 6.1.1.

The presence of weight decay does not significantly impact recovery in practice, as the bias is what is keeping the neuron activations in their saturation regions. We display the validation accuracies over the 200 epochs in Figure 6.1.1 and testing accuracies in Table 6.1.2, with each data point averaged over 10 trials. We observe that TeLU tends to begin its successful stride towards recovery before any other nonlinearity, due to having stronger inactive gradients near the origin where expected inputs are likely to fall. We observe Mish, SiLU, Smish, and Logish begin their recovery sooner than ELU, GELU, and ReLU, which is what we would expect to see from our analysis. Furthermore, TeLU’s accuracy approaches a competitive level at a faster rate than do the competing activation functions.

We demonstrate the consistency of this experiment by initializing the bias learned parameters to -20, pushing neurons further into their saturation regions. We plot the validation accuracies for each epoch, averaged over 9 trials in Figure 6.1.2. We notice that TeLU continues to be the first to recover from vanishing gradients, giving further empirical evidence that it is less prone to stalling learning when TeLU neurons are pushed into their inactive regions. We also note that the order of each architecture begins their recovery in the same order as before, adding the relevance of the asymptotic decay classifications. Mish, SiLU, Logish, and Smish; belonging to the same asymptotic decay class as TeLU but having weaker gradients near the origin; being their recovery before ELU, GELU, and ReLU. We summarize our test accuracies averaged over 10 trials per data point for the 200 epoch experiments in Table 6.1.2. We recognize that within the 200 epochs TeLU architectures generalize to a superior testing accuracy in both cases.

For biases intialized to -20, we notice that 200 epochs may not be enough time for GELU to begin its recovery. Therefore we increase the number of epochs to 300 to gain further insight into the remainder

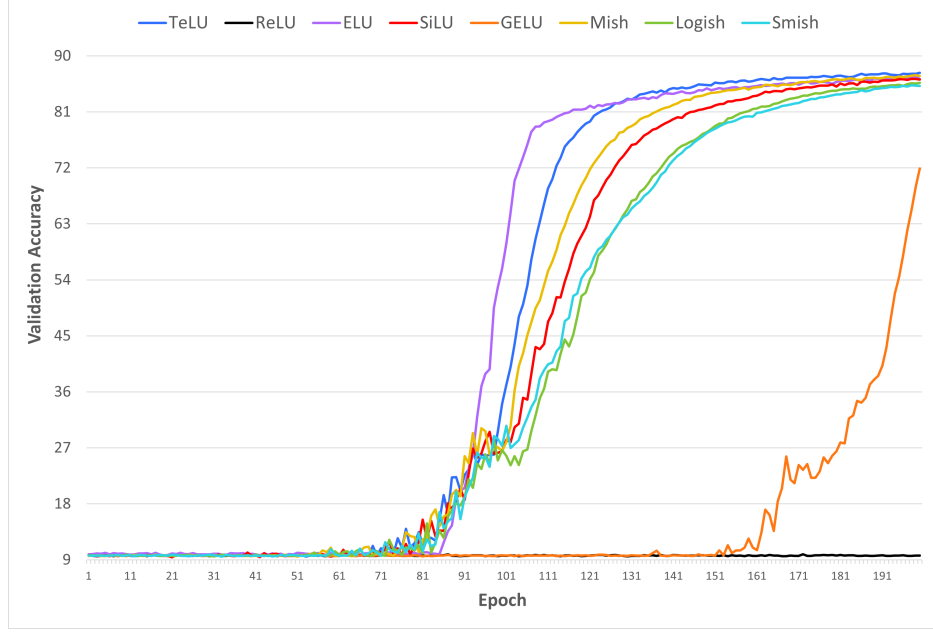


Figure 6.1.1: MLP FashionMNIST Light-Bias 200-Epoch Validation Accuracy. Validation accuracy progression of MLP models employing different hidden activation functions. Each architecture is initialized with bias values of -10 and trained over 300 epochs. Each data point is averaged over 10 trials.

of each architecture’s recovery. Figure 6.1.3 plots the 300 epoch journey of each architecture, averaged over 5 trials. Again, we see nonlinearities begin their recovery in the order of TeLU, Mish, SiLU, Logish, Smish, ELU, and GELU. It is worth noting that although the order of recovery follows our asymptotic classifications, the rate of recovery once begun depends on other factors we have not yet accounted for. We hypothesize that TeLU’s near-linearity and ELU’s linearity throughout positive inputs helps both shoot up to their concluding validation accuracies faster than other nonlinearities. Throughout all trials, ReLU exhibits no ability to recover, as is expected from its zero-gradient off state, which characterizes its dying neuron problem.

In practice, the Hyperbolic Tangent is generally preferred over the Logistic Sigmoid for preventing vanishing gradient problems when properly regularized. Similarly, we believe that TeLU and SiLU offer more persistent gradients with proper regularization, which may explain why Mish, Logish, and Smish are often considered to have a stronger self-regularizing effect. We capitalize the practical applications of slow gradient decay by initializing neuron biases in a multi-layer perceptron (MLP) to varying negative values. We then perform training on the MNIST datasets to test the ability of neural networks employing different hidden nonlinearities to reactivate their inactive neurons. To simplify results, we take nonlinearities that have shown a difference in their decay rates, numerical underflow regions, or inactive gradient calculations. Therefore, we use ReLU, ELU, GELU, Mish, and TeLU nonlinearities.

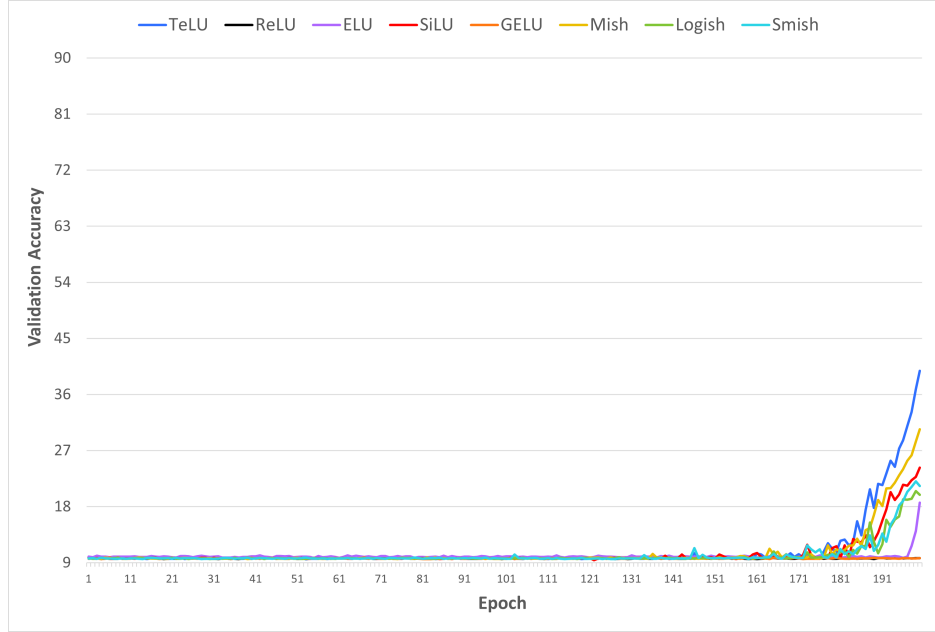


Figure 6.1.2: MLP FashionMNIST Heavy-Bias 200-Epoch Validation Accuracy. Validation accuracy progression of MLP models employing different hidden activation functions. Each architecture is initialized with bias values of -20 and trained over 200 epochs. Each data point is averaged over 10 trials.

Table 6.1.2: MLP FashionMNIST Negative-Bias Test Accuracy Averaged over 10 Trials. Testing accuracies of MLP models employing various hidden activation functions after being initialized to negative biases. This experiment tests the ability of each architecture to recover from vanishing gradients over the course of 200 epochs, as an indication of being able to continue learning effectively despite vanishing gradients.

$f(x)$	bias=-10	bias=-20
$TeLU(x)$	86.41 $\pm$ 0.37	39.98 $\pm$ 9.27
$ReLU(x)$	10.00 $\pm$ 0.00	10.0 $\pm$ 0.00
$ELU(x)$	85.58 $\pm$ 0.36	18.53 $\pm$ 15.6
$SiLU(x)$	85.31 $\pm$ 0.24	24.99 $\pm$ 3.18
$GELU(x)$	71.49 $\pm$ 5.25	10.0 $\pm$ 0.00
$Mish(x)$	86.09 $\pm$ 0.25	31.01 $\pm$ 2.65
$Logish(x)$	85.00 $\pm$ 0.69	20.04 $\pm$ 2.21
$Smish(x)$	84.606 $\pm$ 0.73	21.55 $\pm$ 1.68

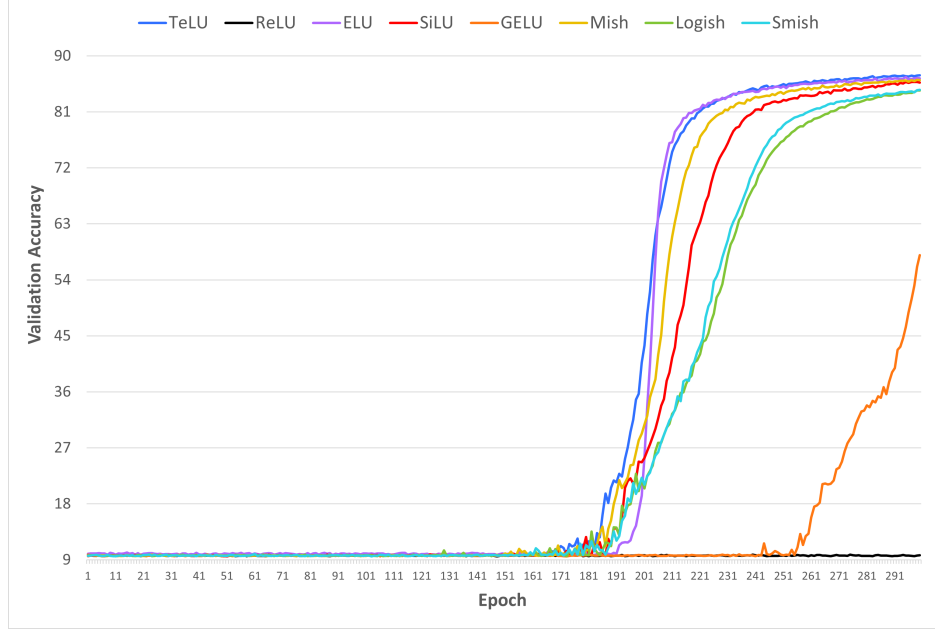


Figure 6.1.3: MLP FashionMNIST Heavy-Bias 300-Epoch Validation Accuracy. Validation accuracy progression of MLP models employing different hidden activation functions. Each architecture is initialized with bias values of -20 and trained over 300 epochs. Each data point is averaged over 5 trials.

6.1.2 CIFAR-10 Dataset with DenseNet CNN

We hypothesize that the pronounced vanishing gradient effects associated with GELU and ReLU activation functions may be particularly harmful in CNN architectures lacking bypass connections. CNNs process local information from input images through convolution, where small square filters are applied around each pixel. This local information is passed through multiple convolution layers, often combined with pooling operations to reduce the signal’s dimensionality. In architectures like DenseNet [52], fully connected neural blocks are interleaved between convolutional steps. Unlike Residual Networks (ResNet) [41], DenseNets do not have bypass connections to facilitate proper gradient flow to all layers. Consequently, vanishing gradients can significantly impede the learning process of such architectures. In extreme cases, this stalling can have lasting effects, ultimately reducing the model’s ability to fit the data. Therefore, we anticipate that activation functions prone to “dying neurons,” such as ReLU and GELU, may severely limit the performance of DenseNets.

To evaluate our hypothesis, we use a CNN without residual connections to compare the testing accuracies of architectures that employ TeLU, GELU, and ReLU as their hidden layer activations. Specifically, we implement TeLU, GELU, and ReLU versions of the DenseNet121 architecture [52] and train them on the CIFAR10 dataset [60] for 200 epochs each. The dataset is partitioned into 40,000 training images, 10,000 validation images, and 10,000 testing images. We use an SGD optimizer with a base learning rate of 0.1, along

Table 6.1.3: DenseNet CIFAR-10 Configuration. Static Configuration of experiment ran of the DenseNet architecture on the CIFAR-10 dataset.

Hyperparameter	Value
Train/Val/Test Split	40,000 / 10,000 / 10,000
Normalization	Standard Score, detailed in Table A.2.1
Architecture	DenseNet
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD with momentum
Batch Size	128
Weight Decay	0.0007
Base Learning Rate	0.1
Learning Scheduler Period	60
Learning Scheduler Decay	0.2
Data Augmentations	Random 4-padded cropping and horizontal flipping
Number of Epochs	200
Number of Trials	10

with a learning rate scheduler that reduces the learning rate by a factor of 0.2 every 60 epochs. Additionally, we apply L2 regularization with a weight decay coefficient of 0.0005 and perform mini-batch gradient descent [56] using a batch size of 128. Further experimental configurations are detailed in Table 6.1.3.

We observe the test accuracies of each architecture, averaged over 10 trials, in Table 6.1.4. We also view the progression of validation accuracy of the TeLU, GELU, and ReLU architectures, averaged across 10 trials, in Figure 6.1.4. We notice that TeLU, an activation function that features stronger mitigation of vanishing gradients in the deactivation region, performs significantly better than both the GELU and ReLU architectures. These results are consistent with those of Dubey et al [23], which highlight that residual connections are integral to the success of architectures employing GELU or ReLU activation functions.

Table 6.1.4 presents the test accuracies of each architecture, averaged over 10 trials, while Figure 6.1.4 shows the progression of validation accuracy for the TeLU, GELU, and ReLU architectures, also averaged over 10 trials. Notably, TeLU outperforms both GELU and ReLU, highlighting its advantage due to its persistent gradients in the deactivation region, which more effectively mitigate the vanishing gradient problem. This persistent gradient flow allows for more robust learning, especially in architectures lacking residual connections. These results are consistent with the findings of Dubey et al. [23], which highlight the necessity of residual connections for the success of architectures using GELU or ReLU activation functions. In contrast, TeLU effectively addresses the vanishing gradient and numerical underflow issues related to the dying neuron problem, eliminating the need for residual connections to maintain high performance. As a result, TeLU offers greater architectural flexibility by reducing dependencies on specific model configurations.

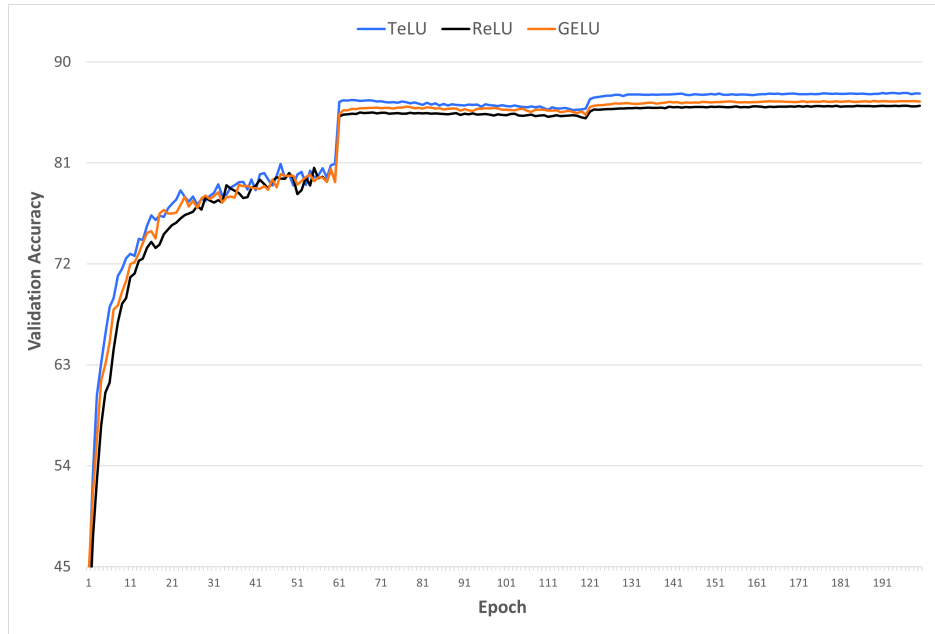


Figure 6.1.4: DenseNet CIFAR-10 Validation Accuracy Progression. Progression of validation accuracy of DenseNet121 architecture on CIFAR-10 dataset over 200 epochs, averaged over 10 trials per data point.

Table 6.1.4: DenseNet CIFAR-10 Accuracy Summary. Training, validation, and testing accuracies of DenseNet121 architecture on CIFAR-10, averaged over 10 trials.

Function	Train	Validation	Testing
TeLU	99.897	87.620 $\pm$ 0.197	86.806 $\pm$ 0.282
ReLU	99.910	86.669 $\pm$ 0.176	85.825 $\pm$ 0.279
GELU	99.946	86.243 $\pm$ 0.179	85.386 $\pm$ 0.200

6.2 Near-Linearity of Active Region

In Section 5.2, we investigated how effectively various linear units approximate a linear function within their active regions. This was done by calculating the integral difference between each activation function and the line representing its slope as $x \rightarrow \infty$. Among the smooth linear units, TeLU was found to minimize this integral difference. While both ReLU and ELU are defined as the identity for positive inputs, we argue that TeLU offers superior learning efficiency due to its persistent gradients, which help mitigate the vanishing gradient problem and prevent learning slowdowns.

6.2.1 ImageNet Dataset with ResNet18 Architecture

We first test the practical applicability of the theoretical improvement in convergence rate with the ImageNet dataset [21]. We directly implement Pytorch’s example ImageNet training on the ResNet18 architecture [41]. Residual Neural Networks (ResNets) are a type of Convolutional Neural Networks (CNNs) [64] that define bypass connections between hidden convolutional layers to treat effective model depth as a learnable parameter and provide strong gradients to earlier neural layers during backpropagation. We use the predefined base learning rate of 0.1, weight decay coefficient of 0.0001, and minibatch size of 256. We optimize according to Stochastic Gradient Descent (SGD) with a momentum of 0.9 over 90 epochs, dividing our learning rate by 10 at periods of 30 epochs. Due to the computational requirements and the size of the ImageNet dataset, we only ran experiments with the ReLU and TeLU nonlinearities. This configuration is detailed in Table 6.2.1. Our first trial was run with a seed of 1111, but the following two trials were unseeded due to performance drop. We plot the averaged validation accuracy metrics over the trials in Figure 6.2.1. Additionally, we show the test accuracy in Table 6.2.2. We observe that TeLU outperforms ReLU with baseline hyperparameters when training for 90 epochs.

Given that the unseeded trials still required 14 days to train on 4 GTX 1080 Ti Graphics Processing Units (GPU), we wish to take advantage of TeLU’s theoretical improvements in convergence speed. Therefore, we lower the number of training epochs from 90 to 50 and expect to see comparable results. To keep the learning rate scheduler roughly proportional, we lowered the learning rate decay period from 30 to 20. We show the average validation epoch accuracy metrics over the 50 epochs in Figure 6.2.2. We also note a comparable performance of both activation function architectures, but a more-unchanged TeLU accuracy in Table 6.2.2. We attribute this to TeLU’s distant saturation region helping the network stay relevant across forward passes as well as TeLU’s strong gradients for positive inputs.

Table 6.2.1: ResNet18 ImageNet Experiment Configuration. Configuration of experiments ran on a ResNet18 architecture training on the ImageNet dataset over 90 epochs.

Hyperparameter	Value
Train/Val Split	1,281,167/50,000
Normalization	Standard Score, detailed in Table A.2.1
Architecture	ResNet18 [41]
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD
Momentum	0.9
Batch Size	256
Learning Scheduler Period	30
Learning Scheduler Decay	0.1
Learning Rate	0.01
Weight Decay	0.0001
Number of Epochs	90, 50, or 20
Number of Trials	1, 3, or 3 in respect to number of epochs

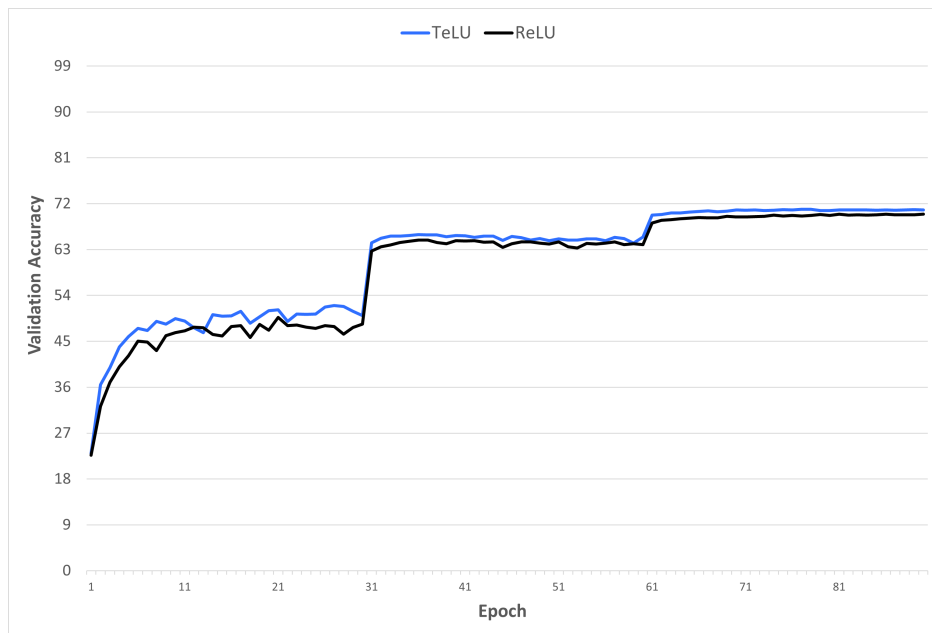


Figure 6.2.1: ResNet18 ImageNet Validation Accuracy over 90 Epochs. Progression of validation accuracy of ResNet34 architectures employing either TeLU or ReLU nonlinearities over the course of 90 training epochs.

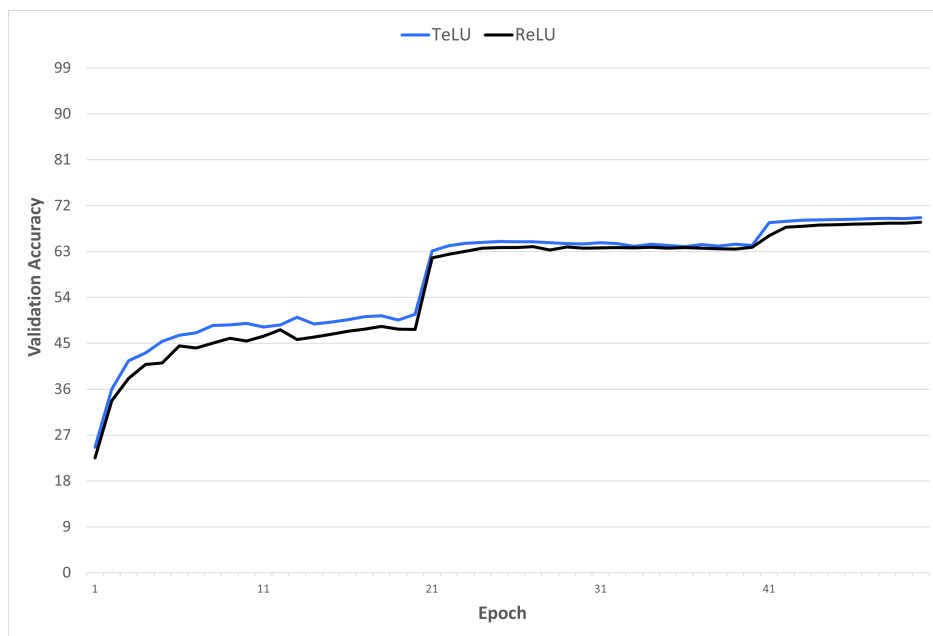


Figure 6.2.2: ResNet18 ImageNet Validation Accuracy over 50 Epochs. Progression of validation accuracy of ResNet34 architectures employing either TeLU or ReLU nonlinearities over the course of 50 training epochs. Each data point is averaged over 3 trials.

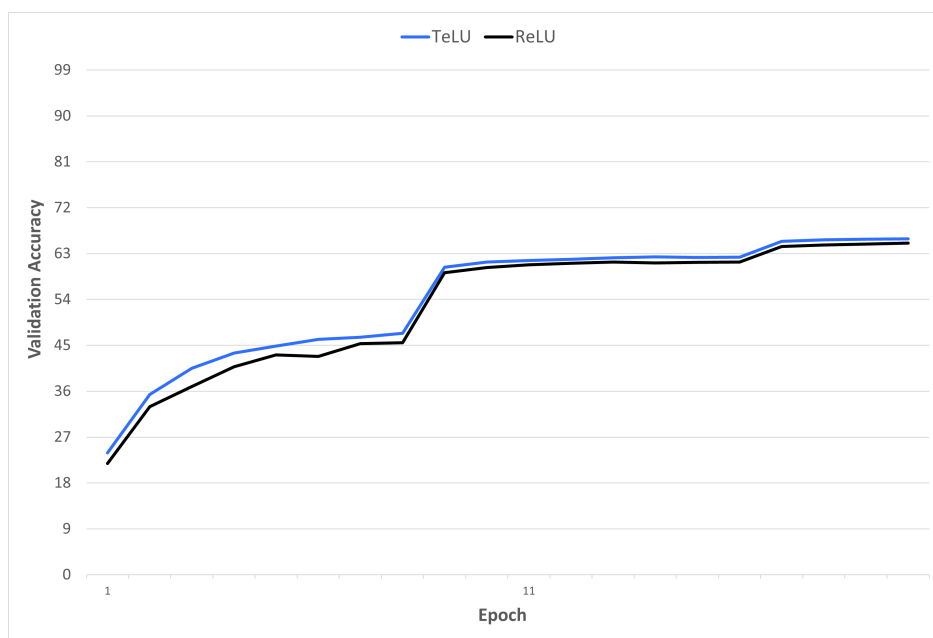


Figure 6.2.3: ResNet18 ImageNet Validation Accuracy over 20 Epochs. Progression of validation accuracy of ResNet34 architectures employing either TeLU or ReLU nonlinearities over the course of 20 training epochs. Each data point is averaged over 3 trials.

Table 6.2.2: ResNet18 ImageNet Test Accuracy Summary. Top-1 and top-5 test accuracies for ResNet34 architectures employing either TeLU or ReLU nonlinearities. Experiment was repeated for 90, 50, and 20 epochs to demonstrate the convergence speed advantages of TeLU. Each testing accuracy is averaged over 3 trials for the 50 epoch and 20 epoch experiments.

Name	Top-1 Accuracy %	Top-5 Accuracy %
TeLU 90 Epoch	70.86	89.70
ReLU 90 Epoch	69.96	89.27
TeLU 50 Epoch	69.76 $\pm$ 0.095	89.16 $\pm$ 0.063
ReLU 50 Epoch	68.736 $\pm$ 0.092	88.47 $\pm$ 0.006
TeLU 20 Epoch	65.87 $\pm$ 0.044	86.67 $\pm$ 0.107
ReLU 20 Epoch	65.08 $\pm$ 0.041	86.24 $\pm$ 0.082

Table 6.2.3: Dynamic-Pooling Transformer Text8 Experiment Configuration. Configuration of experiments ran on a dynamic-pooling transformer architecture on the Text8 dataset.

Hyperparameter	Value
Train/Val/Test Split	$90 \cdot 10^6 / 5 \cdot 10^6 / 5 \cdot 10^6$
Architecture	Dynamic-Pooling Transformer [84]
Model Dimension	512
Number of Heads	8
Head Dimension	64
dropout	0.06
computational precision	Floating Point 16
gradient clipping	0.25
Initialization	Gaussian with 0.02 average and 0.01 stdv
Optimizer	Mini-batch Adam
Batch Size	16
Base Learning Rate	0.0002
Learning Scheduler	Cosine Scheduler [73]
Warm-Up Steps	4000
Learning Scheduler Period	150,000 or 200,000
Mini-Batch Steps	100,000 or 200,000
Number of Trials	3 or 9, in respect to mini-batch steps
Random Seed	[0000, 1111, ..., 9999]

6.2.2 Text8 Dataset with Dynamic Pooling Transformer

We then proceeded to test TeLU’s baseline convergence rate on a transformer [109] architecture. For performance improvements, we utilized a dynamic pooling transformer architecture [84] with 8 heads of attention. We set the fully connected and attention neurons to drop out at a rate of 0.12 during training. We utilized a base learning rate of 0.0002 that updated according to a 200,000 step cosine scheduler with 4,000 initial warm-up steps. We used the ADAM optimizer [59], given that we did not employ weight decay. We detail our configuration in Table 6.2.3. With this configuration, we ran 3 trials and recorded the averaged progression of validation error metrics over the 20 evaluation steps in Figure 6.2.4.

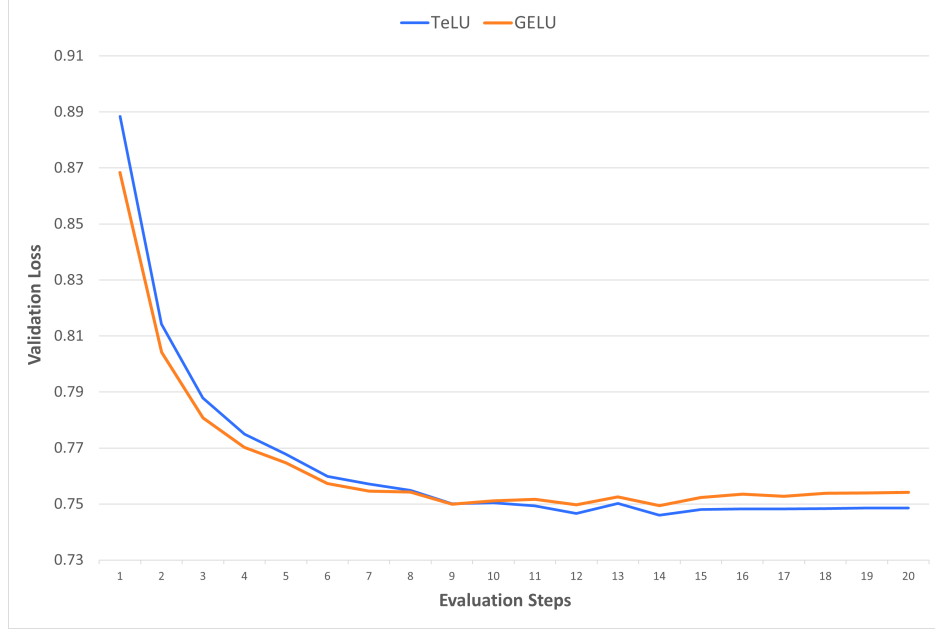


Figure 6.2.4: Dynamic-Pooling Transformer Text8 Validation Loss. Shows the progression of validation Loss throughout 200,000 mini-batches of the Dynamic-Pooling Transformer architecture being trained on the Text8 dataset with the Mini-batch Adam optimizer. Each data point has been averaged over 3 trials.

Table 6.2.4: Dynamic-Pooling Transformer on Text8 Test Loss. Shows the progression of validation Loss throughout 200,000 mini-batches of the Dynamic-Pooling Transformer architecture being trained on the Text8 dataset with the Mini-batch Adam optimizer. Each data point has been averaged over 9 trials.

Name	100 Epoch	200 Epoch
TeLU	0.7985 $\pm$ 0.00089	0.7970 $\pm$ 0.00078
ReLU	0.7998 $\pm$ 0.00055	-
GELU	0.7999 $\pm$ 0.0027	0.8034 $\pm$ 0.0023

As shown in Figure 6.2.4, GELU initially achieves lower validation loss during the early evaluation steps. However, TeLU’s faster convergence allows its loss to drop below GELU’s over time. Notably, TeLU’s validation loss stabilizes by evaluation step 15, whereas GELU begins to experience increasing validation error beyond step 14. These validation improvements with TeLU translate into superior test loss, as seen in Table 6.2.4. Furthermore, TeLU’s stability at favorable validation accuracy results in lower standard deviation in test loss compared to GELU.

To demonstrate TeLU’s improved convergence rate further, we reduced the training steps from 200,000 to 100,000 and updated the cosine scheduler to a period of 150,000. This was empirically chosen to allow the architectures to continue applying meaningful updates during the last few training steps. With a shorter experiment, our budget now allows for a ReLU architecture to be included. We view the progression of the validation error in Figure 6.2.5 and the test error summary in Table 6.2.4, with data averaged over 9 trials.

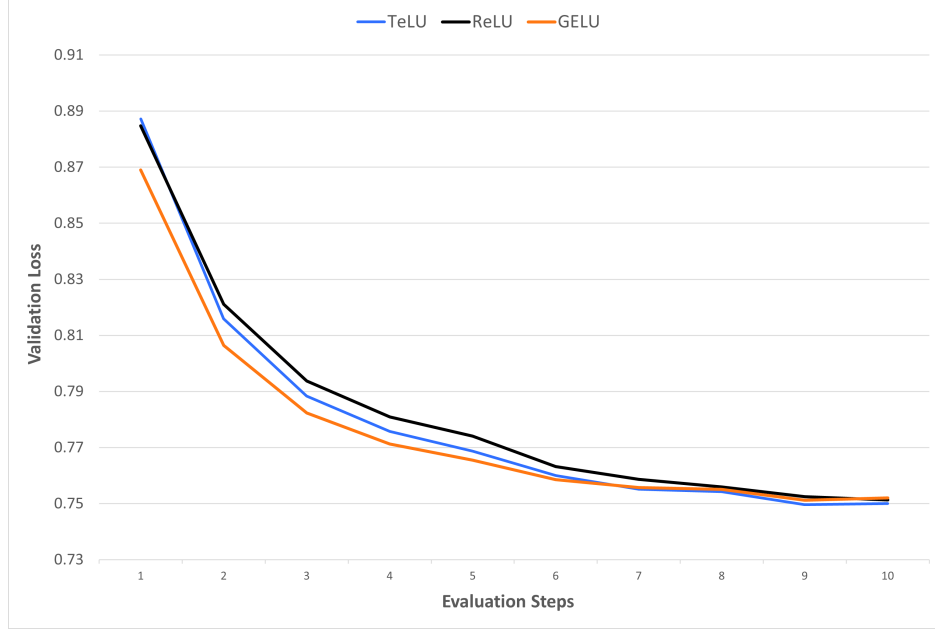


Figure 6.2.5: Dynamic-Pooling Transformer on Text8 Validation Loss Short Experiment. Shows the progression of validation Loss throughout 100,000 mini-batches of the Dynamic-Pooling Transformer architecture being trained on the Text8 dataset with the Mini-batch Adam optimizer. Each data point has been averaged over 9 trials.

The shortening of the cosine scheduler period [73] enables TeLU to outperform GELU at earlier evaluation steps, as anticipated. ReLU, like GELU, displays strong gradients in the active region but converges more slowly. This slower convergence can likely be attributed to the dying neuron problem, which inhibits learning in ReLU architectures. As a result, both ReLU’s dying neuron issue and GELU’s sub-linearity in the active region contribute to the slower and less consistent convergence observed.

6.3 Runtime Efficiency

Using the function definitions provided in Table 5.3.1, we measured the computational efficiency of forward and backward passes for a neuron employing the baseline implementation of each nonlinearity. To ensure accurate delay measurements, we first minimized operating system scheduler disturbances [2] [68] by running only essential Windows operating system programs on the system in addition to the experiment itself. Our host system had 12 Intel i7 processors, 32 GB of RAM, and an NVIDIA RTX-2070 Graphics Processing Unit (GPU) [75]. We defined an input vector of 10^6 randomly initialized 32-bit floating point values and performed a sequence of forward and backward passes over 10^6 iterations. This approach helped evenly distribute any remaining operating system interrupts across each nonlinearity’s measurement. We repeated the experiment after its completion to ensure that the sequential execution of each function’s forward and

Table 6.3.1: Timing of 10^6 Iterations of Single Neuron with Input Size of 10^6 on RTX2070. Delay in seconds of 1 million forward and backward passes of a single-neuron architectures with each of the focused non-linearities. Experiment executed in system utilizing NVIDIA Geforce 2070. Input Size totaled over 1 million iterations.

Function	Forward	Backward
TeLU	139.56s	322.35s
ReLU	107.75s	299.77s
ELU	167.64s	326.48s
SiLU	187.80s	535.00s
GELU	210.27s	457.67s
Mish	205.23s	380.03s
Logish	244.66s	574.30s
Smish	285.02s	621.04s

Table 6.3.2: Timing of 10^6 Iterations of Single Neuron with Input Size of 10^6 on Server GPUs. Delay in seconds of 1 million forward and backward passes of a single-neuron architectures with each of the focused non-linearities. Experiment repeated for systems utilizing A100 and 1080Ti GPUs. Input Size totaled over 1 million iterations.

	A100 GPU		1080Ti GPU	
Function	Forward	Backward	Forward	Backward
TeLU	77.337s	150.98s	119.35s	279.83s
ReLU	65.356s	148.46s	96.525s	248.87s
ELU	103.00s	171.44s	156.60s	281.62s
SiLU	97.036s	195.08s	162.31s	444.18s
GELU	114.06s	209.38s	174.27s	387.38s
Mish	108.41s	170.07s	173.37s	315.67s
Logish	120.66s	211.11s	208.95s	479.72s
Smish	137.34s	222.90s	233.01s	515.59s

backward passes did not provide any bias to our experiment. This repeat experiment confirmed that system performance factor differences between the times of each nonlinearity’s calculations were negligible. The delay of each activation function is recorded in Table 6.3.1, where we observed that TeLU’s performance was second only to ReLU’s.

We repeated the baseline definition experiment on two Linux systems, each equipped with 64 GB of RAM. One system featured an A100 GPU, while the other used a TI-1080 GPU. This time, we used input vector of length 10^6 and 10^7 to ensure that each GPU would require multiple operations to process the larger 32-bit floating point input vector. The timing comparisons between these experiments was proportional, as seen in Tables 6.3.2 and 6.3.3 indicating minimal variance in nonlinearity delays between the two systems. The concise formulation of TeLU allows for significant computational efficiency improvements over other smooth functions, making TeLU-based architectures more scalable and leading to reduced training times. This speedup in training suggests not only an improvement in TeLU’s convergence speed but also enhanced energy efficiency during operation.

Table 6.3.3: Timing of 10^6 Iterations of Single Neuron with Input Size of 10^7 . Delay in seconds of 10 million forward and backward passes of a single-neuron architectures with each of the focused non-linearities. Experiment repeated for systems utilizing A100 and 1080Ti GPUs. Input Size totaled over 1 million iterations.

	A100 GPU		1080Ti GPU	
Function	Forward	Backward	Forward	Backward
TeLU	206.54s	517.10s	771.53s	2032.20s
ReLU	127.60s	453.21s	442.32s	1719.73s
ELU	251.17s	505.88s	926.03s	1943.89s
SiLU	282.00s	872.09s	1074.99s	3464.10s
GELU	313.81s	757.55s	1188.41s	2953.31s
Mish	311.79s	593.34s	1181.49s	2324.54s
Logish	386.72s	947.87s	1491.52s	3775.24s
Smish	439.14s	1023.43s	1700.14s	4086.84s

Table 6.3.4: Timing of 10^7 Iterations of Single Neuron with Input Size of 10^6 . Delay in seconds of 1 million forward and backward passes of a single-neuron architectures with each of the focused non-linearities. Experiment repeated for systems utilizing A100 and 1080Ti GPUs. Input Size totaled over 10 million iterations.

	A100 GPU		1080Ti GPU	
Function	Forward	Backward	Forward	Backward
TeLU	772.77s	1511.7s	1176.7s	2786.8s
ReLU	661.14s	1489.03s	966.6s	2509.2s
ELU	1041.3s	1706.5s	1556.9s	2820.8s
SiLU	973.83s	1952.3s	1707.4s	4547.6s
GELU	1124.1s	2102.2s	1707.5s	3864.6s
Mish	1084.0s	1694.9s	1865.3s	3287.5s
Logish	1230.7s	2099.5s	2080.2s	4802.1s
Smish	1346.7s	2242.4s	2311.0s	5148.4s

This experiment was repeated for 10^7 iterations of input vectors of size 10^6 , with results shown in Table 6.3.3. Across all cases, TeLU demonstrates substantial performance improvements over all nonlinearities except ReLU. Although ReLU is faster to compute, TeLU compensates with faster convergence, achieving high performance with fewer training iterations. These findings are particularly meaningful, as they highlight TeLU’s superior computational efficiency compared to other smooth activation functions. This efficiency plays a critical role in deep neural networks because activation functions heavily influence the overall training speed. Faster training times not only enhance productivity but also enable the use of larger models, increased data throughput, and more frequent experimentation. Moreover, computationally efficient activation functions can reduce hardware demands, lower energy consumption, and facilitate real-time applications, making them vital for scalable and sustainable AI solutions.

6.4 ReLU Compatibility

Optimizing hyperparameter selection for training deep learning models is a time-intensive process. The time required can span weeks or even months, depending on the number of hyperparameters involved, the complexity of the architecture, and the duration of the training. The challenge arises from the fact that hyperparameter values can impact model accuracy, overfitting, convergence speed, and stability in nonlinear and interdependent ways. Furthermore, when designing experiments from scratch, each new setup demands its own lengthy tuning process, as we encountered in our initial experiments.

To enhance our productivity and streamline the process of finding competitive configurations, we began utilizing hyperparameter settings sourced from GitHub repositories or published papers. These configurations were primarily designed for ReLU-based architectures. Fortunately, we discovered that TeLU remained competitive without requiring any modifications to these ReLU configurations. In practice, adapting ReLU architectures to TeLU architectures proved to be straightforward—simply by defining the TeLU nonlinearity and substituting it for the ReLU nonlinearity in the hidden layers. This observed compatibility suggests that ReLU and TeLU perform well within similar hyperparameter subspaces, which could facilitate the adoption of TeLU. Given that ReLU is arguably the most popular activation function for hidden neurons, it is likely that data scientists will initially test a novel nonlinearity within an existing ReLU architecture using ReLU configurations. Therefore, TeLU’s similarity to ReLU across different architectures could help it make a positive first impression.

Table 6.4.1: MLP FashionMNIST Across Weight Decays Configuration. Configuration of experiments ran on an MLP model on the FashionMNIST dataset across various weight decay coefficient values.

Hyperparameter	Value
Train/Val/Test Split	50,000 / 10,000 / 10,000
Hidden Layer Count	8
Hidden Layer Width	128
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD
Momentum	0.9
Batch Size	128
Learning Rate	0.005
Weight Decay	[0.0001, 0.0003, 0.0005, 0.001, 0.003, 0.005]
Number of Epochs	100
Number of Trials	6 trials per weight decay

6.4.1 FashionMNIST Dataset with MLP with Varied Weight Decay

To experimentally demonstrate this compatibility, we designed MLP architectures utilizing various activation functions, including ReLU, ELU, GELU, SiLU, Mish, Logish, Smish, and TeLU. Each architecture was defined with 8 hidden layers, each with a width of 128 neurons. The batch size was set to 128, and the learning rate was fixed at 0.005. We used the Fashion MNIST dataset, dividing it into training, validation, and testing partitions with 50,000; 10,000; and 10,000 samples, respectively. Each configuration was tested over 6 trials across 6 different weight decay values: 0.0001, 0.0003, 0.0005, 0.001, 0.003, and 0.005. We chose to focus on weight decay because these activation functions seemed to differ most in their degree of self-regularization. The complete experimental configuration can be viewed in Table 6.4.1. The resulting accuracies for each configuration are presented in Figure 6.4.1.

It is observed that TeLU exhibits superior accuracy when using a weight decay coefficient that is optimal for ReLU. ReLU and TeLU nonlinearities generally benefit from higher levels of L2 regularization, while Logish and Smish require minimal or no weight decay to maintain stability. ReLU likely benefits more from increased weight decay due to its tendency to cause neurons to become permanently inactive during training. A larger weight decay helps mitigate the dying ReLU problem by keeping preactivations close to zero, which allows neurons to stay near the threshold needed to activate or deactivate based on different input signals. In contrast, nonlinearities with complex nested subfunctions, such as Smish, experience instability at weight decay levels that are beneficial for ReLU. When external regularization pushes the activation of preceding Smish neurons towards zero, subsequent layers receive inputs close to zero. This can lead to numerical underflow as the nested nonlinearities are calculated from the inside out, resulting in dying neurons that cause subsequent layers to produce zero activation, further propagating numerical instability. Overall, we

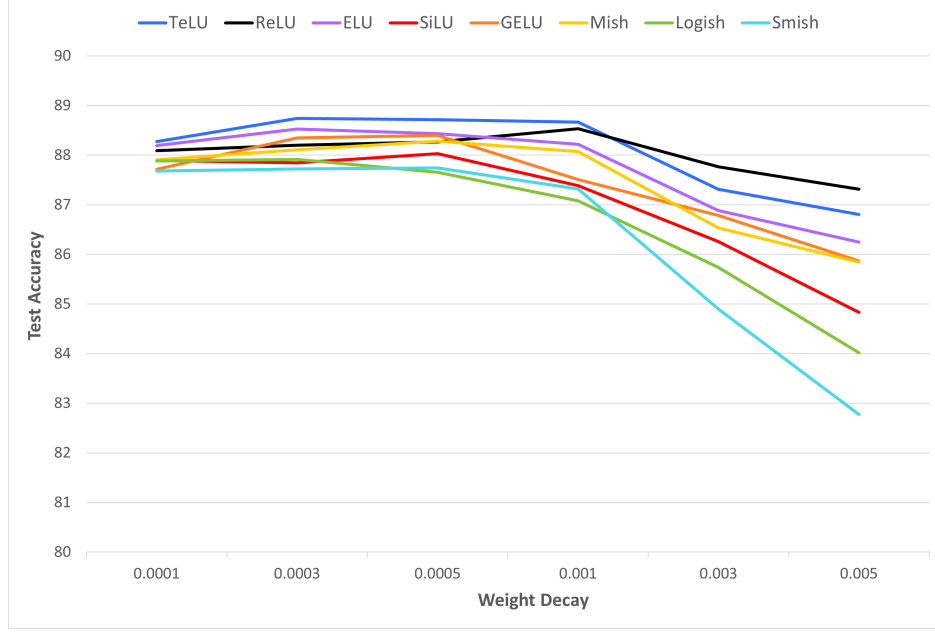


Figure 6.4.1: MLP FashionMNIST Test Accuracy Across Wide Range of Weight Decays. Visualizing the testing accuracy of different MLP architectures when the weight decay coefficient of their loss function is treated as an independent variable. Wider weight decay set of 0.0001, 0.0003, 0.0005, 0.001, 0.003, 0.005 shows broad spectrume of the testing accuracy of different MLP architectures.

observe a strong correlation between the mathematical complexity of a nonlinearity and its ideal level of L2 regularization, with the exception of SiLU and ELU.

We repeat this experiment for weight decay coefficient values 0.0004, 0.0006, 0.0008, 0.001, 0.0012, 0.0014, and 0.0016. As shown in Figure 6.4.2, the same key observation persists: where ReLU performs optimally, TeLU outperforms it. This is significant because ReLU is widely adopted in deep neural networks, meaning many existing projects rely on its configuration. With TeLU acting as a straightforward substitute for ReLU, researchers and developers can seamlessly switch to TeLU and immediately benefit from faster convergence and enhanced stability without requiring additional tuning, leading to greater productivity and minimal adjustment effort.

6.4.2 CIFAR-10 Dataset with SqueezeNext with Varied Optimizers

To further validate the compatibility between ReLU and TeLU configurations, we evaluated the performance of various nonlinearities when trained with hyperparameters optimized for ReLU. We utilized a SqueezeNext CNN architecture with ReLU as the activation function in its hidden layers. We then fine-tuned the learning rate, weight decay, and learning rate scheduler scaling hyperparameters for each of four optimization methods. We utilized Minibatch Stochastic Gradient Descent (SGD) [57, 92], SGD with momentum [71], AdamW [72], and RMSprop [38]. The resulting learning rate and weight decay hyperparameters for

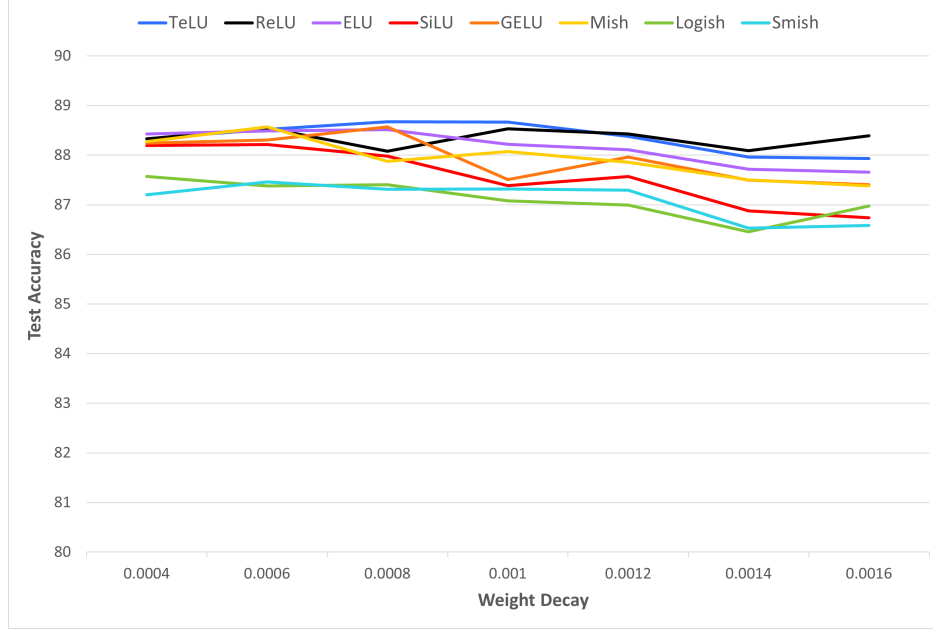


Figure 6.4.2: MLP FashionMNIST Test Accuracy Across Focused Range of Weight Decays. Visualizing the testing accuracy of different MLP architectures when the weight decay coefficient of their loss function is treated as an independent variable. Narrower weight decay set of 0.0004, 0.0006, 0.0008, 0.001, 0.0012, 0.0014, 0.0016 meant to focus on configurations that optimize ReLU and TeLU hidden non-linearities. Each data point shown is averaged over 6 trials.

each optimizer are detailed in 6.4.3. The portion of the hyperparameters that stay constant throughout optimizer choice is detailed in Table 6.4.2. The learning rate scheduler for each optimizer scaled the learning rate according to the learning decay value once every 60 epochs. For the experiment, we set aside 10,000 of the 60,000 CIFAR-10 images to define a validation partition, trained on the remaining 50,000 images, and tested on the standard 10,000-image testing split. We then substituted ReLU with the other nonlinearities and summarized the testing accuracies for each architecture over 10 trials in Table 6.4.4.

We observe that in the momentum, AdamW, and RMSprop optimizer-oriented ReLU configurations, TeLU performs presents a significant improvement over ReLU. This reinforces our ongoing observation that TeLU tends to be superior over ReLU in configurations that optimize ReLU performance. The SGD-oriented ReLU configuration presented the only case where ReLU performed best out of all activation functions. We note that the SGD optimizer was the most challenging to tune due to their smaller viable hyperparameter subspaces, yet it yielded the highest accuracies for the architecture being optimized once an adequate tuning was found. This behavior regarding SGD is common [119], and may help explain why TeLU did not outperform ReLU in this context. Since the success of SGD is particularly inconsistent between similar configurations, it is expected that the substitution of activation functions within an architecture

Table 6.4.2: SqueezeNext CIFAR-10 ReLU-Focused Configuration. Static Configuration of experiments ran on the SqueezeNext architecture on the CIFAR-10 dataset across various optimization algorithms.

Hyperparameter	Value
Train/Val/Test Split	45,000 / 5,000 / 10,000
Normalization	Standard Score, detailed in Table A.2.1
Architecture	SqueezeNext [32]
Initialization	Xavier Uniform
Batch Size	128
Learning Scheduler Period	60
Data Augmentations	Random 4-padded cropping and horizontal flipping
Number of Epochs	200
Number of Trials	5 per optimizer

Table 6.4.3: SqueezeNext CIFAR-10 Optimizer-Dependent Hyperparameters. Learning rate, weight decay coefficient, and learning rate scheduler decay hyperparameter values which vary across optimizer choice. Each hyperparameter value has been tuned to optimize the performance of SqueezeNext architectures employing the ReLU hidden non-linearity.

Optimizer	learning rate	weight decay	learning decay
SGD	0.1	0.003	0.2
Momentum	0.1	0.0007	0.2
AdamW	0.005	0.005	0.2
RMSprop	0.0002	0.005	0.4

Table 6.4.4: SqueezeNext CIFAR-10 Test Accuracy with Optimizers. average test accuracy of the SqueezeNext architecture training over 200 epochs on the CIFAR-10 dataset across various optimization algorithms. Each data point has been averaged over a minimum of 5 trials.

Name	SGD	Momentum	AdamW	RMSprop
TeLU	91.40±0.11	90.96±0.29	91.22±0.07	89.86±0.28
ReLU	91.84±0.33	90.77±0.16	90.31±0.44	88.59±0.14
ELU	90.96±0.24	90.05±0.42	90.94±0.15	89.45±0.39
SiLU	78.61±6.3	84.10±1.1	90.24±0.38	66.00±1.3
GELU	88.42±0.28	89.33±0.24	90.37±0.11	80.68±1.2
Mish	89.87±0.21	90.04±0.25	90.85±0.30	87.39±0.17
Logish	61.44±29	66.10±3.6	86.20±1.1	43.20±19
Smish	77.28±3.0	68.60±2.2	82.54±1.0	66.91±2.3

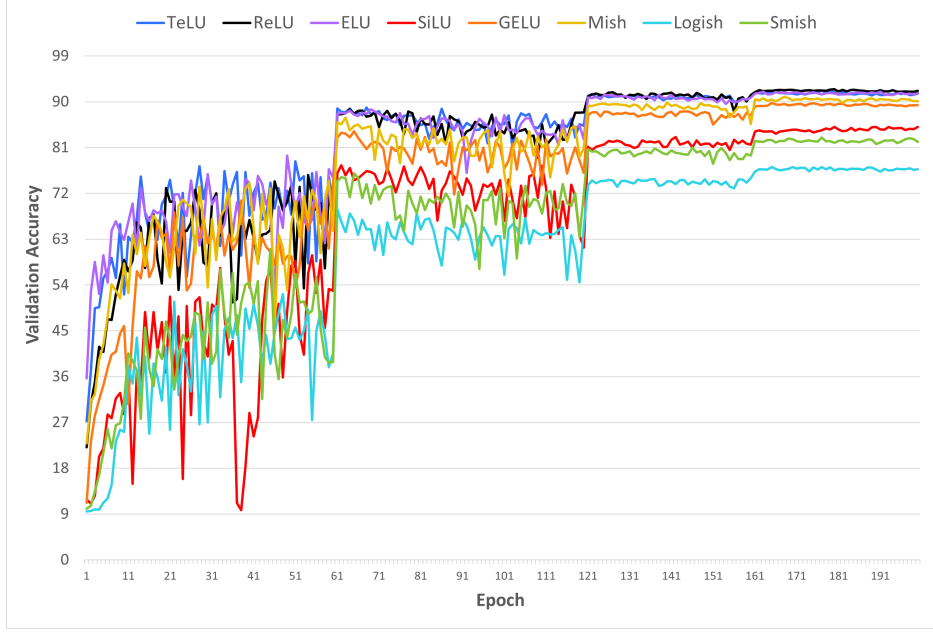


Figure 6.4.3: SqueezeNext on CIFAR-10 with SGD Validation Accuracy. Shows the progression of validation accuracy throughout 200 epochs of the SqueezeNext architecture being trained on the CIFAR-10 dataset with the Mini-batch SGD optimizer. Each data point has been averaged over 5 trials.

adds some additional tuning costs. TeLU, however, retains comparable performance with ReLU in the SGD case and shows significant improvement over all other activation functions.

To visualize the empirical similarity between TeLU and ReLU further, we plot the progression of validation accuracies in each architecture while optimizing according to the SGD and momentum-accelerated SGD optimizers in Figures 6.4.3 and 6.4.4. We find that architectures employing each nonlinearity learn in consistent patterns in both cases. ReLU, having little to no empirical self regularization according to our observations so far, benefits from large weight decays. Therefore, we see that more self-regularized activation functions cause their architecture’s validation accuracy to saturate to a lower value in this high regularization context. We also notice that TeLU and ELU architectures initially converge at a faster rate than their competition, similar to the persistent gradient experiments. We hypothesize that this occurs for the same reasons of having strong gradients in the unbounded region of activation without suffering from dying neuron problems. Eventually; ReLU, TeLU, and ELU architectures saturate to a similar validation accuracy and TeLU generalizes to a superior test accuracy.

We emphasize for these ReLU-similarity experiments that all experiments are done with configurations that are meant to favor the learning of ReLU architectures. This way, an activation function’s empirical similarity to ReLU in terms of viable configurations can be highlighted. Based on the resulting observations, we conclude that TeLU and ReLU share similar ideal hyperparameter subspaces, allowing them to perform

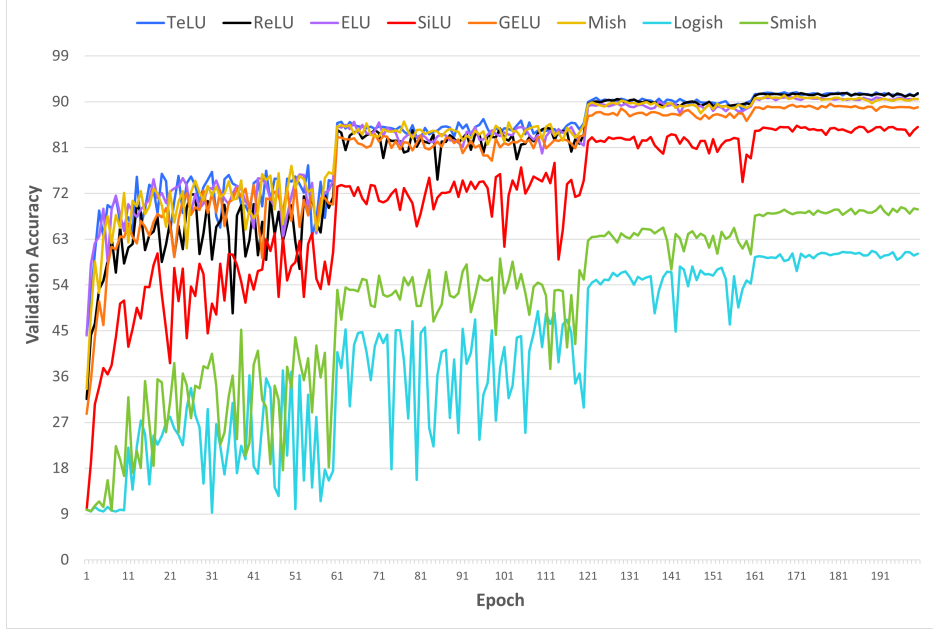


Figure 6.4.4: SqueezeNext on CIFAR-10 with Momentum-Accelerated SGD Validation Accuracy. Shows the progression of validation accuracy throughout 200 epochs of the SqueezeNext architecture being trained on the CIFAR-10 dataset with the momentum-accelerated Mini-batch SGD optimizer. Each data point has been averaged over 10 trials.

competitively after training with the same configurations. This suggests that architectures and tuning strategies optimized for ReLU can also be effectively applied to TeLU, minimizing the cost of substituting the hidden nonlinearities. In practice, we found that TeLU performs well with ReLU hyperparameters but may achieve even better results with a 20% reduction in weight decay and learning rate. Conversely, ReLU appears to perform optimally with a 25% increase in weight decay and learning rate when applied to a TeLU configuration.

6.5 Analytic Universal Approximation

6.5.1 MNIST Dataset with Variational AutoEncoder

To highlight the practical relevance of TeLU being an analytic universal approximation, we compare its training and testing loss to that of ReLU on the data compression and expansion of MNIST samples. A Variational AutoEncoder (VAE) [58] architecture is used with encoder layers of size 784, and 512, and 32. Unlike the standard autoencoder [47], the encoder of a VAE concludes with a probability distribution of the latent space rather than the latent space itself. From this, the forward pass of a VAE’s decoder samples from the distribution to generate a latent space representation before continuing to reconstruct the image. We apply the standard KL Divergence Loss [62] to the sampling at the start of decoding as well the mean

Table 6.5.1: Variational AutoEncoder on MNIST Configuration. Configuration of experiments ran on a Variational AutoEncoder model on the MNIST dataset.

Hyperparameter	Value
Train/Val	60,000 / 10,000
Architecture	VAE
Layer Widths	784, 512, 256, 32, 256, 512, 784
Initialization	Xavier Uniform
Loss Functions	BCE and KLD
Optimizer	Adam
Batch Size	100
Learning Rate	0.001, 0.005, or 0.0001
Number of Epochs	50
Number of Trials	10 per learning rate

Table 6.5.2: VAE MNIST Reconstruction Error. Training and testing errors of our Variational AutoEncoder compressing and decompressing samples, averaged over 5 trials for each combination.

Name	Train Error	Test Error
TeLU lr=0.001	100.77 $\pm$ 0.240	100.56 $\pm$ 0.245
ReLU lr=0.001	102.00 $\pm$ 0.028	101.78 $\pm$ 0.052
TeLU lr=0.0005	95.102 $\pm$ 0.090	96.367 $\pm$ 0.097
ReLU lr=0.0005	96.537 $\pm$ 0.248	97.884 $\pm$ 0.145
TeLU lr=0.0001	95.234 $\pm$ 0.126	97.242 $\pm$ 0.174
ReLU lr=0.0001	96.785 $\pm$ 0.164	98.844 $\pm$ 0.397

squared error (MSE) [94] reconstruction loss. We use a batch size of 100 and train ReLU and TeLU networks over 50 epochs. Table 6.5.1 summarizes these hyperparameters. In Table 6.5.2, we observe the training and testing errors of each model across varying learning rates to show that the results are consistent. ReLU, a piecewise-linear universal approximator is not analytic and therefore does not form smooth approximations to the target representation of the task. Therefore, we observe that TeLU reconstructions outperform ReLU reconstructions.

6.5.2 Penn TreeBank Dataset with RNN Architectures

Additionally, we show that TeLU performs competitively when used as the hidden nonlinearity in Recurrent Neural Networks (RNN) [112] architectures on the Penn TreeBank (PTB) [78] dataset. First, we define an Elman architecture [24], the simplest form of a recurrent neural network. In an Elman recurrent neural network, hidden neurons perform their forward pass by taking their previous activation as inputs, allowing for the introduction of neuron states. We set an input embedding size of 450, a hidden layer width of 650, and 2 hidden layers. We train over 40 epochs with a batch size of 20, a learning rate of 0.2, and a hidden neuron dropout rate of 0.2. We truncate the backpropagation through time (BPTT) [112] steps to 25 and perform gradient clipping to limit the absolute value of each computer gradient during the backward pass

Table 6.5.3: RNN Penn Treebank Test PPL Summary. Elman and LSTM RNN architectures’ testing PPL, averaged over 10 trials per non-linearity.

Name	Elman	LSTM
TeLU	132.65 $\pm$ 2.48	81.842 $\pm$ 0.37
ReLU	140.63 $\pm$ 4.25	86.855 $\pm$ 0.62
Tanh	138.65 $\pm$ 5.39	82.123 $\pm$ 1.25

to 0.1. We compare the Perplexity (PPL) [7] of TeLU, ReLU, and Tanh architectures in Table 6.5.3 over 10 seeded trials and observe that TeLU networks offer significant improvements over networks employing ReLU and Tanh nonlinearities. These 10 trials were seeded with the first 10 whole numbers, 0 through 10. The hyperparameter values used for this Elman network experiment are detailed in Table 6.5.4.

We follow this success to demonstrate similar improvements in Long Short Term Memory (LSTM) RNNs. We utilize the same batch size, number of hidden layers, and number of epochs as in the Elman experiments. Since LSTM networks benefit from additional memory and gating over Elman networks, we increase the number of backpropagation steps to 35. We also empirically determine that the embedding size of 550 and the hidden layer size of 750 are preferred by LSTMs. To balance this modification, we increase our hidden neuron dropout rate to 0.65 and reduce our gradient clipping to 0.25 to reduce overfitting of the model. We maintain the gating activation function as logistic sigmoids and hyperbolic tangents like in standard LSTMs so that they may continue functioning as gates. Since these sigmoid gates suffer from vanishing gradients, we increase our learning rate to 20. Lastly, we modify only the LSTM cells’ output activation function to employ either TeLU, ReLU, or Tanh. We observe significant improvements of TeLU over ReLU, and slight improvements over Tanh in Table 6.5.3. For reproducibility, we detail this seeded LSTM experiment in Table 6.5.5.

6.5.3 FashionMNIST Dataset with MLP Robustness Experiment

To observe the robustness benefits of a smooth universal approximator, we designed an experiment with an MLP on the Fashion-MNIST dataset [22]. We performed ten trials, seeded from integers 0 to 9, where an architecture employing the TeLU, ReLU, and ELU nonlinearities as its hidden activation function trains over 100 epochs. We partition the FashionMNIST according to 50,000 training samples; 10,000 validation samples; and 10,000 testing samples. In order to reach robust solutions, we use an SGD optimizer [92] with a large learning rate of 0.05 and a large weight decay coefficient of 0.001. We set our momentum parameter update acceleration coefficient to 0, upon noticing that momentum tends to produce less robust models in this simple experiment. We define a batch size of 256 with the intention to perform more regularized weight parameter update steps, resulting in mini-batch gradient descent [56]. After each epoch of training, we

Table 6.5.4: Elman RNN on Penn Treebank Experiment Configuration. Configuration of experiments ran on an Elman RNN model on the Penn Treebank dataset.

Hyperparameter	Value
Train/Val/Test Split	930,000/74,000/82,000 words
Architecture	Elman Network
Embedding Size	450
Hidden Layer Count	2
Hidden Layer Size	650
Dropout	0.2
Initialization	Uniform(-0.1,0.1)
Optimizer	Mini-batch SGD
Batch Size	20
BPTT Steps	25
Gradient Clipping	1.0
Seeds	0, 1, ..., 9
Learning Rate	0.2
Number of Epochs	40
Number of Trials	9

Table 6.5.5: LSTM RNN on Penn Treebank Experiment Configuration. Configuration of experiments ran on an LSTM RNN model on the Penn Treebank dataset.

Hyperparameter	Value
Train/Val/Test Split	930,000/74,000/82,000 words
Architecture	LSTM Network [103]
Embedding Size	550
Hidden Layer Count	2
Hidden Layer Size	750
Dropout	0.65
Initialization	Uniform(-0.1,0.1)
Optimizer	Mini-batch SGD
Batch Size	20
BPTT Steps	35
Gradient Clipping	0.25
Seeds	0000, 1111, ..., 9999
Learning Rate	0.2
Number of Epochs	40
Number of Trials	9

Table 6.5.6: MLP FashionMNIST Robustness Configuration. Configuration of experiments ran on an MLP model on the FashionMNIST dataset and tested on various standard deviations of zero-centered Gaussian noise.

Hyperparameter	Value
Train/Val/Test Split	50,000 / 10,000 / 10,000
Hidden Layer Count	8
Hidden Layer Width	128
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD
Momentum	0.0
Batch Size	256
Learning Rate	0.05
Weight Decay	0.001
Number of Epochs	100
Number of Trials	10

Table 6.5.7: MLP FashionMNIST Robustness Test Accuracy. Average test accuracy of our MLP model training over 100 epochs on the FashionMNIST dataset with increasing levels of Gaussian noise applied to the testing samples. Each data point has been averaged over 10 trials.

Stdv	TeLU	ReLU	ELU	ELU Stable
0.00	88.64	88.53	25.65	88.27
0.02	88.62	88.51	25.62	88.14
0.04	88.54	88.35	25.60	88.03
0.06	88.29	88.18	25.59	87.97
0.08	88.07	87.94	25.52	87.60
0.10	87.76	87.54	25.44	87.21
0.12	87.35	86.91	25.32	86.61
0.14	86.81	86.43	25.25	86.26
0.16	86.28	85.77	25.08	85.40
0.18	85.42	84.98	24.89	84.49
0.20	84.66	83.90	24.77	83.89

evaluate our model on the validation partition and save a checkpoint if validation accuracy has improved. After 100 epochs, we evaluate the best validation accuracy checkpoint on 11 instances of our test dataset, each with an increasing degree of Gaussian noise standard deviation summed to the inputs. We detail our experimental setup in Table 6.5.6.

We record our testing accuracies over the 11 noise standard deviations in Table 6.5.7. We note that while tuning the hyperparameters of the experiment to optimize robustness without reducing accuracy, ELU experienced numerical instability at favorable configurations. To focus on the robustness of the activation functions, rather than the stability, we separate ELU’s testing accuracies into two columns. The first column depicts the average ELU test accuracy for each noise level, while the second “ELU Stable” column calculates the average accuracies for stable ELU trials. We also visualize the stable behavior of all three architectures in Figure 6.5.1 across the gradually increasing noise distributions. We did not include the overall testing

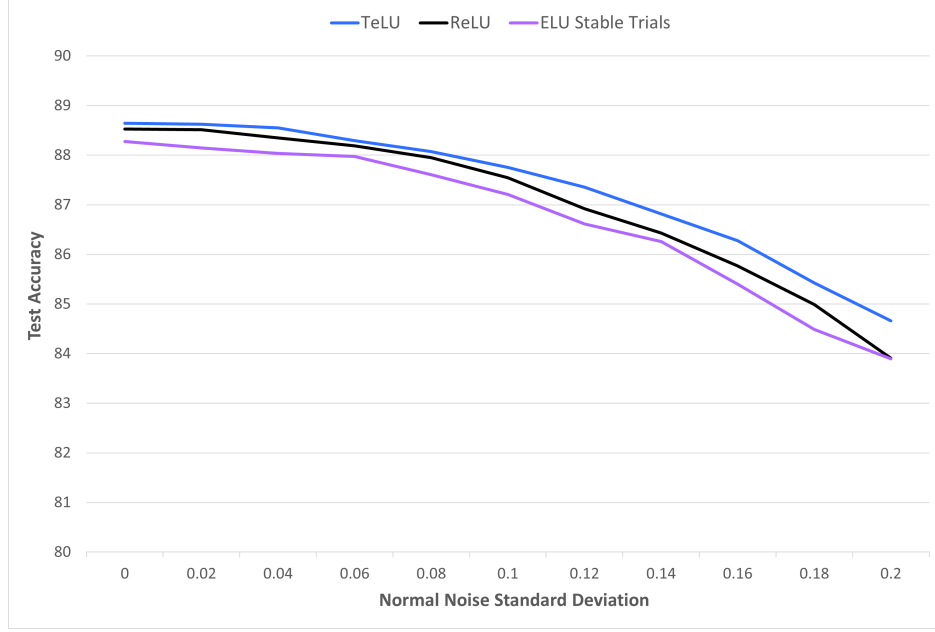


Figure 6.5.1: MLP FashionMNIST Robustness Test Accuracy. Shows TeLU, ReLU, and stable ELU robustness accuracies across various standard deviations of Gaussian noise.

average of ELU architectures in this Figure, as this would significantly reduce the resolution needed to valuably compare the robustness exhibited each activation function. We note that TeLU, an analytic universal approximator, significantly outperforms the piecewise ReLU and ELU activation functions when employed as the hidden layer activation of an MLP.

6.6 Stability

In Section 5.6, we explored how an activation function’s properties; such as simple formulation, mitigation of vanishing and exploding gradients, smoothness, and reduced output bias; contribute to the overall stability of a neural network. With these foundational concepts established, we now shift from theory to practice by validating these benefits through experimental analysis. In this section, we will demonstrate how these theoretical advantages translate into real-world improvements in training stability, convergence speed, and model performance. By conducting empirical tests across various architectures and datasets, we aim to showcase the tangible impact of these activation functions on deep learning tasks, reinforcing the connection between theory and practical application.

Table 6.6.1: MLP FashionMNIST Cross-Depth Stability Experiment Configuration. Configuration of experiments ran on MLP architectures with different hidden layer counts on the FashionMNIST dataset.

Hyperparameter	Value
Train/Val/Test Split	50,000 / 10,000 / 10,000
Hidden Layer Count	[16, 44]
Hidden Layer Width	128
Initialization	Xavier Uniform
Optimizer	Mini-batch SGD
Momentum	0.9
Batch Size	128
Learning Rate	0.005
Weight Decay	Repeated with 0.0005 and 0.001
Number of Epochs	100
Number of Trials	10 trials per depth

6.6.1 FashionMNIST Dataset with MLP with Varied Network Depths

First we demonstrate that TeLU architectures offer stable learning across a wide range of model depths. We define MLP architectures that employ hidden activations of TeLU, ReLU, and Mish with a starting number of 16 hidden layers of width 128. We then train models with a momentum-accelerated Mini-batch Gradient Descent [56] [71] on the MNIST dataset [22] with a weight decay coefficient of 0.0005; setting aside 10,000 validation images and 10,000 testing images. We repeat this process for incrementally increasing hidden layers until reaching a hidden depth of 44. We detail our precise experimental configuration in Table 6.6.1 and show the average testing accuracies of each architecture in Figure 6.6.1. We observe that ReLU, being a piecewise linear function with greater output bias than TeLU and Mish, succumbs to instability as depth increases. TeLU and Mish, both being smooth non-monotonic functions with less biased outputs, maintain their stability and perform predictably across all depths. Our detailed configuration is given by Table 6.6.1.

We then repeat the experiment for hidden layer counts between 34 and 44 with an increased weight decay coefficient of 0.001. We observe in Figure 6.6.2 how ReLU, being less self-regularized, stabilizes with greater L2 regularization. Mish, however, exhibits drastic instability with this minor increase in weight decay. We understand that Mish is a self-regularized non-monotonic function [83] with a more complex formulation than TeLU and ReLU. In this context, $Mish(x) = x \cdot \tanh(\ln(1 + e^x))$ may be interpreted as a version of $TeLU(x) = x \cdot \tanh(e^x)$ with the addition a nested natural logarithm nonlinearity. This extra intermediate computation exposes each Mish activation to numerical underflow. Once underflow occurs, it may propagate along the remaining layers until dictating a particular choice of inference. At the worst case of a depth of 44, this underflow is more likely to occur somewhere in the network and the average accuracy of the model be comparable to that of a random guess.

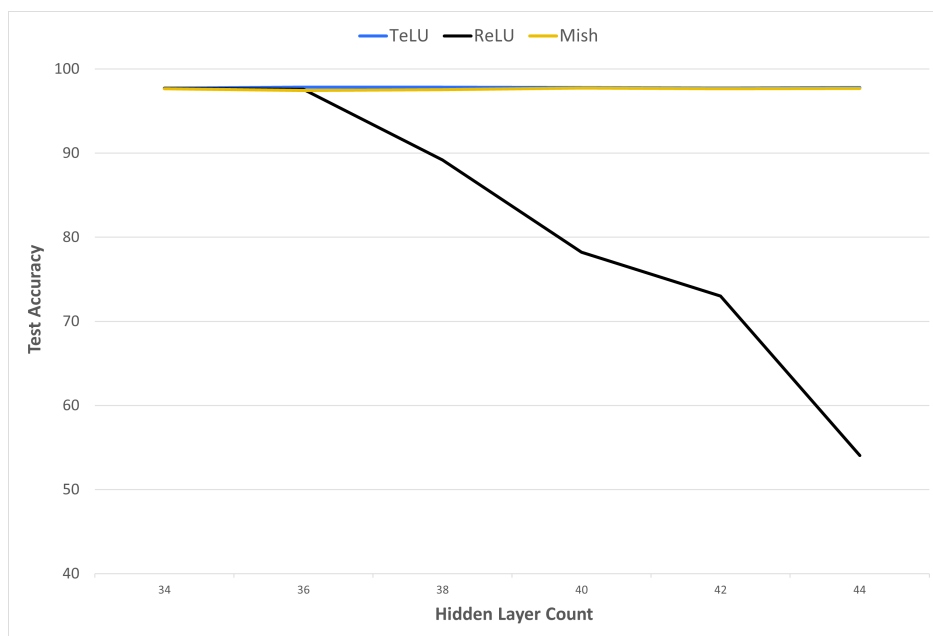


Figure 6.6.1: MLP FashionMNIST Test Accuracy Across Depths with Weight Decay 0.0005. Architecture stability as number of hidden layers increases while using weight decay coefficient of 0.0005.

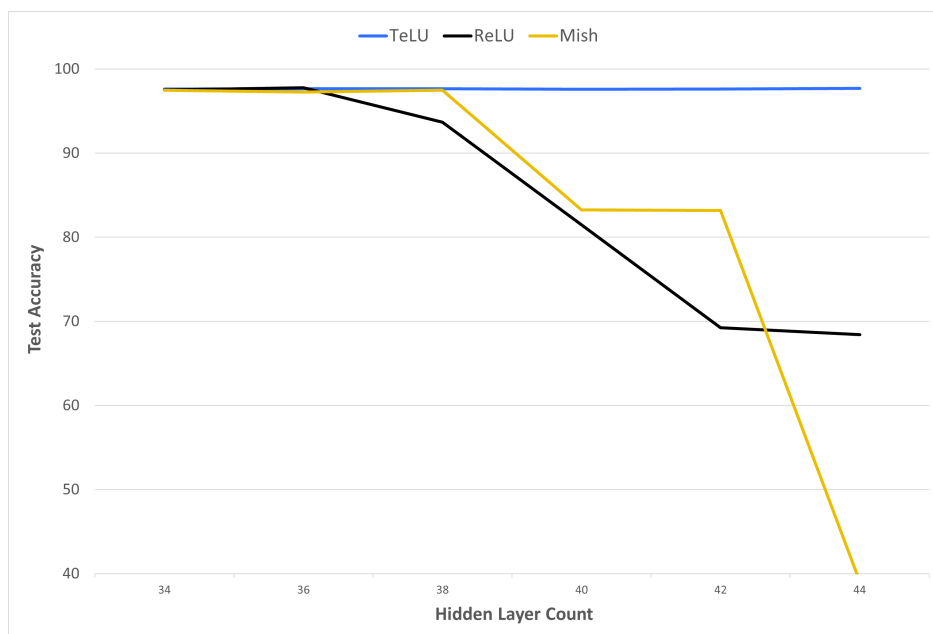


Figure 6.6.2: MLP FashionMNIST Test Accuracy Across Depths with Weight Decay of 0.001. Architecture stability as number of hidden layers increases while using weight decay coefficient of 0.001.

Table 6.6.2: MLP FashionMNIST Cross-Initializer Stability Configuration. Configuration of experiments ran on an MLP model on the FashionMNIST dataset across various learnable parameter initialization methods.

Hyperparameter	Value
Train/Val/Test Split	50,000 / 10,000 / 10,000
Hidden Layer Count	8
Hidden Layer Width	128
Initialization	Xavier Uniform, Xavier Normal, He Uniform, He Normal
Optimizer	Mini-batch SGD
Momentum	0.9
Batch Size	128
Learning Rate	0.1
Weight Decay	0.001
Number of Epochs	50
Number of Trials	6 trials per initializer

6.6.2 FashionMNIST Dataset with MLP with Varied Initialization Methods

We perform a similar experiment with the MLP architecture on the FashionMNIST dataset [22]. This time we treat our weight initialization method as an independent variable instead of our model’s depth. We define TeLU, ReLU, ELU, SiLU, GELU, Mish, Logish, and Smish MLP architectures by changing only the activation function of hidden layers. We initialize each architecture according to Xavier Uniform, Xavier Normal, He Uniform, and He Normal for different experiments. To be able to observe the impact of the different initializations, we limit the number of epochs to prevent eventual convergence on similar accuracies across initializers. We configure our experiment hyperparameters according to Table 6.6.2. We train each combination of initializer and architecture over 10 trials. We view the averaged testing accuracies for each architecture on each initializer in Figure 6.6.3. We observe how TeLU maintains superior performance over other architectures across all initializers. We also note that the ReLU architectures also exhibit considerable accuracy, but all other activation architectures seem to vary in success across initializers.

6.6.3 CIFAR-100 Dataset with SqueezeNext with Varied Optimizers

We go on to showcase the comparative stability and performance of the TeLU, ReLU, and GELU activation functions when utilized as the hidden activation function in a SqueezeNext CNN [32]. We train the SqueezeNext architecture on the CIFAR-100 dataset [60] for 200 epochs with the Mini-batch SGD [56] [92], momentum-accelerated SGD [71], AdamW [72], and RMSpropr [38]. We utilize a learning rate scheduler that updates the base learning rate every 60 epochs. We detail the training hyperparameter values that stay constant across optimizer settings in Table 6.6.3, as well as the hyperparameters that change in Table 6.6.4. After each training epoch, we check to see if our validation has improved. If it has, we overwrite

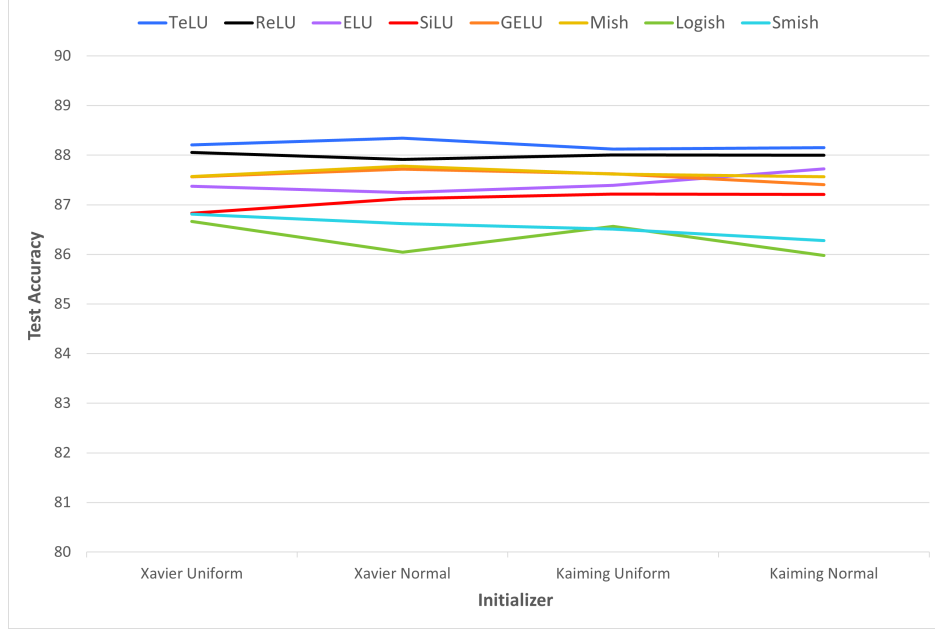


Figure 6.6.3: MLP FashionMNIST Test Accuracy Across Initializers. average test accuracy of our MLP model training over 50 epochs on the FashionMNIST dataset across various weight initialization methods. Each data point has been averaged over 10 trials.

our checkpoint with the current weight and bias values of our model. After 5 trials of each optimizer and architecture combination’s training is complete, we evaluate the best validation accuracy checkpoint on the testing partition of 10,000 CIFAR-100 images. We summarize the testing accuracies and standard deviations in Table 6.6.5. Additionally, we compare the progression of the validation accuracies of each architecture for the Minibatch SGD and momentum-accelerated SGD optimizers in Figures 6.6.4 and 6.6.5.

We observe from Table 6.6.5 that TeLU exhibits greater accuracy than both ReLU and GELU accuracies in all cases. Additionally, we notice that TeLU exhibits greater learning stability in most cases, as determined by its minimal standard deviation of test accuracy across the Mini-batch SGD, momentum-accelerated SGD,

Table 6.6.3: SqueezeNext CIFAR-100 Optimizer-Independent Hyperparameters. Static configuration of stability experiments ran on the SqueezeNext architecture on the CIFAR-100 dataset across different optimizers.

Hyperparameter	Value
Train/Val/Test Split	45,000 / 15,000 / 10,000
Normalization	Standard Score, detailed in Table A.2.1
Architecture	SqueezeNext [32]
Initialization	Xavier Uniform
Batch Size	128
Learning Scheduler Period	60
Data Augmentations	Random 4-padded cropping and horizontal flipping
Number of Epochs	200
Number of Trials	10

Table 6.6.4: SqueezeNext CIFAR-100 Optimizer-Dependent Hyperparameters. Learning rate, weight decay coefficient, and learning rate scheduler decay hyperparameter values which vary across optimizer choice.

Optimizer	learning rate	weight decay	gamma
SGD	0.04	0.003	0.2
Momentum	0.003	0.003	0.2
AdamW	0.005	0.005	0.4
RMSprop	0.0002	0.005	0.4

Table 6.6.5: CIFAR-100 SqueezeNext Test Accuracy Summary. A summary of the test accuracies achieved across various optimizers on the CIFAR-100 dataset by the SqueezeNext Architecture checkpoint that achieved optimal validation accuracy throughout 200 epochs of training.

Name	SGD	Momentum	AdamW	RMSprop
TeLU	71.47 $\pm$ 0.08	70.53 $\pm$ 0.25	69.64 $\pm$ 0.07	68.83 $\pm$ 0.33
ReLU	69.52 $\pm$ 0.43	65.05 $\pm$ 0.51	66.31 $\pm$ 0.48	67.99 $\pm$ 0.21
GELU	67.09 $\pm$ 0.36	66.26 $\pm$ 29	66.50 $\pm$ 0.44	65.19 $\pm$ 0.25

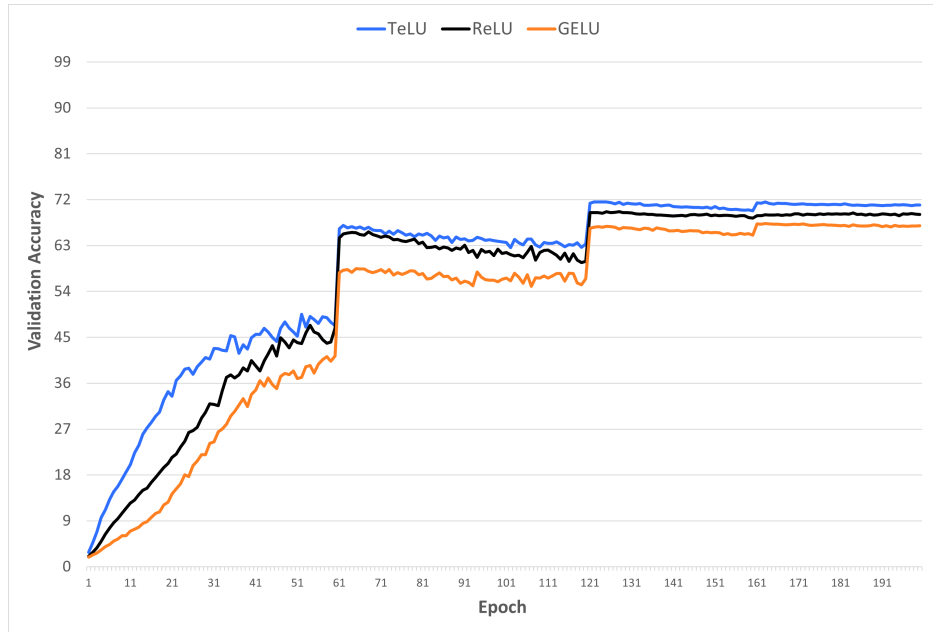


Figure 6.6.4: SqueezeNext on CIFAR-100 with SGD Validation Accuracy. Shows the progression of validation accuracy throughout 200 epochs of the SqueezeNext architecture being trained on the CIFAR-100 dataset with the Mini-batch SGD optimizer. Each data point has been averaged over 5 trials.

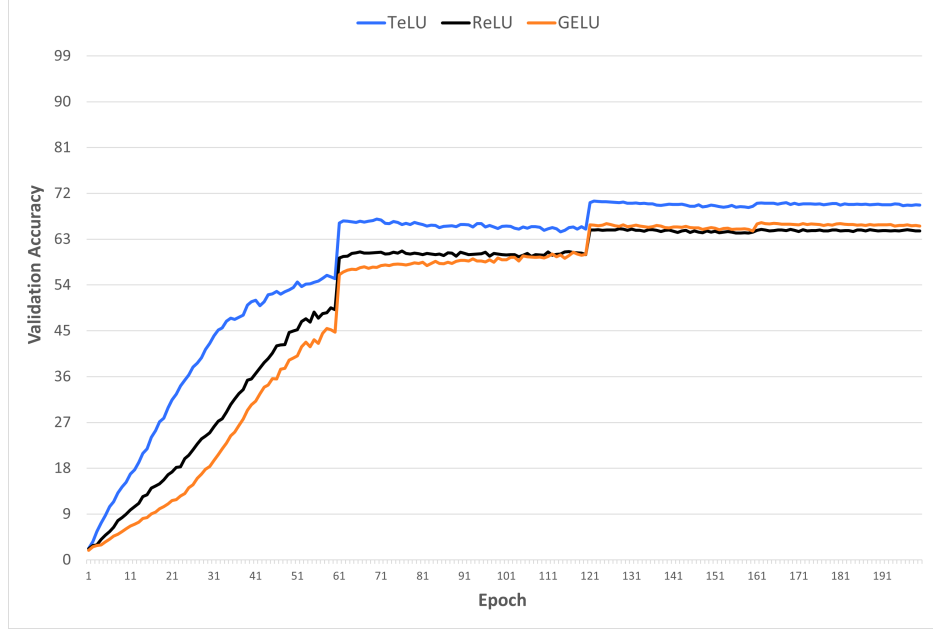


Figure 6.6.5: SqueezeNext on CIFAR-100 with Momentum-Accelerated SGD Validation Accuracy. Shows the progression of validation accuracy throughout 200 epochs of the SqueezeNext architecture being trained on the CIFAR-100 dataset with the momentum-accelerated Mini-batch SGD optimizer. Each data point has been averaged over 5 trials.

and AdamW optimizers. This means that the network is less sensitive to fluctuations during training, leading to more predictable and stable outcomes, which are crucial in real-world applications where reliability is paramount.

6.6.4 Tiny ImageNet Dataset with ResNet34 with Varied Optimizers

We extend our experimentation to include a ResNet34 architecture on TinyImageNet to ensure that this trend persists for larger architectures and datasets. We utilize the configuration detailed in Table 6.6.6 on both Mini-batch SGD and momentum-accelerated SGD optimizers. Across optimizers, we vary only the base learning rate, weight decay coefficient, and learning rate scheduler decay hyperparameters as detailed in Table 6.6.7. We perform each trial by training the respective ResNet34 architecture on the TinyImageNet training partition, consisting of 100,000 images, over the course of 200 epochs. After each epoch, we see if the validation accuracy that the updated model exhibits is an improvement over the current validation accuracy. If the previous epoch of training has improved validation accuracy, we save a new checkpoint of our model. After training concludes, we evaluate our best checkpoint on the unseen testing partition to determine our top-1 and top-5 testing accuracies.

Table 6.6.6: ResNet34 TinyImageNet Optimizer-Independent Hyperparameters. Static configuration of stability experiments ran on the ResNet34 architecture on the TinyImageNet dataset across different optimizers.

Hyperparameter	Value
Train/Val/Test Split	100,000 / 10,000 / 10,000
Normalization	Standard Score, detailed in Table A.2.1
Architecture	ResNet34
Initialization	Xavier Uniform
Batch Size	128
Learning Scheduler Updates	60, 100, 140, 170
Data Augmentations	Random 4-padded cropping and horizontal flipping
Number of Epochs	200
Number of Trials	10

Table 6.6.7: ResNet34 TinyImageNet Optimizer-Dependent Hyperparameters. Learning rate, weight decay coefficient, and learning rate scheduler decay hyperparameter values which vary across optimizer choice.

Optimizer	learning rate	weight decay	gamma
SGD	0.05	0.001	0.3
Momentum	0.04	0.0004	0.4
AdamW	0.0005	0.004	0.5
RMSprop	0.0001	0.0002	0.6

We record our test results across 5 trials for the Mini-batch SGD optimizer in Table 6.6.8 and observe that TeLU offers greater average test accuracy and less variance in top-1 test scores. For another perspective on these accuracy improvements, we plot the progression of validation accuracy across the 200 epochs of training in Figure 6.6.6. Here, we observe that the TeLU architecture exhibits faster convergence speeds after the learning rate updates at epochs 60 and 100. After this, both the ReLU and TeLU architectures experience a saturation in validation accuracies, with TeLU architectures ending up at a greater validation accuracy.

We also summarize the test results across 10 trials for the momentum-accelerated Mini-batch SGD optimizer in Table 6.6.9. It is evident that the ReLU-based ResNet34 architecture experiences significant instability in both top-1 and top-5 accuracy metrics due to numerical issues. Notably, 3 out of the 10 trials resulted in 'not a number' (NaN) training accuracies across all 200 epochs, indicating that numerical

Table 6.6.8: ResNet34 TinyImageNet SGD Test Accuracy. Test accuracies of TeLU and ReLU ResNet34 architectures averaged across 10 trials at the state at which they achieved optimal validation accuracy when training on TinyImageNet dataset with a Mini-batch SGD optimizer.

Name	Top-1 Test	Top-5 Test
TeLU	62.34 $\pm$ 0.173	81.86 $\pm$ 0.337
ReLU	61.16 $\pm$ 0.314	80.51 $\pm$ 0.263

Table 6.6.9: ResNet34 TinyImageNet Momentum-Accelerated SGD Test Accuracy Summary. Test accuracies of TeLU and ReLU ResNet34 architectures averaged across 10 trials at the state at which they achieved optimal validation accuracy when training on TinyImageNet dataset with a momentum-accelerated Mini-batch SGD optimizer.

Name	Top-1 Test	Top-5 Test
TeLU	62.09 $\pm$ 0.222	82.28 $\pm$ 0.453
ReLU	38.37 $\pm$ 34.6	50.32 $\pm$ 43.8

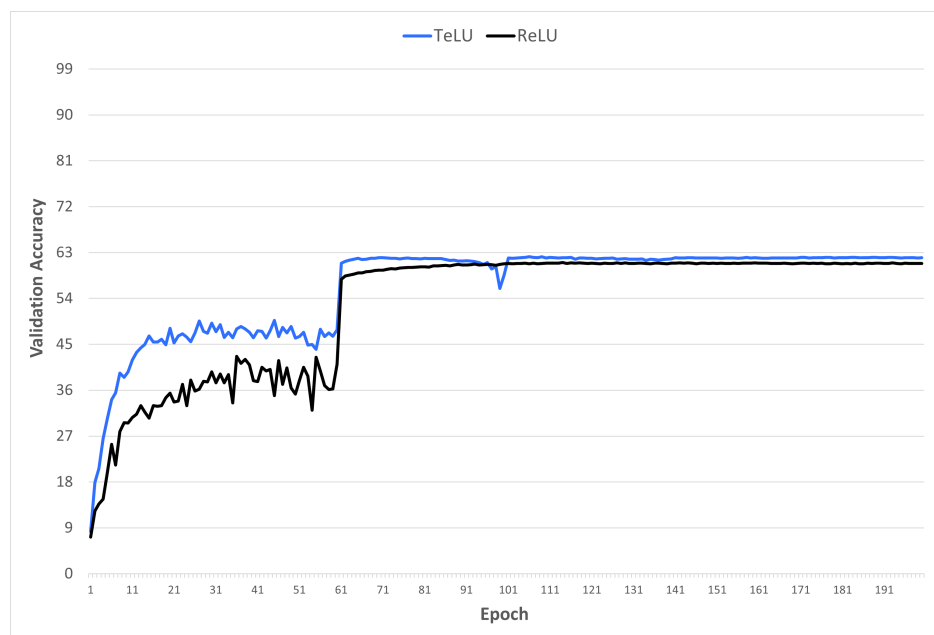


Figure 6.6.6: ResNet34 TinyImageNet SGD Validation Accuracy. Progression of validation accuracies of TeLU and ReLU ResNet34 architectures averaged across 10 trials when training on TinyImageNet dataset with a Mini-batch SGD optimizer.

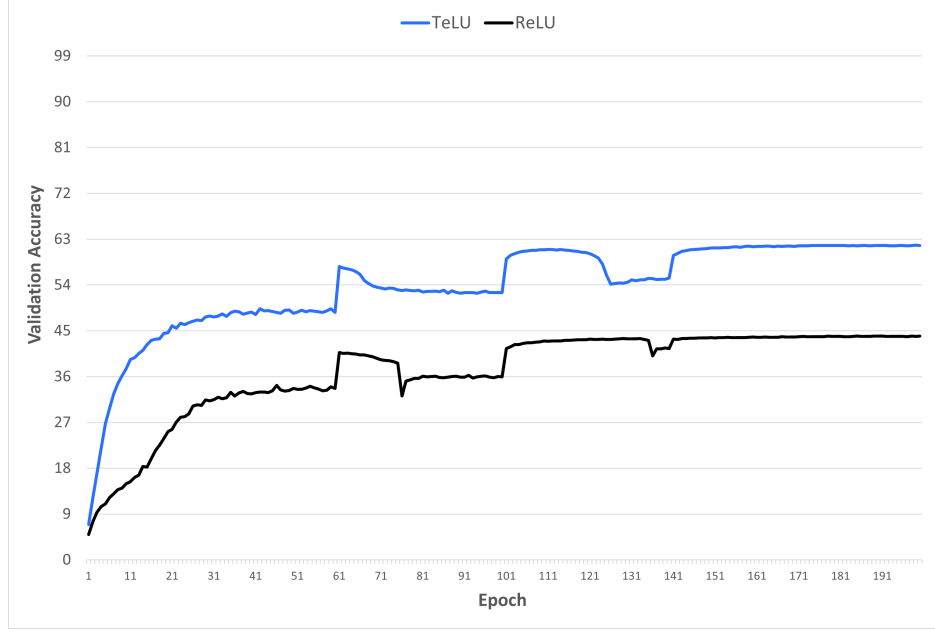


Figure 6.6.7: ResNet34 TinyImageNet Momentum-Accelerated SGD Validation Accuracy. Progression of validation accuracies of TeLU and ReLU ResNet34 architectures averaged over 10 trials when training on TinyImageNet dataset with a momentum-accelerated Mini-batch SGD optimizer

instability arises as early as the first epoch. When training accuracy is compromised by such instability, the neurons transmitting it dictate the output, leading to random validation accuracies, where the network effectively guesses output values. This behavior is further illustrated in Figure 6.6.7, where we observe that the ReLU ResNet34 architecture underperforms by approximately 20% in average validation accuracy compared to the TeLU model. These findings confirm that TeLU-based architectures maintain stability across different optimizers.

6.6.5 Summary of Stability Experimental Validation

Throughout previous sections of this chapter, we have observed similar cases of instability with other activation functions. Figure 6.4.1 shows that SiLU, GELU, Mish, Logish, and Smish experience significantly more instability than other nonlinearities when employed within MLP architectures when the weight decay coefficient is increased beyond 0.001. Figure 6.4.2 adds further evidence to this observation, with SiLU, GELU, Mish, Logish, and Smish architectures resulting in inferior testing accuracies after training concludes. Furthermore, we observe in Figure 6.4.3 that SiLU, Logish, and Smish SqueezeNext architectures result in reduced test accuracies when trained on the CIFAR-10 dataset with a Mini-batch SGD optimizer. This drop in accuracy is amplified for Logish and Smish when momentum is introduced in 6.4.4. Across all experiments, TeLU experiences the most stable and consistent results.

The reasons for TeLU's unique degree of learning stability lie in its ability to mitigate both vanishing and exploding gradients while maintaining smoothness and minimal output bias. This combination ensures that gradients propagate effectively through deep networks, allowing for steady and controlled learning, even in complex architectures. TeLU's ability to deliver high accuracy while ensuring stable learning makes it particularly well-suited for tasks requiring long-term reliability, such as in autonomous systems, financial forecasting, or critical healthcare diagnostics, where instability in learning could result in severe consequences. By delivering consistently better accuracy and reduced variability, TeLU provides a foundation for more resilient, generalizable models across a wide range of applications.

Chapter 7: Discussion

We have introduced the Hyperbolic Tangent Exponential Unit (TeLU), an activation function for deep neural networks defined as $TeLU(x) = x \cdot \tanh(e^x)$. TeLU is an analytic, non-monotonic non-linearity that belongs to the class of linearly-scaled cumulative distribution functions, which consists of linear units such as $ReLU(x) = \max(0, x)$ and $GELU(x) = \frac{x}{2} \cdot (1 + \operatorname{erf}(\frac{x}{\sqrt{2}}))$. Like other linear units, TeLU may be interpreted as the identity function x with an implicit activation dropout as input becomes negative. This results in TeLU comprising a dense deactivation region followed by an active region exhibiting asymptotic identity growth. Throughout this thesis, we have demonstrate TeLU’s persistent gradients of its deactivation region, near-linearity of its active region, computational efficiency, compatibility as a substitute for ReLU, analytic universal approximation, and inherent stability properties. Together, these properties allow TeLU to uniquely overcome vanishing gradient, convergence delay, computational delay, configuration tuning delay, community adoptability, and learning instability challenges.

7.1 Persistent Gradients of Deactivation Region of TeLU

In Section 5.1, we noticed that our lower-bounded activation functions had derivatives that saturate towards deactivation at different rates. To better describe these varying vanishing rates, we extended the asymptotic growth classes to encompass asymptotic decay with classes $\Theta(\frac{x}{e^x})$, $\Theta(\frac{1}{e^x})$, $O(\frac{1}{x!})$, and $\Omega(\frac{1}{(x^2)!})$. We discovered that TeLU’s gradient vanished according to $\Theta(\frac{x}{e^x})$, leading to minimal vanishing gradient concerns among competing functions. We showcased the effect of these varying vanishing gradients numerically by expressing the domain that avoids numerical underflow in each activation function.

In Section 6.1, we showed the direct impact of these asymptotic decays, we designed an experiment where MLP architectures employing different hidden non-linearities have biases initialized to -10 and -20. We trained each architecture on the FashionMNIST dataset and observed each model’s ability to recover from the vanishing gradient conditions found at their saturation regions. In each case, we observed that TeLU emerges first. Remaining architectures lag behind TeLU, in an order that is expected given their asymptotic decay class and in a manner that is reflective of their gradient strengths.

We proceed to provide further experimental validations to the impact of persistent gradients by comparing TeLU, ReLU, and GELU DenseNet architectures on CIFAR-10. DenseNets do not feature any residual bypass connections, so vanishing or dying gradients in GELU and ReLU architectures are especially effective in halting learning prematurely, resulting in models that exhibit poor accuracy. Meanwhile, the TeLU architecture trains and generalizes to improved accuracy in all experiments, showing that TeLU mitigates the vanishing gradient problem.

7.2 Near-Linearity of Active Region of TeLU

In Section 5.2, we focused on TeLU’s near-linear behavior of the active region, featuring strong gradients that are essential for meaningful learnable parameter update steps. This undiminished learning allows for rapid, stable convergence towards superior training accuracy and generalizes to superior test accuracies, as ensured by TeLU’s implicit dropout. In Section 6.2, we provide experimental validation for the rapid, stable convergence of TeLU. We started by comparing TeLU and ReLU ResNet34 architectures on the ImageNet dataset. While TeLU smoothly approaches linear growth rapidly, ReLU exhibits linear growth immediately once its input become positive. With both TeLU and ReLU activation functions exhibiting strong active gradients, we compare the speed and stability of both architectures.

First, we utilized Pytorch’s ResNet34 default training configuration, which encompassed 90 epochs of training. We observed that the TeLU activation allowed for faster and more consistent convergence than ReLU. To accentuate on the benefits of this improved convergence, we limited our number of epochs to 50 and 20 epochs in two separate follow-up experiments. In either case, we observe that TeLU architectures provide improved convergence and superior generalization over ReLU architectures on ImageNet.

We further demonstrate TeLU’s convergence advantages over ReLU and GELU within a dynamic-pooling transformer architecture trained on the Text8 dataset. Initially, we highlighted TeLU’s superior convergence speed and stability compared to GELU in terms of validation loss, with this improvement translating to lower test loss and reduced standard deviation. To examine the impact of TeLU’s faster convergence, we conducted a shorter experiment with a reduced learning rate scheduler period. As expected, TeLU outpaced both ReLU and GELU in terms of convergence speed and stability under these conditions.

7.3 Computational Efficiency of TeLU

In Section 5.3, we quantified the computational complexities of baseline definitions of activation functions. We counted the number of piecewise segmentations, nonlinear functions, arithmetic operations, and constant terms present within each activation function. We discovered that TeLU exhibited minimal computational complexity that was second only to ReLU. Table 5.3.2 summarizes these counts for our focused group of non-linearities.

In Section 6.3, we witness the direct computational benefits on TeLU over other smooth activation functions within custom Pytorch [87] benchmarks run on various systems. Each benchmark consisted of a number of forward and backward passes on neurons that activated according to the TeLU, ReLU, ELU, SiLU, GELU, Mish, Logish, and Smish nonlinearities. The first of these benchmarks was run on a Windows 11 operating system with an NVIDIA RTX 2070 GPU, and consisted of 10^6 activations on an input size of 10^6 , each followed by the corresponding gradient calculation. The total amount of time spent on forward and backward passes was calculated for each non-linearity.

TeLU and ReLU were found to offer the most computational efficiency, as expected from the computational complexity heuristics. The next benchmarks were run on a LINUX batch server and utilized A100 [15] and 1080Ti GPUs, respectively. Tables 6.3.2, 6.3.4, and 6.3.3 summarize the resulting runtime necessary across varying sizes of input and number of iterations. Across each operating system, device, and experimental configuration; we observe TeLU offering optimal run-times that are second only to ReLU.

7.4 Configuration Compatability of ReLU and TeLU

In Section 5.4, we describe how non-monotonic non-linearities are the most suitable for approximating the ReLU activation function. We begin by calculating the area between different activation functions and ReLU, showing an initial heuristic for the closeness of approximation each function provides. This is performed for both negative and positive inputs, to indicate the quality of approximation in both the inactive and active regions. From our candidate pool, we demonstrated that non-monotonic functions are uniquely capable of approximating ReLU as input $x \rightarrow \pm\infty$ and at $x = 0$. Additionally, we found that TeLU serves as the most suitable substitute for ReLU in deep learning applications. TeLU’s active gradients closely approximate ReLU’s strong identity growth, resulting in a comparable convergence speed and low implicit regularization. In contrast, other activation functions with weaker active gradients experience implicit regularization in the form of gradient damping, a behavior that distinguishes them from ReLU and TeLU.

In Section 6.4, we experimentally validated the operational similarities between TeLU and ReLU by testing MLP architectures with different non-linearities as their hidden activation functions. The results

showed that the same weight decay coefficient values led to optimal performance in both TeLU and ReLU architectures, whereas other architectures with different activation functions did not achieve similar results. Furthermore, we show that TeLU provides accuracy improvements over ReLU on the same configuration that optimizes ReLU convergence and generalization. We offer additional experimental evidence of this similarity by demonstrating that TeLU improves accuracy in training configurations optimized to minimize the loss in ReLU CNN architectures. For each optimizer, including Mini-batch SGD [56, 92], momentum accelerated SGD [71], AdamW [72], and RMSprop [38], we tune the training hyperparameters that influence the learning of a SqueezeNext architecture using the ReLU nonlinearity. The hyperparameters are optimized to achieve the highest possible validation accuracy for each optimizer. Again, we observe that TeLU architectures are optimized by configurations designed for ReLU. In addition, the results demonstrate that TeLU outperforms ReLU when using Mini-batch momentum-accelerated SGD, AdamW, and RMSprop optimizers.

7.5 Analytic Universal Approximation of TeLU

In Section 5.5, we establish that TeLU is an analytic universal approximator. Consequently, architectures utilizing TeLU as their activation function exhibit stable and efficient convergence when optimized with gradient-based methods. The smooth nature of analytic approximators, like TeLU, also enhances generalization to unseen data by minimizing overfitting to noise or irrelevant artifacts [93]. This improved robustness leads to more reliable model performance. Moreover, the smooth approximations provided by analytic functions are easier to interpret and analyze, a quality highly valued in mathematical research [14]. Additionally, as an analytic function, TeLU ensures compatibility with second-order optimization techniques that leverage the curvature of the loss landscape, resulting in more stable and efficient learning [9, 79, 80, 96].

In Section 6.5, we perform practical demonstrations of the benefits of utilizing analytic nonlinearities with VAE [58], RNN [24, 103], and MLP robustness [107] experiments. Results show that VAE architectures that utilize the TeLU activation function lead to MNIST sample reconstructions with minimal loss and improved consistency over that of ReLU VAEs. For our RNN experiments, we define Elman [24] and LSTM [103] architectures that employ either TeLU, ReLU, or Tanh activation functions. Again, we find that TeLU architectures lead to minimal perplexity [51] on the Penn TreeBank dataset [78]. Lastly, we demonstrate the improved robustness of analytic universal approximations by comparing the robustness of TeLU MLPs with that of ReLU and ELU models. As expected, we observe that TeLU consistently outperforms its piecewise competitors [93, 114]. Across all experiments, we notice that TeLU offers a strong advantage over non-analytic activation functions.

7.6 Learning and Numerical Stability of TeLU Networks

In Section 5.6, we examined the properties of activation functions that contribute to the learning stability of deep neural network models. Table 5.6.1 provides a summary of key stability factors: This comprises the depth of embedded nonlinear computations, smoothness, susceptibility to the vanishing gradient problem, and output bias for each activation function studied. We demonstrate that smooth linear units like TeLU effectively mitigate the exploding gradient issue through their sub-linear growth in the positive domain. Our findings show that TeLU achieves the best overall balance across these criteria, resulting in notable stability improvements compared to existing activation functions.

In 6.6, we provide experimental validation of our heuristic comparison by demonstrating that TeLU MLP architectures maintain stable performance as model depth increases across multiple configurations. Furthermore, we show that the learning stability of TeLU persists across various weight initialization methods. Specifically, using Xavier Uniform, Xavier Normal [33], Kaiming Uniform, and Kaiming Normal [43] initialization techniques, TeLU consistently outperforms similar MLPs employing other activation functions. This consistent performance extends to our CNN experiments as well. TeLU-based SqueezeNext architectures achieve superior test accuracies across all tested optimizers when trained on the CIFAR-100 dataset. Across all experiments, we find that TeLU architectures consistently exhibit faster convergence and greater learning stability, highlighting the unique advantages of TeLU in deep learning applications.

Chapter 8:

Conclusion

In conclusion, the Hyperbolic Tangent Exponential Linear Unit (TeLU) stands out as a highly effective activation function that addresses several critical challenges in neural network training. One of its primary advantages is the presence of persistent gradients in its inactive region, which mitigates the vanishing gradient problem that can slow down learning in deep networks. By maintaining non-zero gradients even for negative input values, TeLU ensures that all neurons continue to learn, enhancing overall network performance.

Moreover, TeLU closely approximates the identity function for positive input values. This characteristic strikes an ideal balance, preventing both vanishing and exploding gradient issues. The linear approximation allows for consistent and efficient gradient propagation, leading to faster and more stable convergence during training. By minimizing unintended damping of gradients, TeLU ensures that learnable parameter updates are not inadvertently downscaled. This allows optimizers and learning rate schedulers to effectively manage the magnitude of learning steps, maximizing convergence rates and enabling a more modular and flexible training configuration.

Unlike more complex activation functions, TeLU's simple formulation reduces computational overhead, resulting in significant efficiency improvements. TeLU's straightforward design also offers seamless compatibility as a drop-in substitute for the widely used ReLU activation function. This ease of integration encourages adoption within the deep learning community, allowing practitioners to leverage TeLU's benefits without overhauling existing architectures. Unlike ReLU, TeLU is an analytic function, which enhances robustness and stability during training. Its analytic nature not only improves convergence but also makes it compatible with second-order optimization methods. By utilizing the curvature of the loss function, these methods can further enhance convergence efficiency and stability.

These combined properties enable TeLU to exhibit a unique level of learning stability across a wide range of experimental settings. By effectively addressing issues like vanishing gradients, computational inefficiency, and training instability, TeLU presents a compelling advancement in activation functions. Its ability to simplify training configurations while enhancing performance makes it a valuable tool for advancing deep learning models across various applications. The TeLU activation function, therefore, holds significant promise for facilitating more efficient, stable, and robust neural network training in the field of deep learning.

Chapter 9:

Future Work

For future work, we aim to delve into the unique properties of analytic universal approximators by thoroughly examining the convergence guarantees provided by the TeLU activation function. A deeper theoretical investigation could identify the precise conditions under which TeLU guarantees faster and more reliable convergence, broadening its utility across different neural network architectures and aiding researchers in optimizing model performance more efficiently.

TeLU’s role as an analytic universal approximator makes it a promising candidate for higher-order optimization methods. In future work, we will explore its performance with techniques like Newton’s Method [99, 110, 113], Hessian-Free Optimization [79], Natural Gradient Descent [1, 104], and Trust Region methods [18, 54, 98]. These methods offer improved stability and convergence efficiency, so we will investigate how TeLU complements their properties.

In addition, we will extend our research by integrating TeLU with bio-inspired learning algorithms, such as predictive coding networks [82], which leverage higher-order curvature information through quasi-Newton approximations [77]. By incorporating TeLU, we anticipate enhanced stability and faster convergence compared to traditional backpropagation, and our future studies will carefully analyze its performance within these innovative frameworks.

TeLU, as an analytic universal approximator, has a smooth curvature often associated with improved robustness in neural networks [79, 80, 93]. Initial testing has demonstrated promising robustness benefits. To further validate these results, we plan to conduct extensive experiments across challenging datasets and standard benchmarks [44]. This allows smooth activation functions like TeLU to benefit against adversarial attacks [114].

Beyond providing resistance to adversarial attacks, we hypothesize that smooth non-monotonic activation functions enhance security by making it more difficult to reconstruct training images, addressing significant privacy concerns [8]. Their unpredictability disrupts neural network processing patterns, hindering attackers from reverse-engineering or inferring sensitive data. Attack methods like side-channel analysis and bitstream reverse engineering, which rely on predictable behaviors and data correlations, become less effective. This unpredictability strengthens neural network security by reducing the success of traditional attack vectors.

Integrating non-monotonic activation functions thus enhances privacy and bolsters resistance against specific attacks.

Inspired by the success of TeLU, we also intend to experiment with variations of TeLU of the form $x \cdot \tanh(a^x)$, where $a \in [2, 3]$. If $a \in \mathbb{Z}^+$, computational efficiency could be enhanced. When $a + \epsilon = e$, for some small positive ϵ , gradients would decay at an asymptotic rate of $\Theta(\frac{x}{a^x})$, which may help further address the vanishing gradient problem and improve convergence. Conversely, when $a = e + \epsilon$, the function might approximate the identity more closely, potentially enhancing convergence.

To further investigate the computational efficiency of TeLU, we plan to conduct additional experiments using lower-level programming languages like C and C+. Implementing TeLU in these languages may allow for a more precise evaluation of its computational efficiency. We also intend to explore optimizations of TeLU itself, refining its algorithms to improve speed and resource utilization. Additionally, by involving distinct hardware accelerators such as TPUs [5] and FPGAs [102], we aim to assess TeLU’s performance across various architectures. These efforts are motivated by our desire to maximize TeLU’s efficiency and scalability, ensuring it can be effectively utilized in a wide range of real-world applications.

References

- [1] S. Amari and S.C. Douglas. “Why natural gradient?” In: *Proceedings of the 1998 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP '98 (Cat. No.98CH36181)*. Vol. 2. 1998, 1213–1216 vol.2. DOI: 10.1109/ICASSP.1998.675489.
- [2] Thomas E. Anderson et al. “Scheduler activations: effective kernel support for the user-level management of parallelism”. In: *SIGOPS Oper. Syst. Rev.* 25.5 (Sept. 1991), pp. 95–109. ISSN: 0163-5980. DOI: 10.1145/121133.121151. URL: <https://doi.org/10.1145/121133.121151>.
- [3] Andrea Apicella et al. “A survey on modern trainable activation functions”. In: *Neural Networks* 138 (2021), pp. 14–32. ISSN: 0893-6080. DOI: <https://doi.org/10.1016/j.neunet.2021.01.026>. URL: <https://www.sciencedirect.com/science/article/pii/S0893608021000344>.
- [4] Frank Arute et al. “Quantum supremacy using a programmable superconducting processor”. In: *Nature* 574.7779 (2019), pp. 505–510.
- [5] Ahmad Ammar Asyraaf Jainuddin et al. “Performance Analysis of Deep Neural Networks for Object Classification with Edge TPU”. In: *2020 8th International Conference on Information Technology and Multimedia (ICIMU)*. 2020, pp. 323–328. DOI: 10.1109/ICIMU49871.2020.9243367.
- [6] Philip Bachman, Ouais Alsharif, and Doina Precup. *Learning with Pseudo-Ensembles*. 2014. arXiv: 1412.4864. URL: <https://arxiv.org/abs/1412.4864>.
- [7] Lalit R. Bahl, Frederick Jelinek, and Robert L. Mercer. “A Maximum Likelihood Approach to Continuous Speech Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* PAMI-5.2 (1983), pp. 179–190. DOI: 10.1109/TPAMI.1983.4767370.
- [8] Borja Balle, Giovanni Cherubin, and Jamie Hayes. “Reconstructing Training Data with Informed Adversaries”. In: *CoRR* abs/2201.04845 (2022). arXiv: 2201.04845. URL: <https://arxiv.org/abs/2201.04845>.
- [9] S. Becker and Yann Lecun. “Improving the convergence of back-propagation learning with second-order methods”. English (US). In: *Proceedings of the 1988 Connectionist Models Summer School, San Mateo*. Ed. by D. Touretzky, G. Hinton, and T. Sejnowski. Morgan Kaufmann, 1989, pp. 29–37.

- [10] Y. Bengio, P. Simard, and P. Frasconi. “Learning long-term dependencies with gradient descent is difficult”. In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166. DOI: 10.1109/72.279181.
- [11] *Bio-inspired Neurocomputing*. Springer Singapore, 2021. ISBN: 9789811554957. DOI: 10.1007/978-981-15-5495-7. URL: <http://dx.doi.org/10.1007/978-981-15-5495-7>.
- [12] *Bio-inspired Neurocomputing*. Springer Singapore, 2021. ISBN: 9789811554957. DOI: 10.1007/978-981-15-5495-7. URL: <http://dx.doi.org/10.1007/978-981-15-5495-7>.
- [13] Han Cai et al. “Reinforcement Learning for Architecture Search by Network Transformation”. In: *CoRR* abs/1707.04873 (2017). arXiv: 1707.04873. URL: <http://arxiv.org/abs/1707.04873>.
- [14] Augustin-Louis Cauchy. *Cours d’Analyse de l’École Royale Polytechnique*. Debure, 1821.
- [15] Jack Choquette et al. “NVIDIA A100 Tensor Core GPU: Performance and Innovation”. In: *IEEE Micro* 41.2 (2021), pp. 29–35. DOI: 10.1109/MM.2021.3061394.
- [16] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. “Fast and accurate deep network learning by exponential linear units (elus)”. In: *arXiv preprint arXiv:1511.07289* (2015).
- [17] Yann Le Cun, Ido Kanter, and Sara A. Solla. “Eigenvalues of covariance matrices: Application to neural-network learning”. In: *Phys. Rev. Lett.* 66 (18 May 1991), pp. 2396–2399. DOI: 10.1103/PhysRevLett.66.2396. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.66.2396>.
- [18] Frank E. Curtis, Zachary Lubberts, and Daniel P. Robinson. *Concise Complexity Analyses for Trust-Region Methods*. 2018. arXiv: 1802.07843 [math.OC]. URL: <https://arxiv.org/abs/1802.07843>.
- [19] George V. Cybenko. “Approximation by superpositions of a sigmoidal function”. In: *Mathematics of Control, Signals and Systems* 2 (1989), pp. 303–314. URL: <https://api.semanticscholar.org/CorpusID:3958369>.
- [20] Ali Dasdan. “Eleven Simple Algorithms to Compute Fibonacci Numbers”. In: *CoRR* abs/1803.07199 (2018). arXiv: 1803.07199. URL: <http://arxiv.org/abs/1803.07199>.
- [21] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848.
- [22] Li Deng. “The mnist database of handwritten digit images for machine learning research”. In: *IEEE Signal Processing Magazine* 29.6 (2012), pp. 141–142.
- [23] Shiv Ram Dubey, Satish Kumar Singh, and Bidyut Baran Chaudhuri. “Activation functions in deep learning: A comprehensive survey and benchmark”. In: *Neurocomputing* 503 (2022), pp. 92–108. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2022.06.111>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231222008426>.

- [24] Jeffrey L. Elman. “Finding structure in time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211. ISSN: 0364-0213. DOI: [https://doi.org/10.1016/0364-0213\(90\)90002-E](https://doi.org/10.1016/0364-0213(90)90002-E). URL: <https://www.sciencedirect.com/science/article/pii/036402139090002E>.
- [25] Rebecca Emanuel et al. “The effect of activation functions on accuracy, convergence speed, and misclassification confidence in CNN text classification: a comprehensive exploration”. In: *The Journal of Supercomputing* 80 (June 2023), pp. 1–21. DOI: 10.1007/s11227-023-05441-7.
- [26] Alfredo Fernandez and Ankur Mali. *Stable and Robust Deep Learning By Hyperbolic Tangent Exponential Linear Unit (TeLU)*. 2024. arXiv: 2402.02790 [cs.LG]. URL: <https://arxiv.org/abs/2402.02790>.
- [27] Ronald Aylmer Fisher. “Theory of statistical estimation”. In: *Mathematical proceedings of the Cambridge philosophical society*. Vol. 22. 5. Cambridge University Press. 1925, pp. 700–725.
- [28] Pascal Fua and Krzysztof Lis. “Comparing Python, Go, and C++ on the N-Queens Problem”. In: *CoRR* abs/2001.02491 (2020). arXiv: 2001.02491. URL: <http://arxiv.org/abs/2001.02491>.
- [29] Kunihiko Fukushima. “Cognitron: A self-organizing multilayered neural network”. In: *Biological cybernetics* 20.3-4 (1975), pp. 121–136.
- [30] Ken-Ichi Funahashi. “On the approximate realization of continuous mappings by neural networks”. In: *Neural Networks* 2.3 (1989), pp. 183–192. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(89\)90003-8](https://doi.org/10.1016/0893-6080(89)90003-8). URL: <https://www.sciencedirect.com/science/article/pii/0893608089900038>.
- [31] Mudasir Ahmad Ganaie et al. “Ensemble deep learning: A review”. In: *CoRR* abs/2104.02395 (2021). arXiv: 2104.02395. URL: <https://arxiv.org/abs/2104.02395>.
- [32] Amir Gholami et al. “SqueezeNext: Hardware-Aware Neural Network Design”. In: *CoRR* abs/1803.10615 (2018). arXiv: 1803.10615. URL: <http://arxiv.org/abs/1803.10615>.
- [33] Xavier Glorot and Yoshua Bengio. “Understanding the difficulty of training deep feedforward neural networks”. In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Yee Whye Teh and Mike Titterton. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, May 2010, pp. 249–256. URL: <https://proceedings.mlr.press/v9/glorot10a.html>.
- [34] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep Sparse Rectifier Neural Networks”. In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Geoffrey Gordon, David Dunson, and Miroslav Dudík. Vol. 15. Proceedings of Machine Learning Research. Fort Lauderdale, FL, USA: PMLR, Apr. 2011, pp. 315–323. URL: <https://proceedings.mlr.press/v15/glorot11a.html>.

- [35] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [36] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572. URL: <https://arxiv.org/abs/1412.6572>.
- [37] Ian J. Goodfellow et al. *Maxout Networks*. 2013. arXiv: 1302.4389. URL: <https://arxiv.org/abs/1302.4389>.
- [38] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. “Speech recognition with deep recurrent neural networks”. In: *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. 2013, pp. 6645–6649. DOI: 10.1109/ICASSP.2013.6638947.
- [39] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. 3rd. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011. ISBN: 0123814790.
- [40] Moritz Hardt, Benjamin Recht, and Yoram Singer. *Train faster, generalize better: Stability of stochastic gradient descent*. 2016. arXiv: 1509.01240. URL: <https://arxiv.org/abs/1509.01240>.
- [41] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *CoRR* abs/1512.03385 (2015). arXiv: 1512.03385. URL: <http://arxiv.org/abs/1512.03385>.
- [42] Kaiming He et al. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *2015 IEEE International Conference on Computer Vision (ICCV)*. 2015, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.
- [43] Kaiming He et al. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification”. In: *CoRR* abs/1502.01852 (2015). arXiv: 1502.01852. URL: <http://arxiv.org/abs/1502.01852>.
- [44] Dan Hendrycks and Thomas Dietterich. *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*. 2019. arXiv: 1903.12261. URL: <https://arxiv.org/abs/1903.12261>.
- [45] Dan Hendrycks and Kevin Gimpel. “Bridging Nonlinearities and Stochastic Regularizers with Gaussian Error Linear Units”. In: *CoRR* abs/1606.08415 (2016). arXiv: 1606.08415. URL: <http://arxiv.org/abs/1606.08415>.
- [46] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Second. Society for Industrial and Applied Mathematics, 2002. DOI: 10.1137/1.9780898718027. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9780898718027>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>.

- [47] G. E. Hinton and R. R. Salakhutdinov. “Reducing the Dimensionality of Data with Neural Networks”. In: *Science* 313.5786 (2006), pp. 504–507. DOI: 10.1126/science.1127647. eprint: <https://www.science.org/doi/pdf/10.1126/science.1127647>. URL: <https://www.science.org/doi/abs/10.1126/science.1127647>.
- [48] Sepp Hochreiter. “The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions”. In: *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 06.02 (1998), pp. 107–116. DOI: 10.1142/S0218488598000094. eprint: <https://doi.org/10.1142/S0218488598000094>. URL: <https://doi.org/10.1142/S0218488598000094>.
- [49] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8). URL: <https://www.sciencedirect.com/science/article/pii/0893608089900208>.
- [50] Le Hou et al. “Neural Networks with Smooth Adaptive Activation Functions for Regression”. In: *CoRR* abs/1608.06557 (2016). arXiv: 1608.06557. URL: <http://arxiv.org/abs/1608.06557>.
- [51] Yutong Hu et al. *Can Perplexity Reflect Large Language Model’s Ability in Long Text Understanding?* 2024. arXiv: 2405.06105 [cs.CL]. URL: <https://arxiv.org/abs/2405.06105>.
- [52] Gao Huang et al. *Densely Connected Convolutional Networks*. 2018. arXiv: 1608.06993. URL: <https://arxiv.org/abs/1608.06993>.
- [53] Sergey Ioffe and Christian Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. 2015. arXiv: 1502.03167. URL: <https://arxiv.org/abs/1502.03167>.
- [54] Yuntian Jiang et al. *A Universal Trust-Region Method for Convex and Nonconvex Optimization*. 2024. arXiv: 2311.11489 [math.OA]. URL: <https://arxiv.org/abs/2311.11489>.
- [55] Norman P. Jouppi et al. “In-Datacenter Performance Analysis of a Tensor Processing Unit”. In: *CoRR* abs/1704.04760 (2017). arXiv: 1704.04760. URL: <http://arxiv.org/abs/1704.04760>.
- [56] Sarit Khirirat, Hamid Reza Feyzmahdavian, and Mikael Johansson. “Mini-batch gradient descent: Faster convergence under data sparsity”. In: *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. 2017, pp. 2880–2887. DOI: 10.1109/CDC.2017.8264077.
- [57] Jack Kiefer and Jacob Wolfowitz. “Stochastic estimation of the maximum of a regression function”. In: *The Annals of Mathematical Statistics* (1952), pp. 462–466.
- [58] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114. URL: <https://arxiv.org/abs/1312.6114>.

- [59] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.
- [60] Alex Krizhevsky, Geoffrey Hinton, et al. “Learning multiple layers of features from tiny images”. In: (2009).
- [61] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. In: *Commun. ACM* 60.6 (May 2017), pp. 84–90. ISSN: 0001-0782. DOI: 10.1145/3065386. URL: <https://doi.org/10.1145/3065386>.
- [62] S. Kullback and R. A. Leibler. “On Information and Sufficiency”. In: *The Annals of Mathematical Statistics* 22.1 (1951), pp. 79–86. ISSN: 00034851. URL: <http://www.jstor.org/stable/2236703> (visited on 09/23/2024).
- [63] Y. Lecun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.
- [64] Y. Lecun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10.1109/5.726791.
- [65] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. “Deep learning”. In: *nature* 521.7553 (2015), p. 436.
- [66] Yann LeCun, Ido Kanter, and Sara A. Solla. “Eigenvalues of covariance matrices: Application to neural-network learning”. In: *Phys. Rev. Lett.* 66 (18 May 1991), pp. 2396–2399. DOI: 10.1103/PhysRevLett.66.2396. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.66.2396>.
- [67] Yann LeCun et al. “Efficient BackProp”. In: *Neural Networks: Tricks of the Trade*. Ed. by Genevieve B. Orr and Klaus-Robert Müller. Berlin, Heidelberg: Springer Berlin Heidelberg, 1998, pp. 9–50. ISBN: 978-3-540-49430-0. DOI: 10.1007/3-540-49430-8_2. URL: https://doi.org/10.1007/3-540-49430-8_2.
- [68] Juri Lelli et al. “An experimental comparison of different real-time schedulers on multicore systems”. In: *Journal of Systems and Software* 85.10 (2012). Automated Software Evolution, pp. 2405–2416. ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2012.05.048>. URL: <https://www.sciencedirect.com/science/article/pii/S016412121200146X>.
- [69] Moshe Leshno et al. “Multilayer feedforward networks with a nonpolynomial activation function can approximate any function”. In: *Neural Networks* 6.6 (1993), pp. 861–867. ISSN: 0893-6080. DOI: [https://doi.org/10.1016/S0893-6080\(05\)80131-5](https://doi.org/10.1016/S0893-6080(05)80131-5). URL: <https://www.sciencedirect.com/science/article/pii/S0893608005801315>.

- [70] Si Yong Lim and King Hann Lim. “Second-order Derivative Optimization Methods in Deep Learning Neural Networks”. In: *2022 International Conference on Green Energy, Computing and Sustainable Technology (GECOST)*. 2022, pp. 470–475. DOI: 10.1109/GECOST55694.2022.10010399.
- [71] Yanli Liu, Yuan Gao, and Wotao Yin. “An improved analysis of stochastic gradient descent with momentum”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 18261–18271.
- [72] Ilya Loshchilov and Frank Hutter. “Fixing Weight Decay Regularization in Adam”. In: *CoRR* (2017). arXiv: 1711.05101. URL: <http://arxiv.org/abs/1711.05101>.
- [73] Ilya Loshchilov and Frank Hutter. “SGDR: Stochastic Gradient Descent with Restarts”. In: *CoRR* abs/1608.03983 (2016). arXiv: 1608.03983. URL: <http://arxiv.org/abs/1608.03983>.
- [74] Lu Lu et al. “Dying relu and initialization: Theory and numerical examples”. In: *arXiv preprint arXiv:1903.06733* (2019).
- [75] David Luebke and M Harris. “General-purpose computation on graphics hardware”. In: *Workshop, SIGGRAPH*. Vol. 33. 2004, p. 6.
- [76] Andrew L Maas, Awni Y Hannun, Andrew Y Ng, et al. “Rectifier nonlinearities improve neural network acoustic models”. In: *Proc. icml*. Vol. 30. 1. Atlanta, GA. 2013, p. 3.
- [77] Ankur Mali, Tommaso Salvatori, and Alexander Ororbia. *Tight Stability, Convergence, and Robustness Bounds for Predictive Coding Networks*. 2024. arXiv: 2410.04708 [cs.LG]. URL: <https://arxiv.org/abs/2410.04708>.
- [78] Mitchell P. Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. “Building a Large Annotated Corpus of English: The Penn Treebank”. In: *Computational Linguistics* 19.2 (1993). Ed. by Julia Hirschberg, pp. 313–330. URL: <https://aclanthology.org/J93-2004>.
- [79] James Martens. “Deep learning via Hessian-free optimization”. In: *Proceedings of the 27th International Conference on International Conference on Machine Learning*. ICML’10. Haifa, Israel: Omnipress, 2010, pp. 735–742. ISBN: 9781605589077.
- [80] James Martens and Roger Grosse. *Optimizing Neural Networks with Kronecker-factored Approximate Curvature*. 2020. arXiv: 1503.05671. URL: <https://arxiv.org/abs/1503.05671>.
- [81] Warren S. McCulloch and Walter Pitts. “A logical calculus of the ideas immanent in nervous activity”. In: *Bulletin of Mathematical Biology* 52 (1990), pp. 99–115. URL: <https://api.semanticscholar.org/CorpusID:15619658>.

- [82] Beren Millidge, Anil K. Seth, and Christopher L. Buckley. “Predictive Coding: a Theoretical and Experimental Review”. In: *CoRR* abs/2107.12979 (2021). arXiv: 2107.12979. URL: <https://arxiv.org/abs/2107.12979>.
- [83] Diganta Misra. “Mish: A Self Regularized Non-Monotonic Neural Activation Function”. In: *CoRR* abs/1908.08681 (2019). arXiv: 1908.08681. URL: <http://arxiv.org/abs/1908.08681>.
- [84] Piotr Nawrot et al. “Efficient Transformers with Dynamic Token Pooling”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2023. DOI: 10.18653/v1/2023.acl-long.353. URL: <http://dx.doi.org/10.18653/v1/2023.acl-long.353>.
- [85] Levon Nurbekyan, Wanzhou Lei, and Yunan Yang. *Efficient Natural Gradient Descent Methods for Large-Scale PDE-Based Optimization Problems*. 2023. arXiv: 2202.06236. URL: <https://arxiv.org/abs/2202.06236>.
- [86] Chigozie Nwankpa et al. “Activation Functions: Comparison of trends in Practice and Research for Deep Learning”. In: (Dec. 2020). DOI: 10.48550/arXiv.1811.03378.
- [87] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *CoRR* abs/1912.01703 (2019). arXiv: 1912.01703. URL: <http://arxiv.org/abs/1912.01703>.
- [88] W.H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd ed. Cambridge University Press, 2007. ISBN: 9780521880688. URL: <http://nr.com/>.
- [89] Tapani Raiko, Harri Valpola, and Yann LeCun. “Deep learning made easier by linear transformations in perceptrons”. In: *Artificial intelligence and statistics*. PMLR. 2012, pp. 924–932.
- [90] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. “Searching for Activation Functions”. In: *CoRR* abs/1710.05941 (2017). arXiv: 1710.05941. URL: <http://arxiv.org/abs/1710.05941>.
- [91] Prajit Ramachandran, Barret Zoph, and Quoc V. Le. “Swish: a Self-Gated Activation Function”. In: *arXiv: Neural and Evolutionary Computing* (2017). URL: <https://api.semanticscholar.org/CorpusID:196158220>.
- [92] Herbert E. Robbins. “A Stochastic Approximation Method”. In: *Annals of Mathematical Statistics* 22 (1951), pp. 400–407. URL: <https://api.semanticscholar.org/CorpusID:16945044>.
- [93] Mihaela Rosca et al. *A case for new neural network smoothness constraints*. 2021. arXiv: 2012.07969 [stat.ML]. URL: <https://arxiv.org/abs/2012.07969>.

- [94] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. “Learning internal representations by error propagation”. In: *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Vol. 1: Foundations*. Cambridge, MA, USA: MIT Press, 1986, pp. 318–362. ISBN: 026268053X.
- [95] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986), pp. 533–536. URL: <https://api.semanticscholar.org/CorpusID:205001834>.
- [96] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986), pp. 533–536. URL: <https://api.semanticscholar.org/CorpusID:205001834>.
- [97] Nicol N. Schraudolph. “Centering Neural Network Gradient Factors”. In: *Neural Networks: Tricks of the Trade: Second Edition*. Ed. by Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 205–223. ISBN: 978-3-642-35289-8. DOI: 10.1007/978-3-642-35289-8_14. URL: https://doi.org/10.1007/978-3-642-35289-8_14.
- [98] John Schulman et al. *Trust Region Policy Optimization*. 2017. arXiv: 1502.05477 [cs.LG]. URL: <https://arxiv.org/abs/1502.05477>.
- [99] Mrinmay Sen and C Krishna Mohan. “NOAH: Newton Method of Optimization with Approximated Hessian”. In: *2023 IEEE International Conference on High Performance Computing*. 2023, pp. 355–360. DOI: 10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00056.
- [100] Shai Shalev-Shwartz and Shai Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.
- [101] Sagar Sharma, Simone Sharma, and Anidhya Athaiya. “Activation functions in neural networks”. In: *Towards Data Sci* 6.12 (2017), pp. 310–316.
- [102] Ahmad Shawahna, Sadiq M. Sait, and Aiman El-Maleh. “FPGA-Based Accelerators of Deep Learning Networks for Learning and Classification: A Review”. In: *IEEE Access* 7 (2019), pp. 7823–7859. DOI: 10.1109/ACCESS.2018.2890150.
- [103] Alex Sherstinsky. “Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network”. In: *Physica D: Nonlinear Phenomena* 404 (2020), p. 132306.
- [104] Rajesh Shrestha. *Natural Gradient Methods: Perspectives, Efficient-Scalable Approximations, and Analysis*. 2023. arXiv: 2303.05473. URL: <https://arxiv.org/abs/2303.05473>.

- [105] Nitish Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56 (2014), pp. 1929–1958. URL: <http://jmlr.org/papers/v15/srivastava14a.html>.
- [106] Tomasz Szandala. “Review and Comparison of Commonly Used Activation Functions for Deep Neural Networks”. In: *CoRR* abs/2010.09458 (2020). arXiv: 2010.09458. URL: <https://arxiv.org/abs/2010.09458>.
- [107] Christian Szegedy et al. *Intriguing properties of neural networks*. 2014. arXiv: 1312.6199. URL: <https://arxiv.org/abs/1312.6199>.
- [108] Aditya Singh Tomar et al. “A Comparative Analysis of Activation Function, Evaluating their Accuracy and Efficiency when Applied to Miscellaneous Datasets”. In: *2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*. 2023, pp. 1035–1042. DOI: 10.1109/ICAAIC56838.2023.10140823.
- [109] Ashish Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems* 30 (2017).
- [110] Chien-Chih Wang, Kent Loong Tan, and Chih-Jen Lin. *Newton Methods for Convolutional Neural Networks*. 2018. arXiv: 1811.06100 [stat.ML]. URL: <https://arxiv.org/abs/1811.06100>.
- [111] Xueliang Wang, Hong Ren, and Achuan Wang. “Smish: A Novel Activation Function for Deep Learning Methods”. In: *Electronics* 11.4 (2022). ISSN: 2079-9292. DOI: 10.3390/electronics11040540. URL: <https://www.mdpi.com/2079-9292/11/4/540>.
- [112] P.J. Werbos. “Backpropagation through time: what it does and how to do it”. In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560. DOI: 10.1109/5.58337.
- [113] Pierre Wolinski. *Adapting Newton’s Method to Neural Networks through a Summary of Higher-Order Derivatives*. 2024. arXiv: 2312.03885 [cs.LG]. URL: <https://arxiv.org/abs/2312.03885>.
- [114] Cihang Xie et al. *Smooth Adversarial Training*. 2021. arXiv: 2006.14536. URL: <https://arxiv.org/abs/2006.14536>.
- [115] Bing Xu et al. *Empirical Evaluation of Rectified Activations in Convolutional Network*. 2015. arXiv: 1505.00853. URL: <https://arxiv.org/abs/1505.00853>.
- [116] Bing Xu et al. *Empirical Evaluation of Rectified Activations in Convolutional Network*. 2015. arXiv: 1505.00853. URL: <https://arxiv.org/abs/1505.00853>.
- [117] Chiyuan Zhang et al. “Understanding deep learning requires rethinking generalization”. In: *CoRR* abs/1611.03530 (2016). arXiv: 1611.03530. URL: <http://arxiv.org/abs/1611.03530>.

- [118] Hao Zheng et al. “Improving deep neural networks using softplus units”. In: *2015 International Joint Conference on Neural Networks (IJCNN)*. 2015, pp. 1–4. DOI: 10.1109/IJCNN.2015.7280459.
- [119] Pan Zhou et al. “Towards Theoretically Understanding Why SGD Generalizes Better Than ADAM in Deep Learning”. In: *CoRR* abs/2010.05627 (2020). arXiv: 2010.05627. URL: <https://arxiv.org/abs/2010.05627>.
- [120] Hegui Zhu et al. “Logish: A new nonlinear nonmonotonic activation function for convolutional neural network”. In: *Neurocomputing* 458 (2021), pp. 490–499. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2021.06.067>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231221009917>.

Appendix A: Supplementary Tables

This Appendix chapter provides supplementary tables that offer additional insights into our theoretical analysis and experimentation. These tables serve as an additional resource for readers seeking a deeper understanding of the theoretical analysis and methodologies discussed in the main body of this work.

A.1 Computational Efficiency of Derivatives

Table A.1.1 presents a breakdown of the various mathematical components involved in computing the first derivative of each activation function examined in this study. Specifically, it quantifies the number of piecewise segments, nonlinear operations, arithmetic computations, and constant terms required. This analysis serves as a practical measure of computational complexity, providing insights into how each activation function’s derivative might impact processing time and resource utilization during model training. By comparing these counts, we can better understand the efficiency trade-offs between different activation functions, highlighting the advantages of the proposed function in terms of simplicity and speed.

Table A.1.1: Activation Function Derivative Computational Efficiency. First derivatives of activation functions computational complexity heuristics: Counting instances of unique non-linearity computations, arithmetic operations, and constant terms

Function	Piecewise	Nonlinearity	Arithmetic	Constants
TeLU	0	3	4	0
ReLU	1	0	0	2
ELU	1	1	0	1
SiLU	0	1	6	2
GELU	0	2	8	7
Mish	0	3	16	5
Logish	0	3	12	3
Smish	0	3	16	10

Table A.2.1: Normalization Experimental Specifications. Mean and standard deviation normalization specifications for CIFAR-10, CIFAR-100, and TinyImageNet datasets.

Optimizer	Red	Green	Blue
CIFAR-10	$\mu=0.4914, \sigma=0.2023$	$\mu=0.4822, \sigma=0.1994$	$\mu=0.4465, \sigma=0.2010$
CIFAR-100	$\mu=0.5071, \sigma=0.2675$	$\mu=0.4867, \sigma=0.2565$	$\mu=0.4408, \sigma=0.2761$
TinyImageNet	$\mu=0.485, \sigma=0.229$	$\mu=0.456, \sigma=0.224$	$\mu=0.406, \sigma=0.225$

A.2 Computation Efficiency of Derivatives

Table A.2.1 outlines the standard score normalization applied in experiments with the CIFAR-10, CIFAR-100, and TinyImageNet datasets. For each dataset, the mean and standard deviation are calculated and utilized to normalize each input sample, ensuring consistency and comparability across the experiments. This information is provided to assist anyone attempting to recreate the experiments. By including the computed means and standard deviations for each dataset, we ensure that the standard score normalization process can be accurately replicated, maintaining the integrity and consistency of the experimental conditions. This level of detail is crucial for achieving comparable results and validating the findings of this study.

Appendix B: Supplementary Theorems

This appendix chapter presents supplementary theorems that provide a more detailed and rigorous foundation for the theories proposed in this study. These additional theorems serve to strengthen the theoretical framework, offering deeper insights and comprehensive support for our arguments. By including these formal mathematical statements and proofs, we aim to enhance the clarity and robustness of our theoretical analysis, giving readers a more complete understanding of the underlying principles that drive our findings.

B.1 Close Approximation to ReLU Non-Linearity with TeLU

Lemma B.1.1. *Let $r(x) = \text{ReLU}(x) = \max(0, x)$, and define the active subdomain as $\mathcal{A} = [0, \infty)$ and the inactive subdomain as $\mathcal{I} = (-\infty, 0)$. Consider the following activation functions:*

1. *TeLU: $t(x) = x \cdot \tanh(e^x)$,*
2. *GELU: $g(x) = x \cdot \Phi(x)$, where $\Phi(x)$ is the Gaussian CDF.*

Define the gradient magnitudes over the active and inactive subdomains as:

$$G_{\mathcal{A}}(f) = \int_0^{\infty} |f'(x)| \, dx, \quad G_{\mathcal{I}}(f) = \int_{-\infty}^0 |f'(x)| \, dx.$$

If $G_{\mathcal{A}}(t) > G_{\mathcal{I}}(g)$, then TeLU has a stronger impact on training dynamics in the positive subdomain than GELU has in the negative subdomain. Consequently, neural networks utilizing TeLU more closely resemble ReLU and exhibit stronger positive-side sensitivity.

Proof. To quantify the gradient behavior of $t(x)$ and $g(x)$ in their respective subdomains, we first compute the derivatives:

1. **Derivative of TeLU in the Active Subdomain** The TeLU function is defined as:

$$t(x) = x \cdot \tanh(e^x).$$

Taking the derivative:

$$t'(x) = \tanh(e^x) + x \cdot e^x \cdot \text{sech}^2(e^x).$$

As $x \rightarrow \infty$, $\tanh(e^x) \approx 1$ and $\text{sech}^2(e^x) \approx 0$. Thus, for large x , we have:

$$t'(x) \approx 1.$$

This shows that TeLU's derivative asymptotically approaches ReLU's derivative for $x > 0$.

The gradient magnitude in the active subdomain is:

$$G_{\mathcal{A}}(t) = \int_0^\infty |\tanh(e^x) + x \cdot e^x \cdot \text{sech}^2(e^x)| dx.$$

Since $\tanh(e^x) \approx 1$ and the second term vanishes as x increases, the integral converges to a large value, indicating strong gradient influence in the active subdomain.

2. Derivative of GELU in the Inactive Subdomain The GELU function is defined as:

$$g(x) = x \cdot \Phi(x),$$

where $\Phi(x) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$ is the Gaussian CDF. Taking the derivative:

$$g'(x) = \Phi(x) + x \cdot \phi(x),$$

where $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ is the Gaussian PDF.

For $x < 0$, $\Phi(x) \approx 0$ and $\phi(x)$ rapidly decays to 0 as $x \rightarrow -\infty$. Thus, for large negative x , we have:

$$g'(x) \approx 0.$$

The gradient magnitude in the inactive subdomain is:

$$G_{\mathcal{I}}(g) = \int_{-\infty}^0 |\Phi(x) + x \cdot \phi(x)| dx.$$

Since $\Phi(x) \approx 0$ and $\phi(x) \approx 0$ for $x < 0$, this integral converges to a small value, indicating weak gradient influence in the inactive subdomain.

3. Gradient Magnitude Comparison Comparing $G_{\mathcal{A}}(t)$ and $G_{\mathcal{I}}(g)$:

$$G_{\mathcal{A}}(t) \approx \int_0^\infty 1 dx = \infty, \quad G_{\mathcal{I}}(g) \approx \int_{-\infty}^0 0 dx = 0.$$

This shows that TeLU has a significantly larger gradient magnitude in the active subdomain compared to GELU's gradient in the inactive subdomain.

Conclusion Since the gradient strength of TeLU in $\mathcal{A}$ far exceeds that of GELU in $\mathcal{I}$, TeLU will have a stronger impact on the learning dynamics of a neural network. Therefore, neural networks using TeLU will be more sensitive and responsive in the positive region, closely mimicking ReLU's behavior in practice.

□

B.2 Zero-Centering of TeLU

Before proving that TeLU has better zero-centering capability than ReLU, we will provide the following lemma to support our construction.

Lemma B.2.1. *If $f(x) \geq g(x)$ for all $x \in \mathbb{R}$ and $f(x) > g(x)$ for some defined interval $[a, b]$, where $a, b \in \mathbb{R}$ and $a < b$, then $\int_{-\infty}^{\infty} f(x)dx > \int_{-\infty}^{\infty} g(x)dx$.*

Proof. We can expand $\int_{-\infty}^{\infty} f(x)dx$ as follows:

$$\int_{-\infty}^{\infty} f(x)dx = \int_{-\infty}^a f(x)dx + \int_a^b f(x)dx + \int_b^{\infty} f(x)dx$$

Similarly, we can also expand $\int_{-\infty}^{\infty} g(x)dx$ as follows:

$$\int_{-\infty}^{\infty} g(x)dx = \int_{-\infty}^a g(x)dx + \int_a^b g(x)dx + \int_b^{\infty} g(x)dx$$

Next, we define a condition necessary to prove as follows:

$$\begin{aligned} \int_{-\infty}^{\infty} f(x)dx &> \int_{-\infty}^{\infty} g(x)dx \\ \int_{-\infty}^{\infty} f(x)dx - \int_{-\infty}^{\infty} g(x)dx &> 0 \\ \left(\int_{-\infty}^a f(x)dx - \int_{-\infty}^a g(x)dx \right) + \left(\int_a^b f(x)dx - \int_a^b g(x)dx \right) + \left(\int_b^{\infty} f(x)dx - \int_b^{\infty} g(x)dx \right) &> 0 \end{aligned}$$

Let's analyze these three components :

For all $x \in \mathbb{R}$ we get,

$$\begin{aligned} f(x) &\geq g(x) \\ \int_{-\infty}^a f(x) &\geq \int_{-\infty}^a g(x) \\ \int_{-\infty}^a f(x) - \int_{-\infty}^a g(x) &\geq 0 \end{aligned}$$

Similarly, for all $x \in \mathbb{R}$,

$$\begin{aligned} f(x) &\geq g(x) \\ \int_b^\infty f(x) &\geq \int_b^\infty g(x) \\ \int_b^\infty f(x) - \int_b^\infty g(x) &\geq 0 \end{aligned}$$

Finally, for all $x \in \mathbb{R}^+$ we get,

$$\begin{aligned} f(x) &> g(x) \\ \int_b^\infty f(x) &> \int_b^\infty g(x) \\ \int_b^\infty f(x) - \int_b^\infty g(x) &> 0 \\ k &> 0 \end{aligned}$$

where $k \in \mathbb{R}^+$.

These 3 components can be re-written as follows:

$$\begin{aligned} &\left(\int_{-\infty}^a f(x)dx - \int_{-\infty}^a g(x)dx \right) + \left(\int_a^b f(x)dx - \int_a^b g(x)dx \right) + \left(\int_b^\infty f(x)dx - \int_b^\infty g(x)dx \right) \\ &\geq 0 + \int_a^b f(x)dx - \int_a^b g(x)dx + 0 = \int_a^b f(x)dx - \int_a^b g(x)dx \\ &= k > 0 \end{aligned}$$

because $k \in \mathbb{R}^+$

Therefore, we have shown that

$$\int_{-\infty}^\infty f(x)dx > \int_{-\infty}^\infty g(x)dx$$

for any $f(x)$ and $g(x)$ such that $f(x) \geq g(x)$ for any $x \in \mathbb{R}$ and $f(x) > g(x)$ for any $x \in [a, b]$, which concludes our proof. □

Next, we show that TeLU exhibits greater zero-centering of activation than ReLU.

Theorem B.2.1. *If x is a random variable following a Gaussian probability distribution about zero with standard deviation $\sigma \in \mathbb{R}^+$ expressed as PDF $p(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp(\frac{-x^2}{2\sigma^2})$, $TeLU(x) = x \cdot \tanh(e^x)$, $ReLU(x) = \max(0, x)$, $E[TeLU(x)] = \int_{-\infty}^{\infty} p(x) \cdot TeLU(x) dx$, and $E[ReLU(x)] = \int_{-\infty}^{\infty} p(x) \cdot ReLU(x) dx$; then $|E[TeLU(x)]| < |E[ReLU(x)]|$.*

Proof. We now analyze PDF $p(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp(\frac{-x^2}{2\sigma^2})$ to show that it is positive for all $\sigma \in \mathbb{R}^+$, $x \in \mathbb{R}$:

- $\frac{1}{\sigma\sqrt{2\pi}}$ is positive for all $\sigma \in \mathbb{R}^+$
- $\exp(\frac{-x^2}{2\sigma^2})$ is positive for all $\sigma \in \mathbb{R}^+$, $x \in \mathbb{R}$; because the range of $\exp(x)$ is $(0, \infty)$
- $\therefore p(x) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp(\frac{-x^2}{2\sigma^2})$, as it is the product of two positive terms

We now show that $E[t(x)] < E[r(x)]$ for all $\sigma \in \mathbb{R}^+$, $x \in \mathbb{R}$:

We begin by showing following condition that $p(x) \cdot TeLU(x) < p(x) \cdot ReLU(x)$ for all $\sigma \in \mathbb{R}^+$, $x \in \mathbb{R}^+$:

$$p(x) \cdot TeLU(x) < p(x) \cdot ReLU(x)$$

$$TeLU(x) < ReLU(x)$$

$$x \cdot \tanh(e^x) < x \cdot \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

$$\tanh(e^x) < \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

$$\tanh(e^x) < 1$$

Next we show that $p(x) \cdot TeLU(x) = p(x) \cdot ReLU(x)$ when $x = 0$ for all $\sigma \in \mathbb{R}^+$

$$p(x) \cdot TeLU(x) = p(x) \cdot ReLU(x)$$

$$TeLU(x) = ReLU(x)$$

$$TeLU(0) = ReLU(0)$$

$$0 \cdot \tanh(e^0) = 0 \cdot \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

$$0 = 0$$

and next condition we show is that $p(x) \cdot TeLU(x) < p(x) \cdot ReLU(x)$ for all $\sigma \in \mathbb{R}^+$, $x \in \mathbb{R}^+$:

$$p(x) \cdot TeLU(x) < p(x) \cdot ReLU(x)$$

$$TeLU(x) < ReLU(x)$$

$$x \cdot \tanh(e^x) < x \cdot \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

$$\tanh(e^x) > \begin{cases} 0 & x < 0 \\ 1 & x \geq 0 \end{cases}$$

$$\tanh(e^x) > 0$$

Since we have shown that $p(x) \cdot TeLU(x) \leq p(x) \cdot ReLU(x)$ for all $x \in \mathbb{R}$ and $p(x) \cdot TeLU(x) \leq p(x) \cdot ReLU(x)$ for all $x \in \mathbb{R}^+$, support theorem tells us that $\int_{-\infty}^{\infty} p(x) \cdot TeLU(x) dx < \int_{-\infty}^{\infty} p(x) \cdot ReLU(x) dx$.
 $\therefore E[TeLU(x)] < E[ReLU(x)]$.

Now, we show that $E[TeLU(x)]$ is positive for all $\sigma \in \mathbb{R}^+$:

We analyze TeLU(x):

- e^x is positive for all $x \in \mathbb{R}^+$
- since $\tanh(w)$ is positive for all $w \in \mathbb{R}$, $\tanh(e^x)$ is positive for all $x \in \mathbb{R}$

- x multiplies with positive $\tanh(e^x)$, so $\text{sign}(x) = \text{sign}(x \cdot \tanh(e^x))$
- $p(x) \cdot \text{TeLU}(x)$ is positive for all $x \in \mathbb{R}^+$, since $p(x)$ is always positive
- $p(x) \cdot \text{TeLU}(x)$ is 0 when $x = 0$, since multiplying by $x=0$ leads to 0
- $p(x) \cdot \text{TeLU}(x)$ is negative for all $x \in \mathbb{R}^-$, since $p(x)$ is always positive

For $E[t(x)] = \int_{-\infty}^{\infty} p(x) \cdot \text{TeLU}(x) dx = \int_{-\infty}^{\infty} \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(\frac{-x^2}{2\sigma^2}\right) \cdot \text{TeLU}(x) dx$ to be positive for all $\sigma \in \mathbb{R}^+$, we must show that $|p(-\epsilon)\text{TeLU}(-\epsilon)| < p(\epsilon)\text{TeLU}(\epsilon)$ for all $\epsilon \in \mathbb{R}^+$. In other words, we must show that the positive component of $p(x) \cdot \text{TeLU}(x)$ is always a strict upper bound to the absolute value of its negative component, evaluated symmetrically about $x = 0$. $\epsilon = 0$ can be disregarded, as $p(\epsilon) \cdot \text{TeLU}(\epsilon) = p(0) \cdot \text{TeLU}(0) = 0$.

Hence, we show that $|p(-\epsilon) \cdot \text{TeLU}(-\epsilon)| < p(\epsilon) \cdot \text{TeLU}(\epsilon)$ for all $\epsilon \in \mathbb{R}^+$:

$$\begin{aligned}
|p(-\epsilon) \cdot \text{TeLU}(-\epsilon)| &< p(\epsilon) \cdot \text{TeLU}(\epsilon) \\
-p(-\epsilon) \cdot \text{TeLU}(-\epsilon) &< p(\epsilon) \cdot \text{TeLU}(\epsilon) \\
-\text{TeLU}(-\epsilon) &< \text{TeLU}(\epsilon) \\
-(-\epsilon \cdot \tanh(e^{-\epsilon})) &< \epsilon \cdot \tanh(e^{\epsilon}) \\
\epsilon \cdot \tanh(e^{-\epsilon}) &< \epsilon \cdot \tanh(e^{\epsilon}) \\
\tanh(e^{-\epsilon}) &< \tanh(e^{\epsilon}) \\
e^{-\epsilon} &< e^{\epsilon} \\
-\epsilon &< \epsilon
\end{aligned}$$

Which is true for all $\epsilon \in \mathbb{R}^+$.

In summary, we have shown that $0 < E[\text{TeLU}(x)] < E[\text{ReLU}(x)]$ for all $x \in \mathbb{R}$. This implies that $|E[\text{TeLU}(x)]| < |E[\text{ReLU}(x)]|$ for all $x \in \mathbb{R}$. showing that $\text{TeLU}(x)$ exhibits better zero-centering of activation than $\text{ReLU}(x)$ for any standard deviation $\sigma \in \mathbb{R}^+$ given that input x follows a Gaussian distribution of mean $\mu = 0$.

□

B.3 Isolated Zero of TeLU

Let σ be an activation function given as $y = \sigma(x)$, where x is the input and y is the output. Let $\mathcal{F}(\Theta)$ be the set of parameters using the σ non-linearities. Let the function $\{$ be optimized by the objective function $\mathcal{L}(\Theta)$ using standard backpropagation of error, then we show σ applied on any function f avoids vanishing gradients issues in the neural network.

Theorem B.3.1. *Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a function defined by $f(x) = x \cdot \tanh(e^x)$. The derivative of f , $f'(x)$, is given by*

$$f'(x) = x \cdot (1 - \tanh^2(e^x)) \cdot e^x + \tanh(e^x).$$

Then, the set $\{x \in \mathbb{R} \mid f'(x) \neq 0\}$ is dense in $\mathbb{R}$. Moreover, there exists a countable set $\{x_i\}_{i \in \mathbb{N}} \subset \mathbb{R}$ where $f'(x_i) = 0$ for each i . Each point x_i is isolated, in the sense that for each x_i , there exists an $\epsilon_i > 0$ such that if $x \in (x_i - \epsilon_i, x_i + \epsilon_i)$ and $x \neq x_i$, then $f'(x) \neq 0$.

Proof. The derivative of $f(x)$ with respect to x is given by:

$$f'(x) = \frac{d}{dx} (x \cdot \tanh(e^x)).$$

Applying the product rule and the chain rule, we find:

$$f'(x) = \tanh(e^x) + x \cdot (1 - \tanh^2(e^x)) \cdot e^x.$$

We analyze this derivative of above function in two parts:

- $\tanh(e^x)$ is always positive, as e^x is always positive for and $\tanh(z)$ is bounded between 0 and 1 for all positive z .
- $1 - \tanh^2(e^x)$ is always positive since $|\tanh(z)| < 1$ for all z , and e^x is always positive for all real x

Thus, the second term $x \cdot (1 - \tanh^2(e^x)) \cdot e^x$ is always non-zero unless $x = 0$. However, even at $x = 0$, the first term $\tanh(e^x)$ remains non-zero. Therefore, the entire expression for $f'(x)$ is non-zero for all $x \neq x_i$.

Isolated Zeros: For $f'(x) = 0$, we must have $x \cdot \exp(x) \cdot \text{sech}^2(\exp(x)) = -\tanh(\exp(x))$. Given the properties of $\tanh(x)$ and $\exp(x)$, the solutions to this equation are isolated because both sides of the equation represent continuous, differentiable functions with fundamentally different growth rates, ensuring any intersections are isolated points. The above construction is based on functional analysis. However it is important to note that the function $f'(x)$ has no analytical solution, thus the bound of x_i will change based on system precision. For instance, the numerical solution is needed to find the value of x where $f'(x) \approx 0$.

We observe that using the numerical solution (newton-ramphson method) when $x \approx -1.07886$ the function $f'(x) = 4.6 \times 10^{-48}$. However majority of systems cannot handle such high precision and will equate this to be equal to zero. In other words, based on precision, $f'(x)$ will reach zero, thus the bound or range will change based on precision of the system.

Formal Statement

$$\forall x \in \mathbb{R}, \exists \epsilon > 0, \text{ such that if } |f'(x)| < \epsilon, \text{ then } \epsilon \approx 0, \text{ but } \epsilon \neq 0$$

This means for all real numbers x , there exists an ϵ greater than zero (indicating an extremely small magnitude) such that if the absolute value of $f'(x)$ is less than ϵ , then ϵ is approximately zero. This indicates that while $f'(x)$ may approach very close to zero for some values of x , it does not strictly equal zero except possibly under conditions that are negligible for practical purposes.

Density of Non-Zero Derivative: The non-zero values of $f'(x)$ constitute a dense subset of $\mathbb{R}$ since the conditions for $f'(x) = 0$ require a specific balance that is only met at isolated points, as shown above. Between these points, $f'(x)$ maintains non-zero values, ensuring the gradient does not vanish across these intervals.

Hence, we conclude :

$$f'(x) \neq 0 \text{ for all } x \neq x_i \in \mathbb{R}$$

□

B.4 Bounded Saturation and Growth of TeLU

Next, we prove the network's saturating decay and bounded growth , contributing towards stability during training due to the avoidance of the exploding gradient issue.

Theorem B.4.1. *The function $f(x) = x \cdot \tanh(e^x)$ exhibits stable behavior for any neural network.*

Proof. Bounded Output: The hyperbolic tangent function $\tanh(z)$ has outputs bounded between -1 and 1. Therefore, for any real number x , the product $x \cdot \tanh(e^x)$ will not grow unbounded, contributing to stability. Mathematically, this can be expressed as:

$$-|x| \leq f(x) \leq |x|$$

2. *Non-zero Gradient:* The derivative of $f(x)$, given by

$$f'(x) = \tanh(e^x) + x \cdot (1 - \tanh^2(e^x)) \cdot e^x$$

is always non-zero for all real x besides our isolated zero when $x \approx -1.07886$. This ensures that the gradients do not vanish during backpropagation, which is crucial for stable learning in deep networks.

3. *Controlled Growth for Positive x :* As $x \rightarrow \infty$, the function grows linearly since $\tanh(e^x)$ approaches 1. After being scaled by the identity x , TeLU approaches linear growth as $x \rightarrow \infty$. This linear growth is more stable than exponential growth, which could lead to exploding gradients.

4. *Saturating Behavior for Negative x :* As $x \rightarrow -\infty$, x becomes large in the negative direction with linear growth. However, e^x approaches 0 as $x \rightarrow -\infty$ at an exponential rate. $\tanh(z)$, approaching the identity as inputs approach 0, results in $\tanh(e^x)$ maintaining exponential decay towards 0. This negative linear growth and asymptotic decay as $x \rightarrow -\infty$ is thus evaluated by $\lim_{x \rightarrow -\infty} \frac{-x}{e^x} = 0$. The resulting saturation towards 0 helps prevent the function from contributing to exploding gradients during training.

Therefore, due to its bounded output, non-zero gradient besides an isolated zero, controlled growth for positive values, and saturating behavior for negative values, the function $f(x) = x \cdot \tanh(e^x)$ is shown to be stable in the context of neural network activations. $\square$

B.5 Robustness of TeLU

Next, we show TeLU is more robust to small noise and perturbations compared to ReLU, which is an important property for designing adversarial-resistant neural networks.

Theorem B.5.1. *The function $f(x) = x \cdot \tanh(e^x)$ is more robust compared to Relu ($g(x) = \max(0, x)$) and robust against small perturbations or noise in the input.*

Proof. We analyze the derivative of $f(x)$ to show robustness to small perturbations. The derivative gives the rate of change of the function with respect to changes in the input. A small derivative magnitude indicates robustness to small changes or noise in the input. The derivative of $g(x) = \text{Relu}$ is represented as follows:

$$g'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$

This derivative shows that for $x > 0$, the function is sensitive to changes, as even small positive changes in x will result in a change in output. The function is insensitive to changes for $x < 0$, as the output remains zero. The derivative is undefined at $x = 0$, indicating a discontinuity, which can be problematic for stability.

The derivative of $f(x) = \text{TeLU}$ is given by:

$$f'(x) = \tanh(e^x) + x \cdot (1 - \tanh^2(e^x)) \cdot e^x$$

Consider the behavior of $f'(x)$ for different ranges of x :

For large negative x : As x becomes very negative, e^x approaches 0, making $\tanh(e^x)$ and its derivative small. Thus, $f'(x)$ becomes small, indicating that $f(x)$ is not highly sensitive to small changes in x .

For small x around 0: Here, $\tanh(e^x)$ is approximately equal to e^x , which is close to 1 for small x . The term $x \cdot (1 - \tanh^2(e^x)) \cdot e^x$ is also small. Hence, $f'(x)$ remains moderate, suggesting that $f(x)$ does not change drastically for small perturbations around 0.

For large positive x : Although e^x grows, the term $\tanh(e^x)$ approaches 1, limiting the growth of $f(x)$. The term $x \cdot (1 - \tanh^2(e^x)) \cdot e^x$ becomes small as x increases, due to the saturation of $\tanh(e^x)$. Thus, $f'(x)$ remains bounded.

Since $f'(x)$ does not exhibit large values across the range of x , it indicates that $f(x)$ does not change disproportionately for small changes in x , thereby demonstrating robustness to small perturbations or noise.

□

B.6 Lipschitz Continuity of TeLU

Next, we show a strong property which shows TeLU is Lipschitz continuous, which is important to uniform continuity of the function

Theorem B.6.1. *The function $f : \mathbb{R} \rightarrow \mathbb{R}$, defined by $f(x) = x \cdot \tanh(e^x)$, is Lipschitz continuous on the real line $\mathbb{R}$.*

Proof. To demonstrate that f is Lipschitz continuous, we seek a constant L such that for all $x, y \in \mathbb{R}$, the inequality

$$|f(x) - f(y)| \leq L|x - y|$$

is satisfied. A sufficient condition for this is that the derivative of f , $f'(x)$, is bounded on $\mathbb{R}$.

The derivative of f is given by

$$f'(x) = \tanh(e^x) + x \cdot \frac{e^x}{\cosh^2(e^x)}$$

We analyze the boundedness of $f'(x)$ in two parts:

1. The function $\tanh(e^x)$ is bounded on $\mathbb{R}$ as $\tanh$ outputs values in $(-1, 1)$.
2. For the term $x \cdot \frac{e^x}{\cosh^2(e^x)}$, we consider its behavior as x approaches infinity and negative infinity:

$$\lim_{x \rightarrow \infty} \left| x \cdot \frac{e^x}{\cosh^2(e^x)} \right| = 1$$

$$\lim_{x \rightarrow -\infty} \left| x \cdot \frac{e^x}{\cosh^2(e^x)} \right| = 0$$

Since both limits are finite, the term $x \cdot \frac{e^x}{\cosh^2(e^x)}$ is bounded on $\mathbb{R}$.

Combining these findings, we conclude that $|f'(x)|$ is bounded on $\mathbb{R}$. The maximum value of $|f'(x)|$ is 1, therefore we can take $L = 1$ as the Lipschitz constant.

Hence, $f(x) = x \cdot \tanh(e^x)$ is Lipschitz continuous with a Lipschitz constant $L = 1$. $\square$

B.7 Smooth Loss Landscape of TeLU

Next, we show that TeLU has a smoother loss landscape, which leads to faster convergence.

Theorem B.7.1. *Given a neural network $\mathcal{N}$ with activation function $f(x) = x \cdot \tanh(e^x)$, parameters θ , and a differentiable loss function $\mathcal{L}(\theta)$, the Fisher Information Matrix $I(\theta)$ defined as*

$$I(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla_{\theta} \log p(y|x; \theta) \nabla_{\theta} \log p(y|x; \theta)^{\top}]$$

leads to a smoother optimization landscape during training of $\mathcal{N}$.

Proof. Continuity and Differentiability of $f(x)$

The activation function $f(x) = x \cdot \tanh(e^x)$ and its derivative are analyzed:

$$\begin{aligned} f(x) &= x \cdot \tanh(e^x), \\ \text{where } \tanh(u) &= \frac{e^{2u} - 1}{e^{2u} + 1}. \\ \text{Thus, } f'(x) &= \frac{d}{dx} (x \cdot \tanh(e^x)) \\ &= \tanh(e^x) + x \cdot \frac{d}{dx} \tanh(e^x) \\ &= \tanh(e^x) + x \cdot e^x \cdot (1 - \tanh^2(e^x)). \end{aligned}$$

Since $\tanh(u)$ and e^x are continuously differentiable, $f(x)$ and $f'(x)$ are also continuously differentiable.

Impact on Fisher Information Matrix

Applying the chain rule to compute the gradient of the log-likelihood:

$$\begin{aligned}\nabla_{\theta} \log p(y|x; \theta) &= \frac{\partial \log p(y|x; \theta)}{\partial \mathcal{N}} \cdot \frac{\partial \mathcal{N}}{\partial \theta}, \\ &= \text{Gradient of the output w.r.t. the network's parameters.}\end{aligned}$$

The gradient involves terms from $f'(x)$ due to the activation function in each layer:

$$f'(x) = \tanh(e^x) + x \cdot e^x \cdot (1 - \tanh^2(e^x)).$$

Thus, $I(\theta)$ becomes a matrix of expectations of outer products of these gradients:

$$I(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla_{\theta} \log p(y|x; \theta) \nabla_{\theta} \log p(y|x; \theta)^{\top}].$$

The smoothness of $f'(x)$ translates to a smoother $I(\theta)$.

Smoother Optimization Landscape

In gradient descent, parameter updates are governed by:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_{\theta} \mathcal{L}(\theta^{(t)}),$$

where η is the learning rate. The gradient of the loss function $\nabla_{\theta} \mathcal{L}(\theta)$ is influenced by $I(\theta)$. A smoother $I(\theta)$ results in more stable and consistent gradient updates, avoiding erratic steps often observed in rougher optimization landscapes. This leads to enhanced stability in finding the minima of $\mathcal{L}(\theta)$.

Hence, we can show, that the continuously differentiable nature of $f(x) = x \cdot \tanh(e^x)$ and its derivative ensures that the Fisher Information Matrix $I(\theta)$ in the neural network $\mathcal{N}$ promotes a smoother optimization landscape, facilitating more effective training dynamics.

□

B.8 Global Convergence of TeLU

Based on the properties of TeLU, shown in Theorem B.7.1, we can prove the global convergence of the function under certain conditions.

Theorem B.8.1. *Let $\mathcal{N}$ be a neural network employing the activation function $f(x) = x \cdot \tanh(e^x)$ in its architecture. Assume the network parameters are denoted by θ and the network is trained using a differentiable loss function $\mathcal{L}(\theta)$. If $\mathcal{L}(\theta)$ satisfies the Polyak-Łojasiewicz (PL) condition, then the gradient*

descent optimization on $\mathcal{N}$ converges to a global minimum, significantly influenced by the properties of $f(x)$ and its derivative $f'(x)$.

Proof. Smoothness and Boundedness of $f(x)$ and $f'(x)$:

The function $f(x) = x \cdot \tanh(e^x)$ is continuously differentiable. Its derivative, given by

$$f'(x) = \tanh(e^x) + x \cdot e^x \cdot (1 - \tanh^2(e^x)),$$

is also continuously differentiable and bounded due to the inherent properties of the $\tanh$ function and the exponential function. These properties ensure smooth and well-conditioned gradient computations throughout the optimization process.

Influence on Gradient Descent under PL Condition:

Given the PL condition, for a global minimum θ^* , there exists $\mu > 0$ such that

$$2\mu(\mathcal{L}(\theta) - \mathcal{L}(\theta^*)) \leq \|\nabla_{\theta}\mathcal{L}(\theta)\|^2 \text{ for all } \theta.$$

The gradient descent update rule is

$$\theta^{(t+1)} = \theta^{(t)} - \eta \cdot \nabla_{\theta}\mathcal{L}(\theta^{(t)}),$$

where η is the learning rate.

Convergence Analysis:

Utilizing the smoothness and boundedness of $f'(x)$, along with the PL condition, it can be shown that

$$\mathcal{L}(\theta^{(t+1)}) \leq \mathcal{L}(\theta^{(t)}) - \eta \cdot \|\nabla_{\theta}\mathcal{L}(\theta^{(t)})\|^2,$$

which implies

$$\mathcal{L}(\theta^{(t)}) - \mathcal{L}(\theta^*) \leq (1 - 2\mu\eta)^t (\mathcal{L}(\theta^{(0)}) - \mathcal{L}(\theta^*)).$$

Therefore, $\mathcal{L}(\theta^{(t)})$ converges to $\mathcal{L}(\theta^*)$ as $t \rightarrow \infty$.

□